Chapter 4

그리디 알고리즘

연암공과대학교

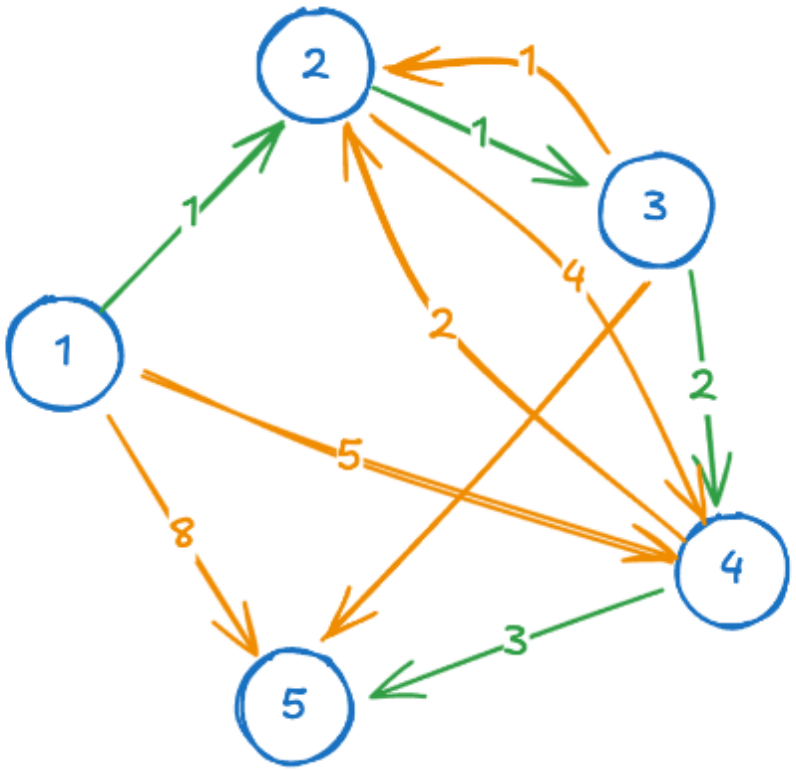
차례

4.1 동전 거스름돈
4.2 최소 신장 트리
4.3 최단 경로 찾기
4.4 부분 배낭 문제
4.5 집합 커버 문제
4.6 작업 스케줄링
4.7 허프만 압축

그리디 (Greedy) 알고리즘

"미래를 보지 않고, 현재에 최선을 다한다."

- 그리디 알고리즘(탐욕법)은 여러 경우 중 하나를 결정해야 할 때마다 그 순간에 최적이라고 생각되는 것을 선택해 나가는 방식으로 진행하여 최종적인 해답에 도달하는 휴리스틱 문제 해결 기법
- 이는 인간의 직관적인 의사결정 방식과 가장 맞닿아 있으며, 복잡한 트리 구조나 그래프 탐색에서 불필요한 연산을 극단적으로 줄여주는 강력한 접근법



그리디 (Greedy) 알고리즘

- 그리디 알고리즘은 최적화 문제를 해결하는 알고리즘
  - 최적화 (optimization) 문제

    가능한 해들 중에서 가장 좋은 (최대 또는 최소) 해를 찾는 문제

- 욕심쟁이 방법, 탐욕적 방법, 탐욕 알고리즘 등으로 불림

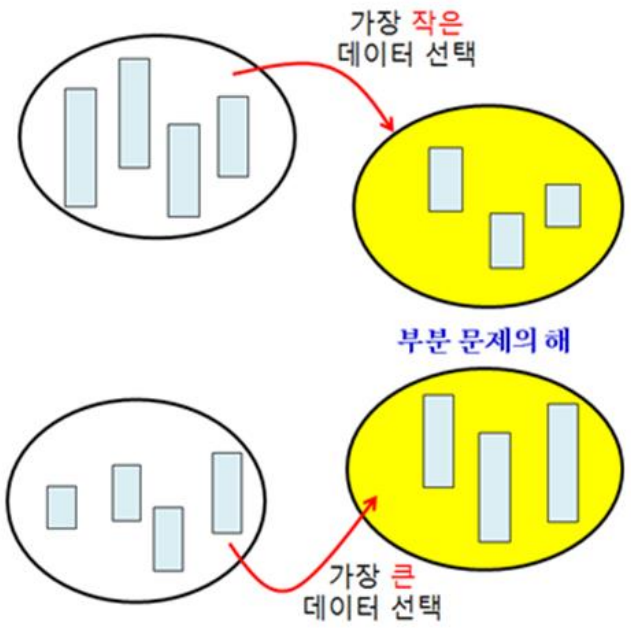
- 그리디 알고리즘은 (입력) 데이터 간의 관계를 고려하지 않고 수행 과정에서 '욕심내어' 최소값 또는 최대값을 가진 데이터를 선택
  - 이러한 선택을 '근시안적'인 선택이라고 말하기도 함

그리디 (Greedy) 알고리즘

- 그리디 알고리즘은 근시안적인 선택으로 부분적인 최적해를 찾고, 이들을 모아서(축적하여) 문제의 최적해를 얻음

최댓값 찾는 문제

최솟값 찾는 문제



그리디 (Greedy) 알고리즘

- 그리디 알고리즘은 일단 한 번 선택하면, 이를 절대로 번복하지 않음
  - 즉, 선택한 데이터를 버리고 다른 것을 취하지 않음

- 이러한 특성 때문에 대부분의 그리디 알고리즘들은 매우 단순하며, 또한 제한적인 문제만이 그리디 알고리즘으로 해결됨

그리디 (Greedy) 알고리즘

탐욕법이 항상 '전역 최적해(Global Optimum)'를 보장하기 위해서는 다음의 두 가지 수학적/구조적 조건이 완벽히 성립해야만 함. 하나라도 위배된다면 근사 알고리즘으로 전락함

01

탐욕적 선택 속성 (Greedy Choice Property)

"앞의 선택이 이후의 선택에 영향을 주지 않는다."

- 현재 단계에서의 탐욕적 선택이 궁극적인 최적해를 구성하는 데 반드시 포함될 수 있음을 수학적으로 증명할 수 있어야 합니다. 선택을 번복(Back tracking)할 필요가 없음을 의미

02

최적 부분 구조 (Optimal Substructure)

"전체의 최적해는 부분의 최적해로 이루어진다."

- 문제 전체에 대한 최적 해결 방법이 하위 문제(Sub-problem)들에 대해서도 최적 해결 방법이어야 함
- 즉, 문제의 스케일을 줄여나가도 본질적인 해의 구조가 유지되어야 함

그리디 (Greedy) 알고리즘

지역적 최적해 (Local Optimum)

- 그리디 알고리즘의 본질적인 시야
- 전체 산맥을 보지 않고, 오직 현재 서 있는 위치에서 가장 가파른 오르막길만 선택하여 올라가는 등산객과 같음
- 이 방식은 연산 속도가 타의 추종을 불허하지만, 자칫하면 가장 높은 정상(Global Maximum)이 아닌 작은 언덕 봉우리(Local Maximum)에 갇히는 치명적인 한계를 지님

전역적 최적해 (Global Optimum)

- 알고리즘이 궁극적으로 도달하고자 하는 최종 목표, 즉 문제 공간 전체에서 가장 완벽한 정답을 의미함
- 탐욕법이 전역 최적해에 도달하려면 매 순간의 지역적 최선의 누적이 전체 최선과 수학적으로 동치(Equivalent)가 되어야 함
- 이를 증명하는 것이 알고리즘 설계의 핵심

4.1 동전 거스름돈 문제

- **목표** : 특정 금액을 만들기 위해 필요한 '최소 동전 개수'를 구하는 문제
- **원리** : 가장 큰 단위의 동전부터 최대한 많이 사용하여 금액을 채워나감
  (예: 500원 → 100원 → 50원 → 10원)
- **전제 조건** : 큰 단위의 동전이 항상 작은 단위 동전의 배수 형태(Canonical)를 이루어야 최적해가 완벽하게 보장됨
- **효율성** : 매번 모든 경우의 수를 계산하는 복잡한 알고리즘 대비 직관적이며 연산 속도가 압도적으로 빠름

알고리즘

CoinChange

입력: 거스름돈 액수 W

출력: 거스름돈 액수에 대한 최소 동전 수

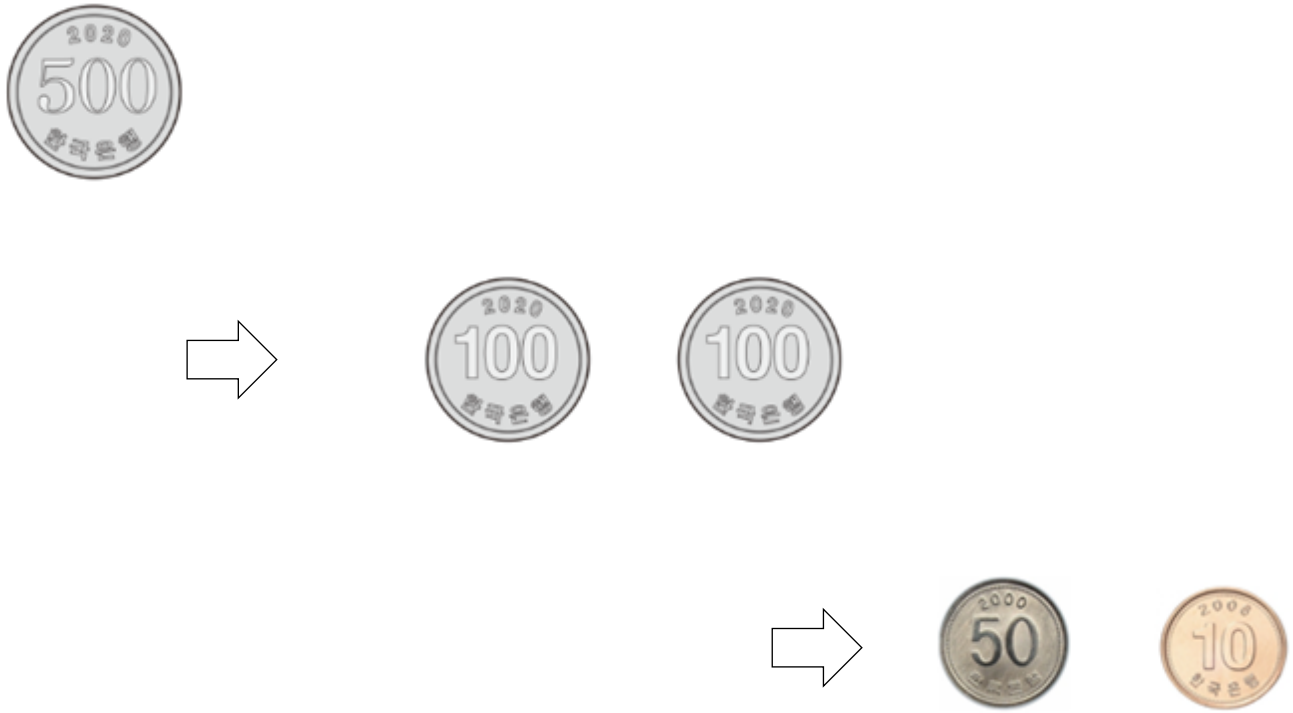
```
1. change=W, n500=n100=n50=n10=n1=0   // n500, n100, n50, n10, n1은 각각의 동전 카운트
2. while ( change ≥ 500 )
       change = change-500, n500++      // 500원 동전 수 1 증가
3. while ( change ≥ 100 )
       change = change-100, n100++      // 100원 동전 수 1 증가
4. while ( change ≥ 50 )
       change = change-50, n50++        // 50원 동전 수 1 증가
5. while ( change ≥ 10 )
       change = change-10, n10++        // 10원 동전 수 1 증가
6. while ( change ≥ 1 )
       change = change-1, n1++          // 1원 동전 수 1 증가
7. return (n500+n100+n50+n10+n1)    // 총 동전 수 리턴
```

CoinChange 알고리즘

- 그리디 알고리즘의 근시안적인 특성
  - CoinChange 알고리즘은 남아있는 거스름돈인 change에 대해 가장 높은 액면의 동전을 거스르며,
  - 500원 동전을 처리하는 line 2에서는 100원, 50원, 10원, 1원 동전을 몇 개씩 거슬러 주어야 할 것인지에 대해서는 전혀 고려하지 않음

수행 과정

760원의 거스름돈에 대해



CoinChange 알고리즘의 문제점

- 한국은행에서 160원 동전을 추가로 발행한다면, CoinChange 알고리즘이 항상 최소 동전 수를 계산할 수 있을까?
  - 거스름돈이 200원이라면, CoinChange 알고리즘은 160원 동전 1개와 10원 동전 4개로서 총 5개를 리턴

CoinChange 알고리즘의 결과

CoinChange 알고리즘의 문제점

- 200원에 대한 최소 동전 수는 100원짜리 동전 2개

CoinChange 알고리즘의 결과

최소 동전의 거스름돈

- CoinChange 알고리즘은 항상 최적의 답을 주지 못함

  그러나 실제로는 거스름돈에 대한 그리디 알고리즘이 적용되도록 동전이 발행됨

사례

일상 속 금융 인프라

자판기 거스름돈 반환 로직

- 가장 전형적인 동전 교환의 사례입니다. 자판기 시스템은 고객에게 거스름돈을 반환할 때, 내부에 보유한 화폐의 지출 개수를 최소화하기 위해 항상 가장 큰 단위의 동전부터 우선적으로 계산하여 지급함

ATM 현금 지급기 인출

- 고객이 원하는 출금액을 은행 ATM에서 인출할 때, 기기는 5만원권, 1만원권 등 가장 고액권부터 최대 장수로 산출하여 지급함
- 이는 기기의 모터 부하를 줄이고 교부 지폐 장수를 최소화하여 편의성을 높임

사례

➢ 외환 및 운송 최적화

환전소 화폐 교환

- 외환 환전소에서 원화를 달러나 유로로 교환할 때, 고객이 여행 중 보관하기 쉽도록 고액권(예 : $100 지폐) 위주로 먼저 환전하여 교부 지폐의 총량을 획기적으로 줄이는 그리디 방식을 실무에 적용함

화물 차량 최적 배차

- 물류 배송 시 10톤, 5톤, 2.5톤, 1톤 등 다양한 크기의 규격 트럭이 존재할 때, 총 화물량을 처리하기 위해 가장 큰 10톤 트럭부터 가득 채워 배차하여 투입 차량 대수와 유류비를 최소화함

사례

➢ 창고 적재 및 포장

공간 효율의 극대화

- 창고의 한정된 공간이나 배송 박스에 다량의 물건을 적재할 때, 부피가 가장 큰 물건이나 표준 규격 중 가장 큰 단위의 포장재부터 먼저 순차적으로 공간에 할당함

- 이는 빈 공간의 발생을 줄이고 포장 효율을 극대화하는 전형적인 그리디 배분 접근법

사례

IT 시스템 인프라 자원 최적화

클라우드 VM 사이징

- 기업이 특정 서버 용량(예: 27GB RAM)을 필요로 할 때 16GB, 8GB 등 가장 큰 인스턴스부터 순차 할당하여 자원 단편화를 막고 프로비저닝 시간을 단축함

네트워크 패킷 분할

- 대용량 데이터를 네트워크로 전송 시, MTU(최대 전송 단위) 규격에 맞춘 가장 큰 프레임 단위로 데이터를 우선 쪼개어 패킷 헤더 오버헤드를 최소화함

파일 시스템 블록 매핑

- OS 디스크 공간 할당 시 4MB, 1MB, 256KB 등 가용한 가장 큰 블록 크기 단위부터 매핑을 진행하여 메타데이터 관리 포인트를 줄이고 I/O 효율을 높임

적용 사례 요약 및 시사점

- **광범위한 확장성:** 단순한 동전 교환 원리는 화폐를 넘어 IT 자원, 물리적 공간, 물류 시스템 등 비즈니스 전반에 폭넓게 적용됨
- **직관성과 비즈니스 효율:** 가장 큰 단위부터 소거해 나가는 직관적 접근법은 복잡한 구조 설계 없이 곧바로 처리 비용 절감과 운영 효율성으로 직결됨
- **실시간 시스템에서의 강점:** 즉각적인 응답이 생명인 금융 결제망, 클라우드 서버 프로비저닝, 실시간 트래픽 제어 시스템에서 알고리즘의 강력한 속도가 진가를 발휘함
- **적용 시 유의사항:** 단, 이 강력한 알고리즘은 각 자원의 단위가 서로 배수 형태(Canonical System)를 이룰 때만 완벽한 최적해 도출이 가능함을 명심해야 함

파이썬 구현 패턴 : 동전 거스름돈 문제

```
# 사용 예시

target = 1260
coin_list = [500, 100, 50, 10]

change_result, left = min_coin_count(target, coin_list)

print(f"--- 거스름돈 결과 (목표 금액: {target}원) ---")
for coin, count in change_result.items():
    print(f"{coin}원 동전: {count}개")

if left > 0:
    print(f"남은 잔돈(거슬러 주지 못한 금액): {left}원")
else:
    print("모든 금액을 성공적으로 거슬러 주었습니다.")
```

파이썬 구현 패턴 : 동전 거스름돈 문제

```
def min_coin_count(target_amount, coins):
    """
    그리디 알고리즘을 이용한 최소 동전 개수 계산 함수
    :param target_amount: 거슬러 줘야 할 전체 금액
    :param coins: 사용 가능한 동전 종류 (리스트)
    :return: 각 동전별 개수가 담긴 딕셔너리
    """
    # 1. 동전을 큰 단위부터 처리하기 위해 내림차순 정렬
    coins.sort(reverse=True)
    result = {}
    remaining = target_amount

    for coin in coins:
        # 2. 현재 동전으로 거슬러 줄 수 있는 최대 개수 계산
        count = remaining // coin

        if count > 0:
            result[coin] = count
            # 3. 남은 금액 업데이트
            remaining %= coin

    return result, remaining
```

4.2 최소 신장 트리

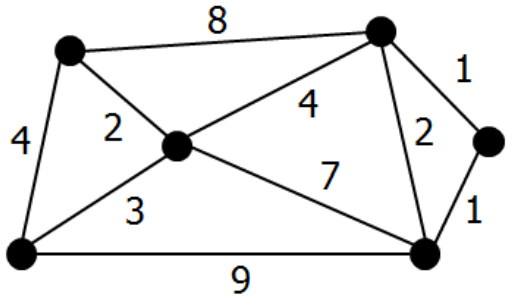
최소 신장 트리란? (MST)

"그래프 내의 모든 정점을 연결하되, 사이클을 형성하지 않으면서
연결선들의 가중치 합이 최소가 되도록 구성된 트리"

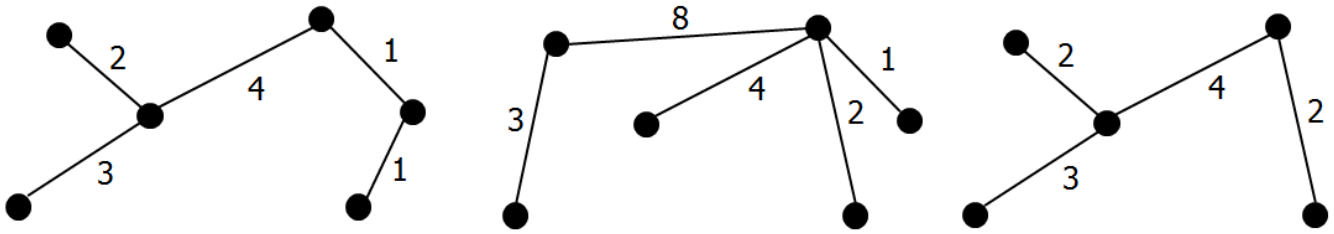
그리디 알고리즘의 가장 강력하고 대표적인 성공 모델

4.2 최소 신장 트리

➢ **최소 신장 트리** (Minimum Spanning Tree)



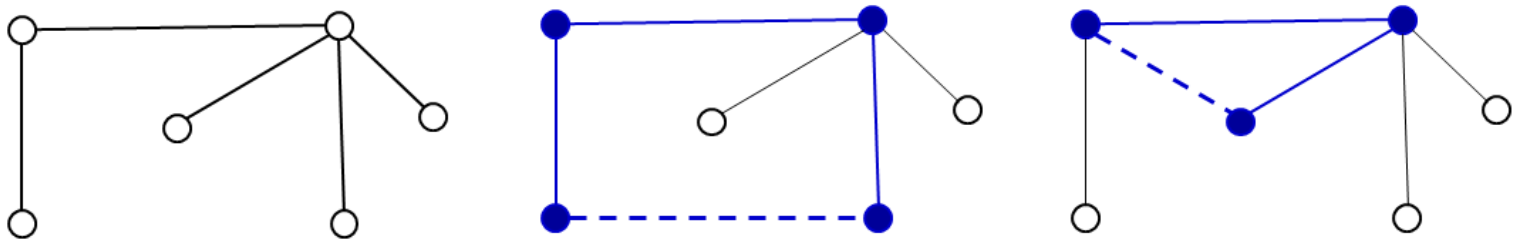
다음 중 최소 신장 트리는?



최소 신장 트리

- 주어진 그래프의 신장 트리를 찾는 방법
  - 사이클이 없도록 모든 점을 연결시킴

- 그래프의 점의 수 = n
  - 신장 트리에는 정확히 (n-1)개의 간선이 있음
  - 트리에 간선을 하나 추가시키면, 반드시 사이클이 만들어짐



트리 점선으로 된 선분을 추가하여 만들어진 사이클

최소 신장 트리 알고리즘

- **크러스컬(Kruskal) 알고리즘**
  - 가중치가 가장 작은 간선이 사이클을 만들지 않을 때에만 '욕심내어' 그 간선을 추가시킴
- **프림(Prim) 알고리즘**
  - 임의의 점 하나를 선택한 후, (n-1)개의 간선을 하나씩 추가시켜 트리를 만듬
- **알고리즘의 입력은 1개의 연결 성분(connected component)으로 된 가중치 그래프**

연결 성분 내의 점들 사이에 경로가 있는 극대부분 그래프(maximal subgraph)를 의미
그래프가 2개 이상의 연결 성분을 가지면, 서로 다른 연결성분의 점들 사이에는 경로가 없다

알고리즘

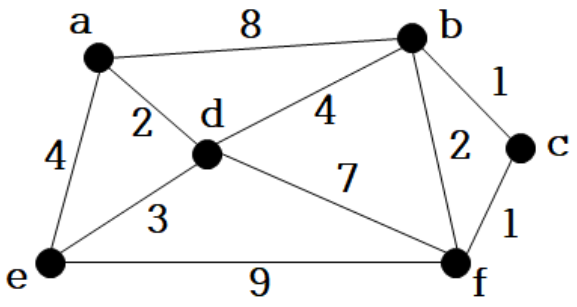
KruskalMST(G)

입력: 가중치 그래프 G=(V,E), |V|=n , |E|=m

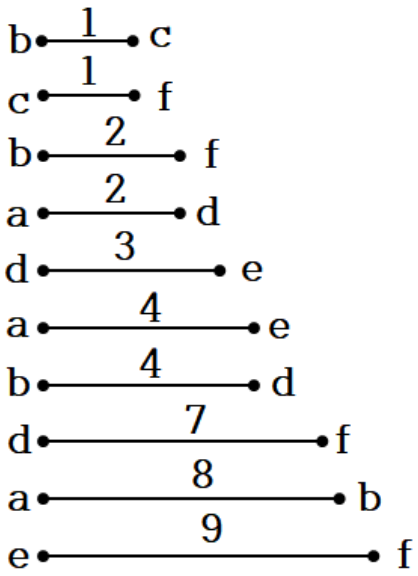
출력: 최소 신장 트리 T

```
1. 가중치의 오름차순으로 간선들을 정렬: L = 정렬된 간선 리스트
2. T=∅        // 트리 T를 초기화
3. while ( T의 간선 수 < n-1 )
4.     L에서 가장 작은 가중치를 가진 간선 e를 가져오고, e를 L에서 제거
5.     if (간선 e가 T에 추가되어 사이클을 만들지 않으면)
6.         e를 T에 추가
7.     else        // e가 T에 추가되어 사이클이 생기는 경우
8.         e를 버린다.
9. return 트리 T    // T는 최소 신장 트리
```

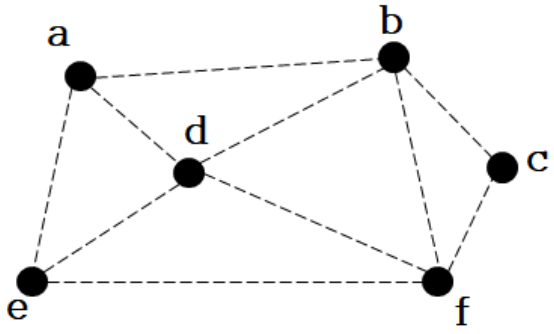
KruskalMST 알고리즘 수행 과정



입력 그래프



정렬된 리스트 L

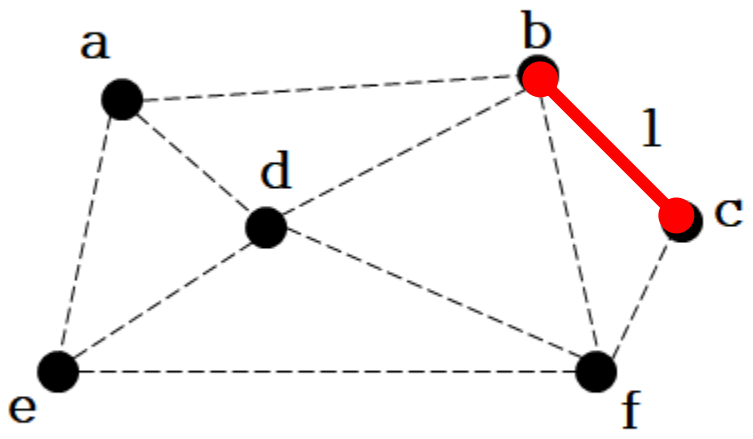


T = ∅

수행 과정

리스트 L

b	1	c
c	1	f
b	2	f
a	2	d
d	3	e
a	4	e
b	4	d
d	7	f
a	8	b
e	9	f

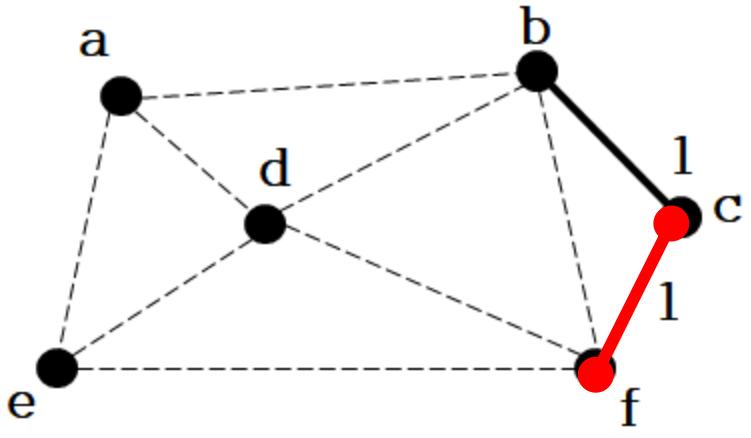


간선 (b, c) 추가

수행 과정

리스트 L

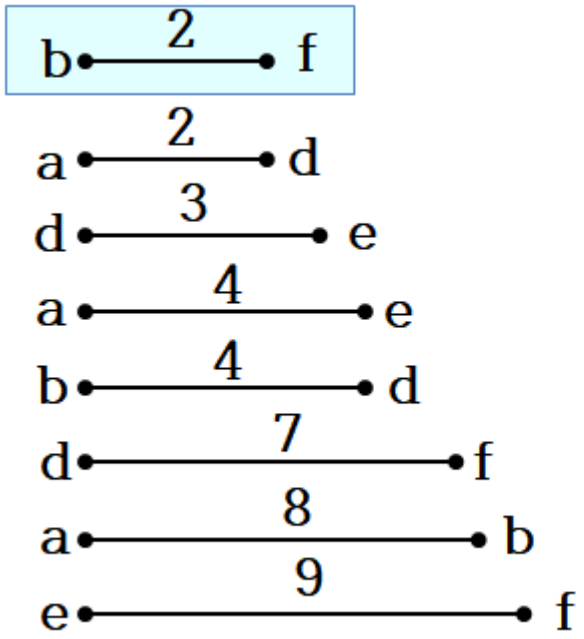
c	1	f
b	2	f
a	2	d
d	3	e
a	4	e
b	4	d
d	7	f
a	8	b
e	9	f

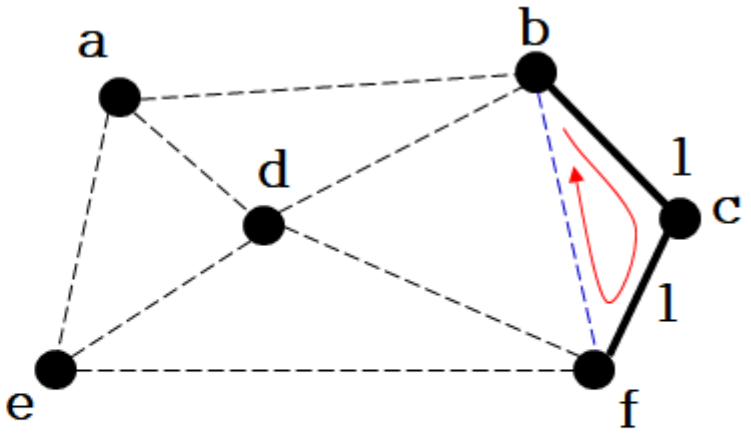


간선 (c, f) 추가

수행 과정

리스트 L





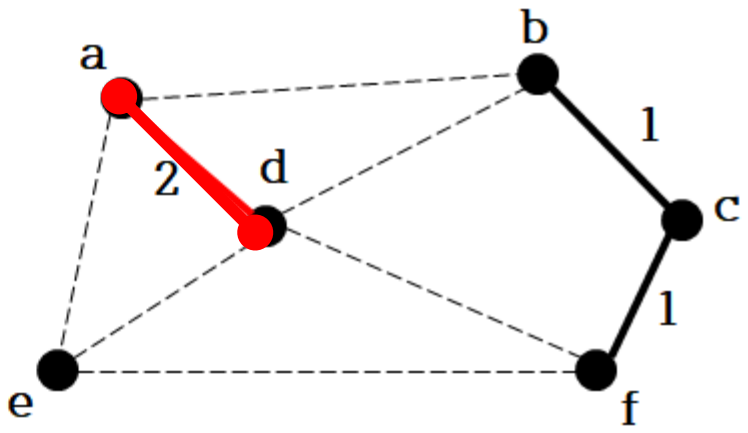
사이클 b-c-f-b

간선 (b, f) 버림

수행 과정

리스트 L

2	a	d
3	d	e
4	a	e
4	b	d
7	d	f
8	a	b
9	e	f

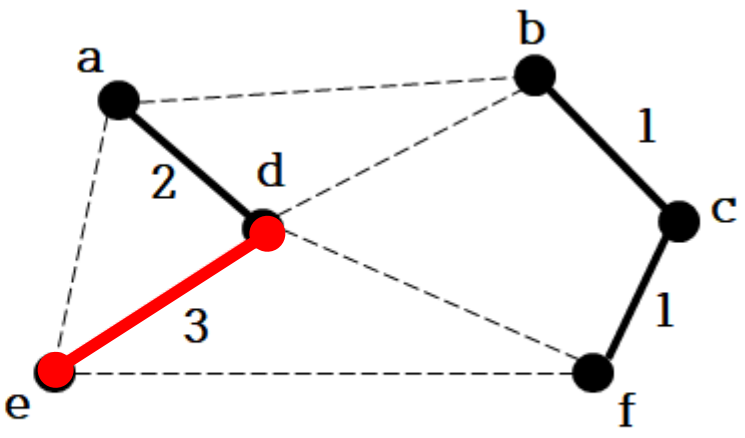


간선 (a, d) 추가

수행 과정

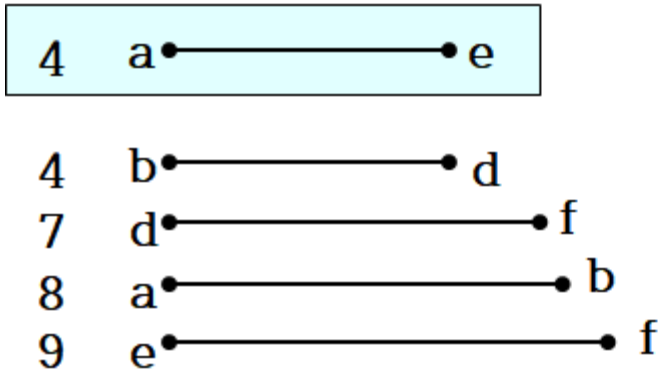
리스트 L

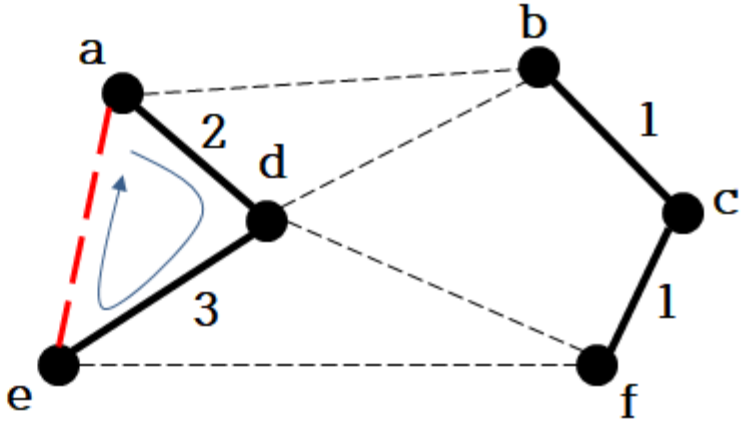
3	d	e
4	a	e
4	b	d
7	d	f
8	a	b
9	e	f



간선 (d, e) 추가

수행 과정





사이클 a-d-e-a

수행 과정

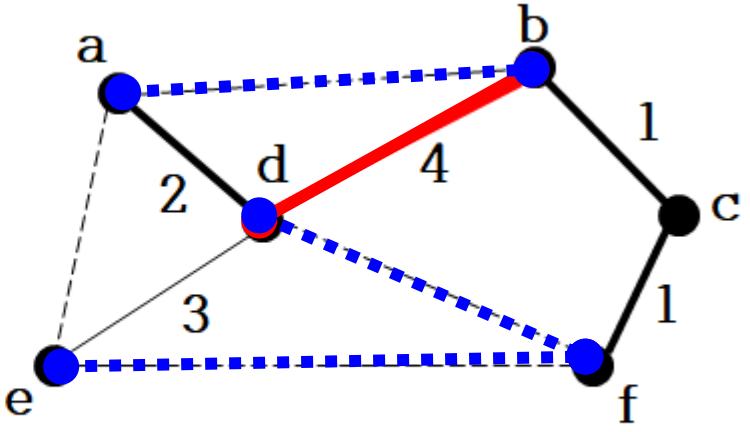
리스트
L

b — 4 — d

d — 7 — f 사이클 형성

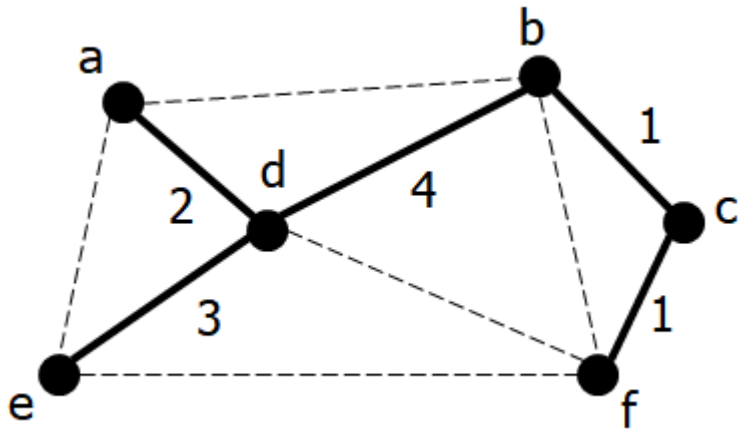
a — 8 — b 사이클 형성

e — 9 — f 사이클 형성



간선 (b, d) 추가

MST



시간 복잡도

- Line 1 : 간선 정렬하는데 O(m log m) 시간

  단, m은 입력 그래프에 있는 간선의 수

- Line 2 : T를 초기화하는 것이므로 O(1) 시간

- Line 3~8
  - while-루프는 최대 m번 수행

    그래프의 모든 간선이 while-루프 내에서 처리되는 경우
  - while-루프 내에서는 L 로부터 가져온 간선 e 가 사이클을 만드는지를 검사하는데 거의 O(1) 시간

- Kruskal 알고리즘의 시간복잡도: O(m log m)

프림 (Prim)의 MST 알고리즘

- 주어진 가중치 그래프에서 임의의 점 하나를 선택한 후, (n-1)개의 간선을 하나씩 추가시켜 트리 생성

- 추가되는 간선은 현재까지 만들어진 트리에 연결시킬 때 '욕심내어' 항상 최소의 가중치로 연결되는 간선

알고리즘

PrimMST(G)

입력: 가중치 그래프 G=(V,E), |V|=n(점의 수), |E|=m(간선의 수)

출력: 최소 신장 트리 T

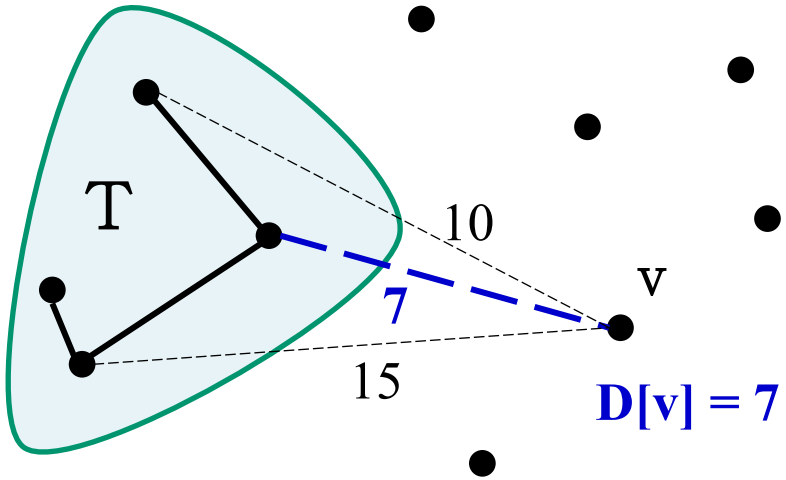
```
1. G에서 임의의 점 p를 시작점으로 선택 D[p] = 0
   // D[v]는 T에 있는 u와 v를 연결하는 간선의 최소 가중치를 저장하기 위한 원소
2. for (점 p가 아닌 각 점 v에 대하여) {     // 배열 D의 초기화
3.     if ( 간선 (p, v)가 그래프에 있으면 )
4.         D[v] = 간선 (p, v)의 가중치
5.     else
6.         D[v]=∞
   }
```

알고리즘

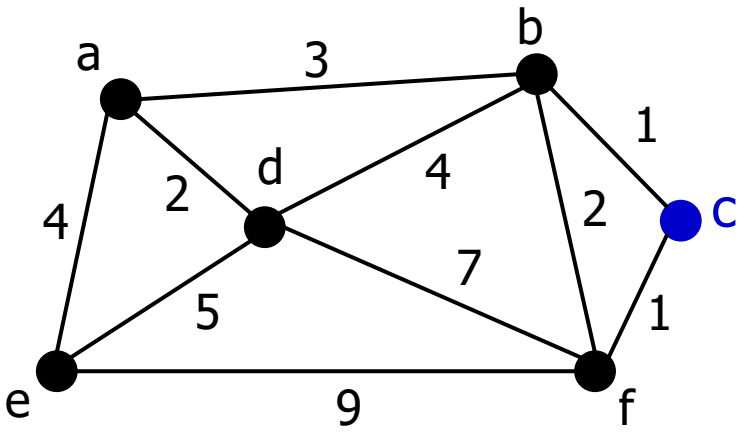
```
T= {p}          // 초기에 트리 T는 점 p만을 가진다.

while (T에 있는 점의 수 < n) {
    T에 속하지 않은 각 점 v에 대하여, D[v]가 최소인 점 v_min과 연결된 간선 (u, v_min)
    을 T에 추가, 여기서 u는 T에 속한 점이고, 점 v_min도 T에 추가
    for (T에 속하지 않은 각 점 w에 대해서) {
        if (간선 (v_min, w)의 가중치 < D[w])
            D[w] = 간선 (v_min, w)의 가중치 // D[w]를 갱신
    }
}

return T        // T는 최소 신장 트리
```

D[v] 설명

- Line 1
  - 임의로 점 p를 선택하고, D[p]=0으로 놓음
  - D[v]에는 점 v와 T에 속한 점들을 연결하는 간선들 중에서 최소 가중치를 가진 간선의 가중치를 저장
  - 그림에서 D[v]에는 10, 7, 15 중에서 최소 가중치인 7이 저장



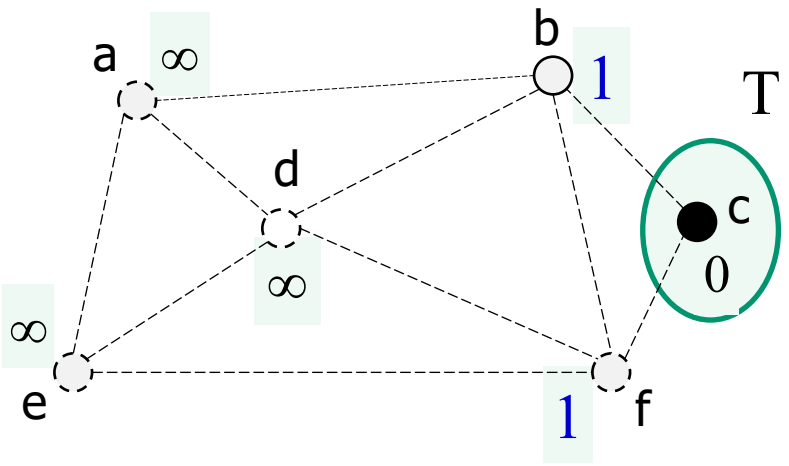
수행 과정



- Line 1: 임의의 점 c 선택, D[c]=0으로 초기화

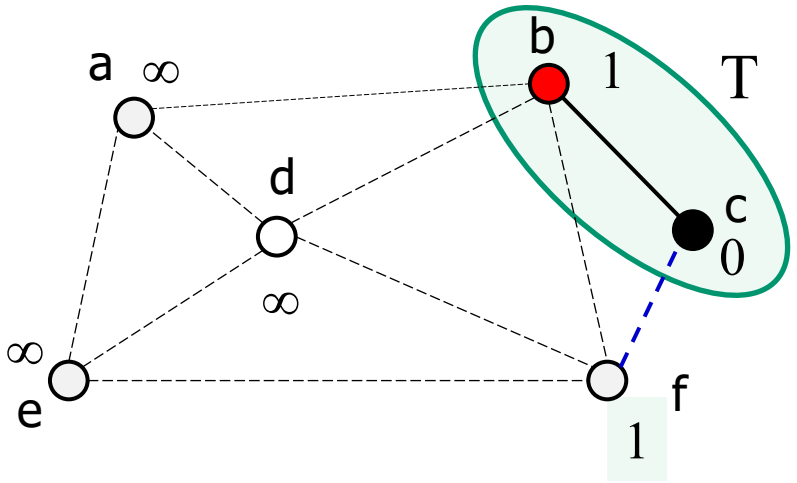
수행 과정

- Line 2~6:
  - 시작점 c와 간선으로 연결된 각 점 v에 대해서, D[v]를 각 간선의 가중치로 초기화
  - 나머지 각 점 v에 대해서, D[v]는 ∞로 초기화



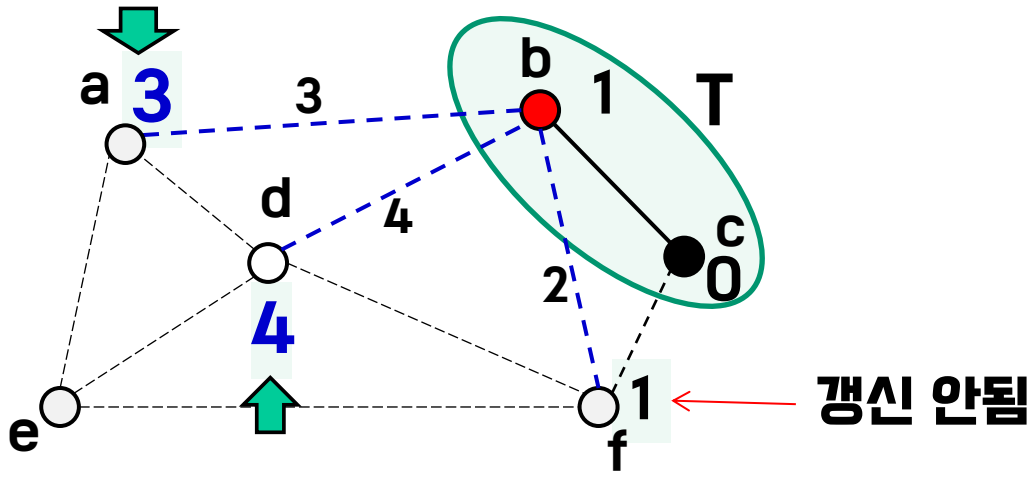
수행 과정

- Line 7: T={c}로 초기화
- Line 8-9: T에 가장 가까운 점 b를 추가

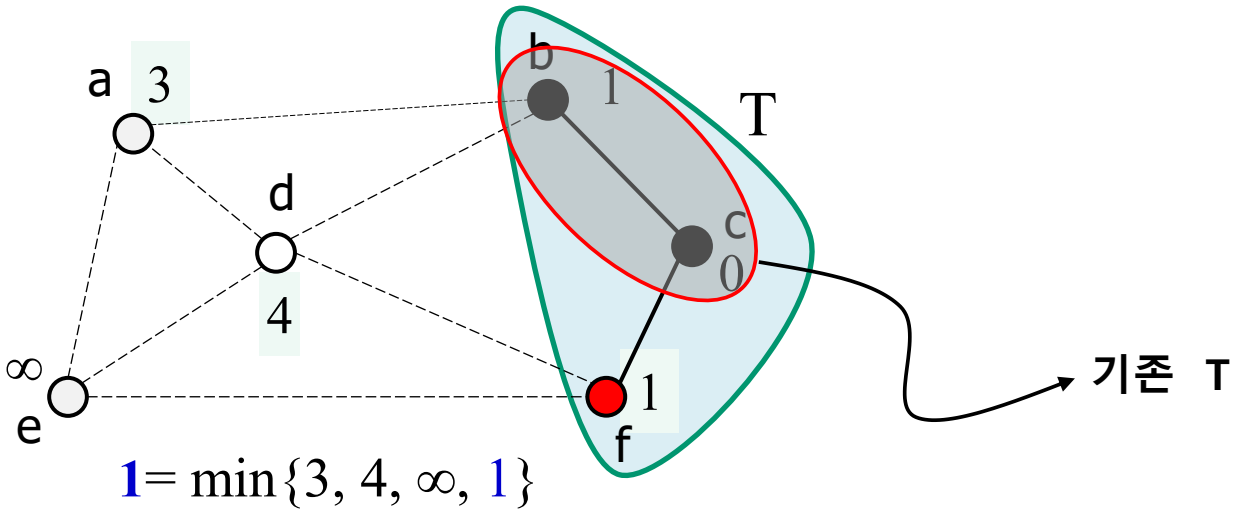


$$1 = \min\{\infty, 1, \infty, \infty, 1\}$$

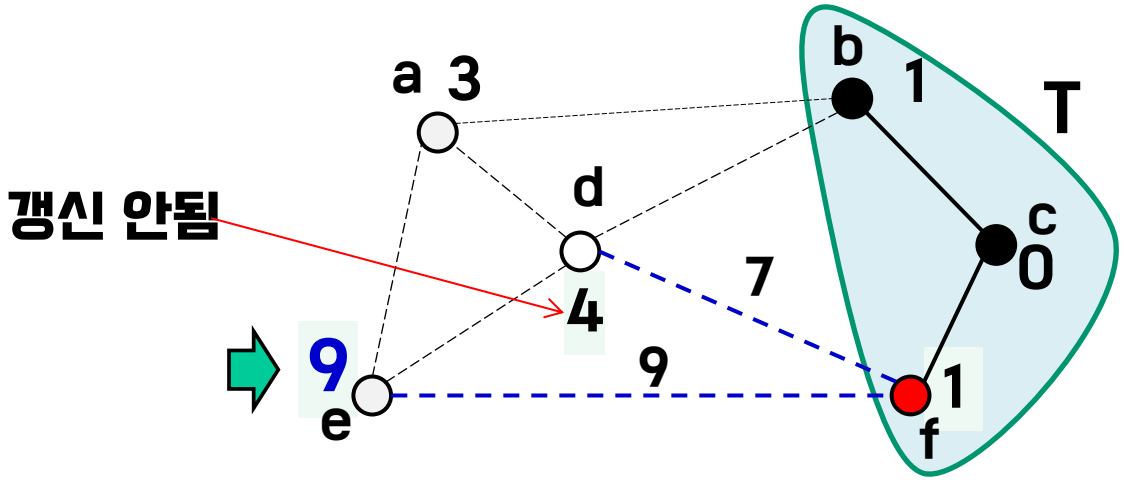
b에 연결된 a와 d의 D[a]와 D[d] 갱신



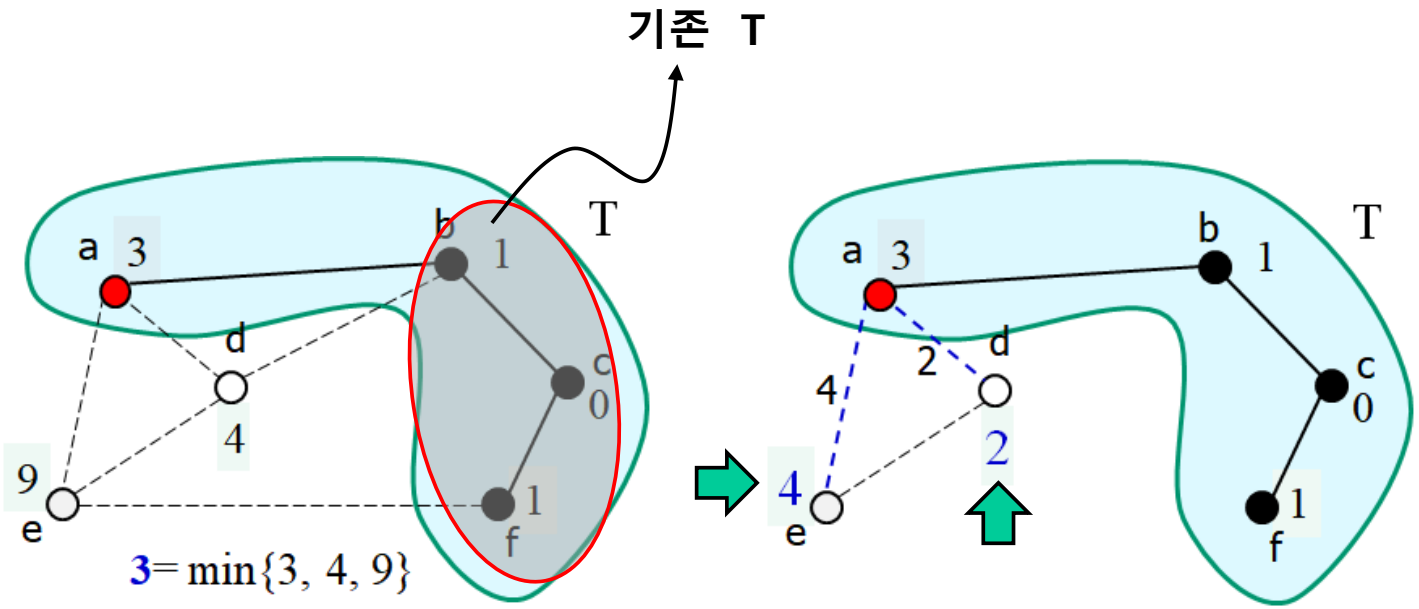
T에 가장 가까운 점 f를 추가



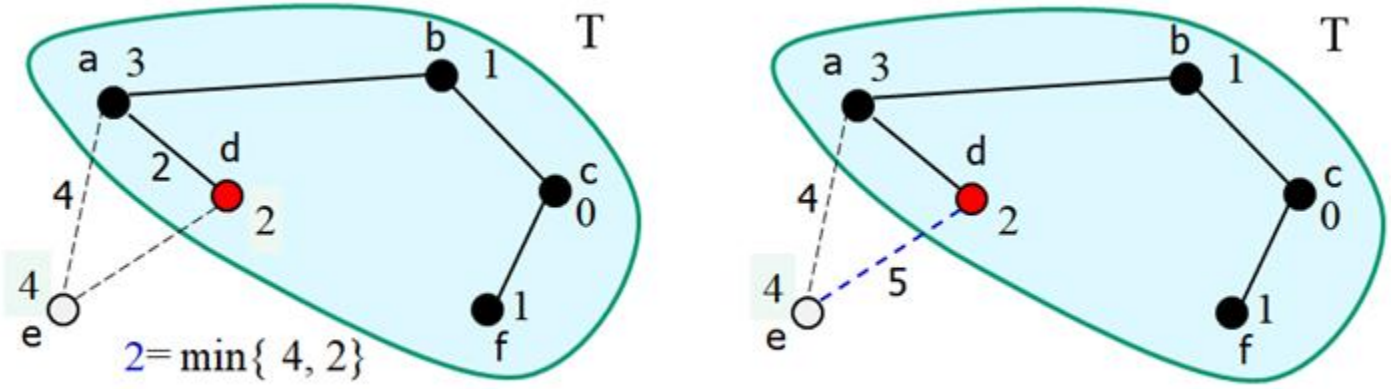
f에 연결된 e의 D[e] 갱신



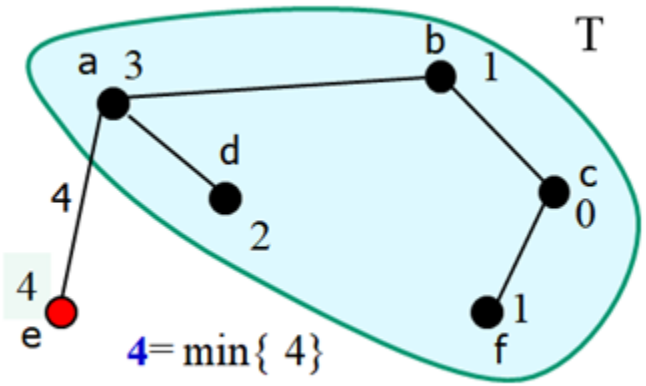
a를 T에 추가

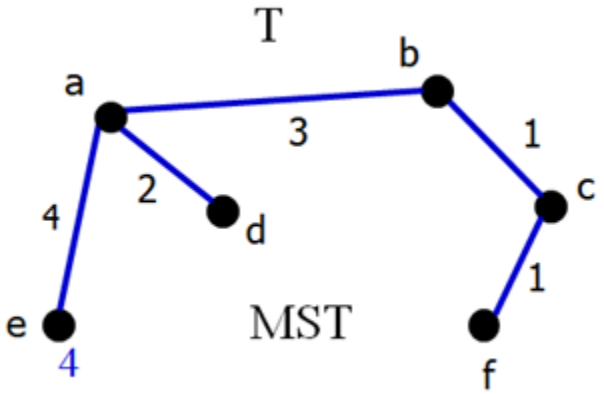


d를 T에 추가



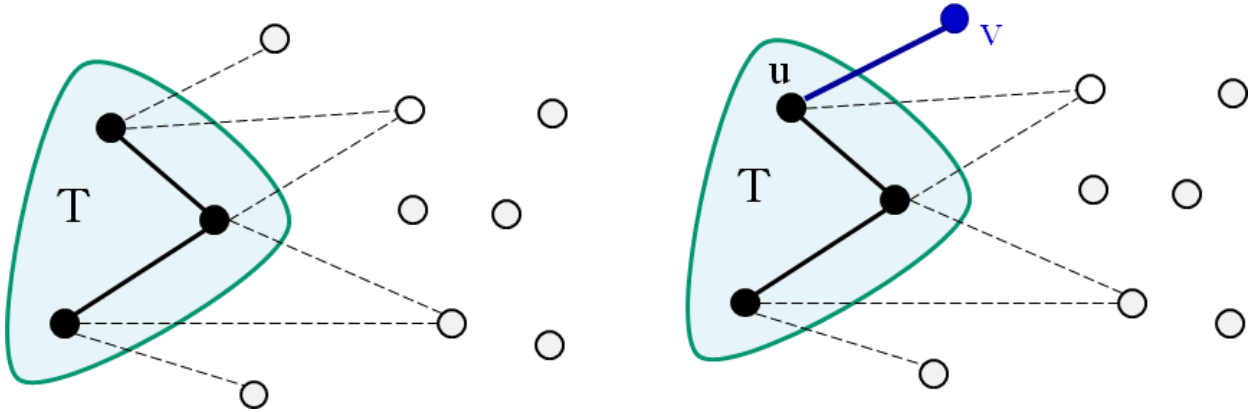
e를 T에 추가





PrimMST가 찾은 T에는 왜 사이클이 없을까?

- 프림 알고리즘은 T 밖에 있는 점을 항상 추가하므로 사이클이 만들어지지 않음



시간 복잡도

- while-루프가 (n-1)회 반복되고,
  - 1회 반복될 때 line 9에서 T에 속하지 않은 각 점 v에 대하여, D[v]가 최소인 점 v_{min}을 찾는데 O(n) 시간 소요
  - 배열 D에서 (현재 T에 속하지 않은 점들에 대해서) 최솟값을 찾는 것이고, 배열의 크기는 n이기 때문

- 프림 알고리즘의 시간 복잡도
  - (n-1) × O(n) = $O(n^2)$
  - 최소 힙(Binary Heap)을 사용하여 v_{min}을 찾으면 O(mlogn), m은 간선 수. 따라서 간선 수가 O(n)이면 O(nlogn)

Kruskal과 Prim 알고리즘의 수행 과정 비교

➢ 크러스컬 알고리즘

- 간선이 1개씩 T에 추가되는데, 이는 마치 n개의 점들이 각각의 트리인 상태에서 간선이 추가되면 2개의 트리가 1개의 트리로 합쳐지는 것과 같음
- 크러스컬 알고리즘은 이를 반복하여 1개의 트리인 T를 생성
- n개의 트리들이 점차 합쳐져서 1개의 신장 트리가 만들어짐

➢ 프림 알고리즘

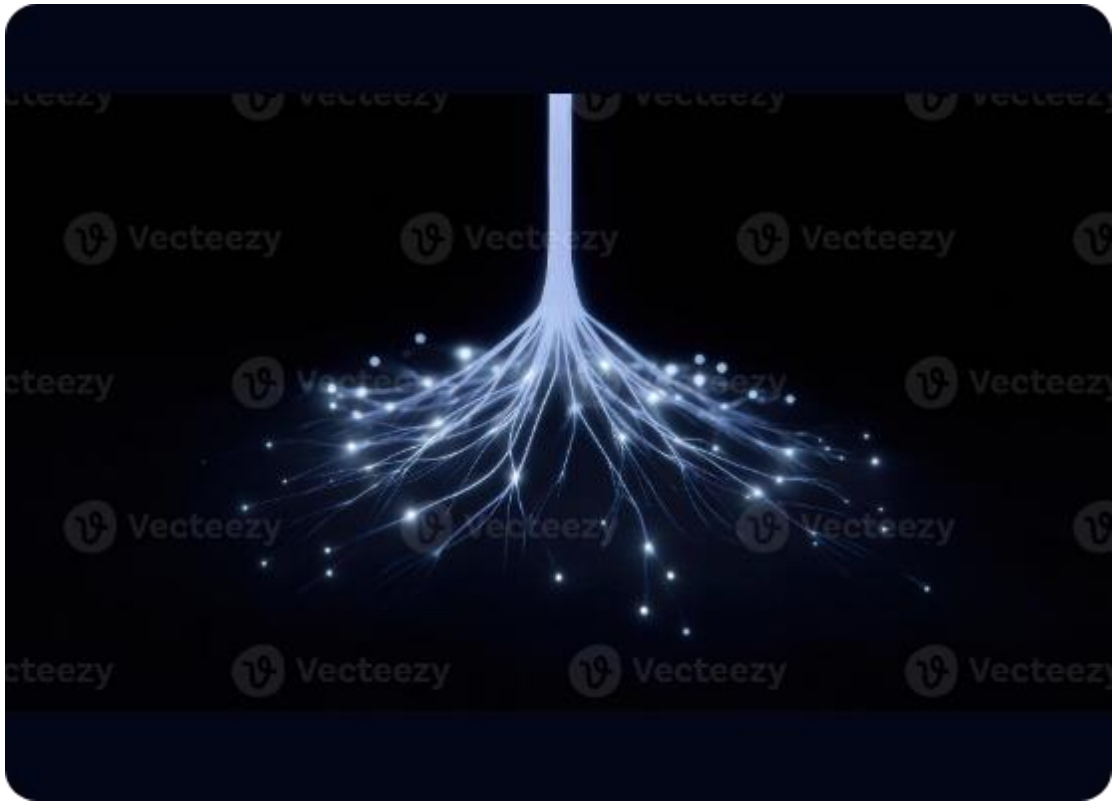
- T가 점 1개인 트리에서 시작되어 간선을 1개씩 추가
- 1개의 트리가 자라나서 신장 트리가 됨

응용

- 최소 비용으로 선로 또는 파이프 네트워크 (인터넷 광 케이블 선로, 케이블 TV선로, 전화선로, 송유관로, 가스관로, 배수로 등)를 설치하는데 활용

사례 1. 통신 네트워크망 (광케이블) 구축

- **배경:** 전국 또는 글로벌 단위의 도시들을 광케이블이나 전화선으로 연결해야 함
- **적용:** 각 도시(노드)를 연결하는 데 드는 케이블 길이(비용)를 계산하고, 가장 비용이 적게드는 구간부터 순차적으로 연결함
- **효과:** 단 하나의 도시도 고립되지 않게 보장하면서, 케이블 매설 및 장비 도입에 드는 막대한 인프라 비용을 전역적으로 최소화함



사례 2. 도시 유틸리티 인프라 설계

국가 전력망 (전기) 배선

- 발전소와 변전소, 그리고 각 지역의 수많은 가정을 전선으로 연결할 때 MST 알고리즘이 사용
- 송전탑 건설 비용과 전선의 길이를 최소화하여 전력 손실을 줄이고 효율적인 전력 공급망을 설계하는 데 핵심적인 역할을 수행함

상수도망 (배관망) 최적화

- 정수장에서 각 거주지나 공업 단지로 물을 공급하기 위해 파이프를 매설할 때, 지형적 특성과 거리를 고려하여 배관 설치 비용을 최소화해야 함
- MST는 모든 지역에 수도가 공급되도록 하면서 파이프의 총길이를 가장 짧게 만듬

사례 3. 물리적 네트워크 및 회로 설계

교통/도로망 건설

- 여러 도시나 거점을 모두 연결하는 고속도로망이나 철도망을 건설할 때, 산맥 통과나 교량 건설 등의 가중치(비용)를 반영하여 전체 건설 예산이 최소화되는 최적의 루트를 산출함

컴퓨터 LAN 네트워크

- 기업의 사옥이나 캠퍼스 내에 있는 수백 대의 컴퓨터와 라우터를 이더넷 케이블로 연결할 때, 선로 구축 비용과 스위칭 장비 도입 비용을 절감하기 위해 MST 토폴로지를 기반으로 설계함

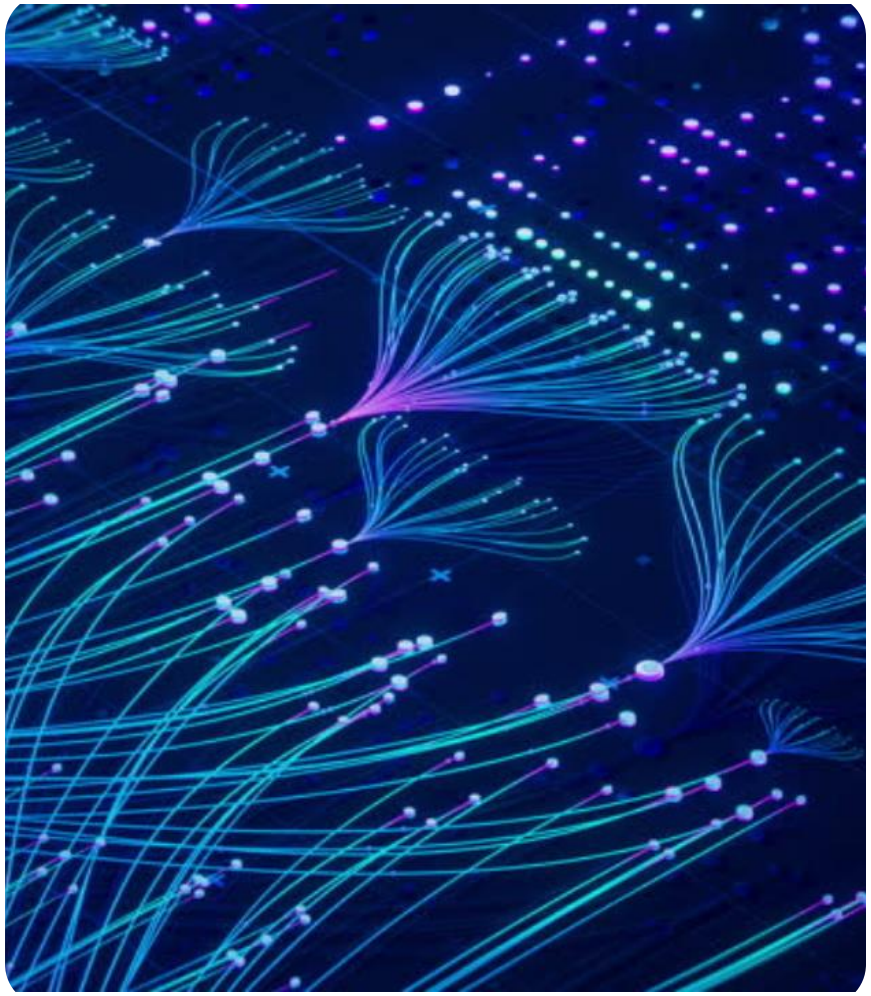
VLSI (반도체 회로) 설계

- 인쇄 회로 기판(PCB)이나 칩셋 내부에서 여러 개의 전자 핀(Pin)을 전기적으로 연결할 때, 와이어의 길이를 최소화하여 신호 지연을 막고 칩의 크기를 소형화하는 데 적용됨

사례 4. 머신러닝 데이터 클러스터링

단일 연결 클러스터링 (Single-linkage)

- 데이터 마이닝 및 비지도 학습에서 유사한 특성을 가진 데이터 포인트들을 그룹화할 때 MST 알고리즘을 활용
- 모든 데이터 포인트를 거리(유사도)에 따라 MST로 연결한 뒤, 가장 가중치가 큰 간선(Edge)을 순서대로 끊어내면, 자연스럽게 데이터가 의미 있는 군집(Cluster)들로 분할되는 강력한 기법



사례 5. 컴퓨터 비전의 이미지 분할

그래프 기반 이미지 세그먼테이션

- 이미지를 구성하는 픽셀들을 노드로, 픽셀 간의 색상이나 명암 차이를 간선의 가중치로 모델링함
- 그리디 방식의 크루스칼(Kruskal) 알고리즘을 응용하여 유사한 속성을 가진 픽셀들을 하나의 신장 트리로 묶어나감으로써, 객체와 배경을 빠르고 효과적으로 분리해냄

적용 분야

- 자율주행 자동차의 객체 인식, 의료 영상(MRI, CT)에서의 종양 및 장기 경계선 추출, 로봇 비전 시스템 등 픽셀 단위의 빠르고 정확한 군집화가 필요한 첨단 비전 기술에 기반 알고리즘으로 적용됨

MST가 비즈니스에 제공하는 가치

- **막대한 인프라 비용 절감:** 가장 적은 비용(거리, 시간, 자원)으로 모든 대상을 하나로 연결하여 CAPEX (자본 지출)를 극적으로 최소화함
- **시스템 안정성 보장:** 사이클을 원천적으로 차단하여 IT 네트워크의 루프(Loop)를 막고 통신 장애를 예방함
- **실시간 최적화 (Scalability):** 그리디 로직의 가벼운 연산 복잡도 덕분에 노드가 수십만 개인 대규모 데이터셋이나 통신망에서도 실시간으로 트리를 재구성할 수 있음
- **데이터 과학 기반 모델:** 물리적 배선을 넘어 머신러닝의 클러스터링, 컴퓨터 비전까지 추상화된 데이터 모델의 기초로 널리 활용됨

파이썬 구현 패턴 1 : 크러스컬 알고리즘 (기본 구현)

- 가장 직관적인 방법으로, 간선(Edge)의 가중치를 오름차순으로 정렬한 뒤 사이클이 발생하지 않는 선에서 간선을 하나씩 추가하는 방식
- 이를 위해 Union-Find(서로소 집합) 자료구조를 사용함

```
# 정점 개수 6개, (노드1, 노드2, 가중치)
edges_data = [(1, 2, 5), (1, 3, 4), (2, 3, 2), (2, 4, 7), (3, 4, 6), (3, 5, 11), (4, 5, 3), (4, 6, 8), (5, 6, 8)]
cost, mst = kruskal_basic(6, edges_data)
print(f"[예제 1] 크루스칼 최소 비용: {cost}")
print(f"선택된 간선들: {mst}")
```

파이썬 구현 패턴 1 : 크러스컬 알고리즘 (기본 구현)

```
# 1. 크루스칼 알고리즘 (Kruskal's Algorithm)
def find_parent(parent, x):
        # 루트 노드가 아니라면, 루트 노드를 찾을 때까지
        # 재귀적으로 호출 (경로 압축)
        if parent[x] != x:
                parent[x] = find_parent(parent, parent[x])
        return parent[x]

def union_parent(parent, a, b):
        a = find_parent(parent, a)
        b = find_parent(parent, b)
        if a < b:
                parent[b] = a
        else:
                parent[a] = b
```

```
def kruskal_basic(v, edges):
        parent = [0] * (v + 1)
        for i in range(1, v + 1):
                parent[i] = I
        result_cost = 0
        mst_edges = []

        # 간선을 비용 순으로 오름차순 정렬 (그리디 접근)
        edges.sort(key=lambda x: x[2])

        for edge in edges:
                a, b, cost = edge
                # 사이클이 발생하지 않는 경우에만 집합에 포함
                if find_parent(parent, a) != find_parent(parent, b):
                        union_parent(parent, a, b)
                        result_cost += cost
                        mst_edges.append((a, b, cost))
        return result_cost, mst_edges
```

파이썬 구현 패턴 2 : 프림 알고리즘

- 시작 정점에서 출발하여, 현재 연결된 트리와 인접한 간선들 중 가장 가중치가 작은 간선을 선택하며 트리를 확장해 나가는 방식
- 최소 힙(heapq)을 사용하여 O(E log V)의 속도를 냄

```
# 그래프를 인접 리스트 형태로 구성

graph_data = {
                    'A': [('B', 5), ('C', 4)],
                    'B': [('A', 5), ('C', 2), ('D', 7)],
                    'C': [('A', 4), ('B', 2), ('D', 6), ('E', 11)],
                    'D': [('B', 7), ('C', 6), ('E', 3), ('F', 8)],
                    'E': [('C', 11), ('D', 3), ('F', 8)],
                    'F': [('D', 8), ('E', 8)]
 }
cost, mst = prim_basic('A', graph_data)
print(f"\n[예제 2] 프림 최소 비용: {cost}")
print(f"선택된 간선들: {mst}")
```

파이썬 구현 패턴 2 :
프림 알고리즘

```python
# 2. 프림 알고리즘 (Prim's Algorithm - 우선순위 큐)
import heapq
def prim_basic(start_node, graph):
    visited = set([start_node])
    mst_edges = []
    total_cost = 0

    # 인접 간선들을 (가중치, 시작노드, 도착노드) 형태로 힙에 삽입
    edges = [(weight, start_node, to_node) for to_node, weight in graph[start_node]]
    heapq.heapify(edges)

    while edges:
        weight, frm, to = heapq.heappop(edges)

        # 아직 방문하지 않은 노드라면 트리에 추가 (그리디 접근)
        if to not in visited:
            visited.add(to)
            mst_edges.append((frm, to, weight))
            total_cost += weight

            # 새롭게 연결된 노드의 인접 간선들을 힙에 추가
            for next_node, next_weight in graph[to]:
                if next_node not in visited:
                    heapq.heappush(edges, (next_weight, to, next_node))
    return total_cost, mst_edges
```

파이썬 구현 패턴 3 : 최대 신장 트리 (Maximum Spanning Tree) 알고리즘

- 응용 예제로, 최소 비용이 아니라 네트워크의 대역폭이나 신뢰성을 최대화해야 할 때 사용
- 크루스칼 알고리즘에서 간선 정렬 방식을 내림차순으로만 바꾸면 간단히 구현할 수 있음

```
# 실행 예시
edges_data_max = [(1, 2, 10), (2, 3, 15), (1, 3, 5), (4, 2, 2), (4, 3, 40)]
cost, mst = max_spanning_tree(4, edges_data_max)
print(f"\n[예제 3] 최대 신장 트리 총 가중치: {cost}")
print(f"선택된 간선들: {mst}")
```

파이썬 구현 패턴 3 :
최대 신장 트리 (Maximum Spanning Tree) 알고리즘

```
# 3. 최대 신장 트리 (크루스칼 응용)
def max_spanning_tree(v, edges):
    parent = [i for i in range(v + 1)]

    def find(x):
        if parent[x] != x: parent[x] = find(parent[x])
        return parent[x]

    def union(a, b):
        a, b = find(a), find(b)
        if a < b:
            parent[b] = a
        else:
            parent[a] = b

    # 최대 신장 트리를 위해 '내림차순' 정렬 (가장 큰 가중치부터 선택)
    edges.sort(key=lambda x: x[2], reverse=True)
    max_cost = 0
    mst_edges = []
    for a, b, cost in edges:
        if find(a) != find(b):
            union(a, b)
            max_cost += cost
            mst_edges.append((a, b, cost))
    return max_cost, mst_edges
```

최단 경로 알고리즘의 본질

“ 현 위치에서 가장 가중치(시간, 거리, 비용)가 적은 노드를 '그리디(탐욕적)'하게 선택해 나감으로써, 출발지에서 목적지까지의 완벽한 최단 경로를 산출 ”

4.3 최단 경로 찾기

- 최단 경로 (Shortest Path) 문제
  - 주어진 가중치 그래프에서 어느 한 출발점에서 또 다른 도착점까지의 최단 경로를 찾는 문제
- 최단 경로를 찾는 가장 대표적인 알고리즘
  - 다익스트라(Dijkstra) 최단 경로 알고리즘
- 다익스트라 알고리즘
  - 주어진 출발점에서 시작
  - 출발점으로부터 최단 거리가 확정되지 않은 점들 중에서 출발점으로부터 가장 가까운 점을 추가하고, 그 점의 최단 거리를 확정

알고리즘

ShortestPath(G, s)

입력: 가중치 그래프 G=(V,E), |V|=n , |E|=m

출력: 출발점 s로부터 (n-1)개의 점까지 각각 최단 거리를 저장한 배열 D

1. 배열 D를 ∞로 초기화. 단, D[s]=0으로 초기화

   // 배열 D[v]에는 출발점 s로부터 점 v까지의 거리를 저장

2. while (s로부터의 최단 거리가 확정되지 않은 점이 있으면)

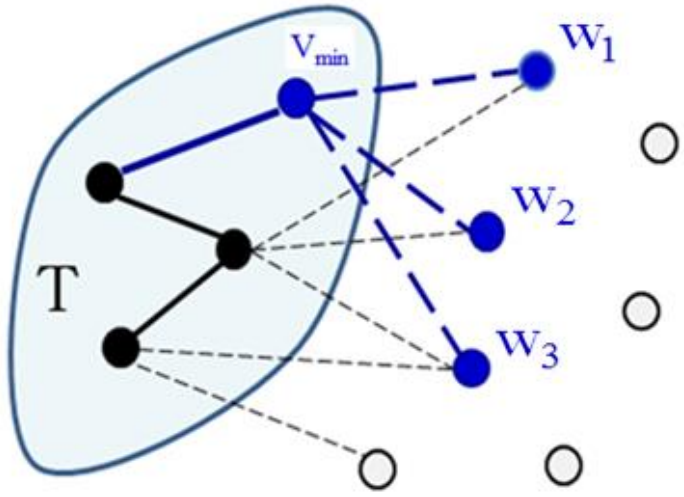
3. 현재까지 최단 거리가 확정되지 않은 각 점 v에 대해서 최소의 D[v]의 값을 가진 점 v_{min}을 선택하고, s로부터 점 v_{min}까지의 최단 거리 $D[v_{min}]$을 확정

4. s로부터 현재보다 짧은 거리로 점 v_{min}을 통해 우회 가능한 각 점 w에 대해서 D[w]를 갱신 // 간선 완화

5. return D

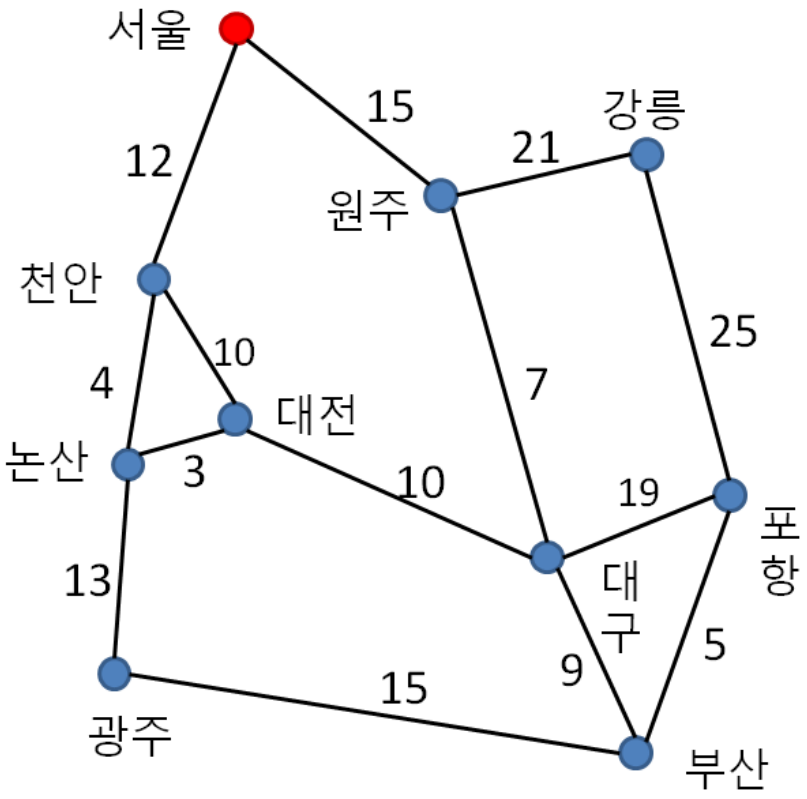
간선 완화(Edge Relaxation)

- Line 4
  - V-T에 속한 점들 중 v_{min}을 거쳐 감 (경유함)으로서 s로부터의 거리가 현재보다 더 짧아지는 점 w가 있으면, 그 점의 D[w]를 갱신
  - v_{min}이 T에 포함된 상태에서 v_{min}에 인접한 점 w_1, w_2, w_3 각각에 대해서 만일 ($D[v_{min}]$+간선(v,w_i)의 가중치) < $D[w_i]$이면, $D[w_i]$ = ($D[v_{min}]$+간선(v_{min},w_i)의 가중치)로 갱신



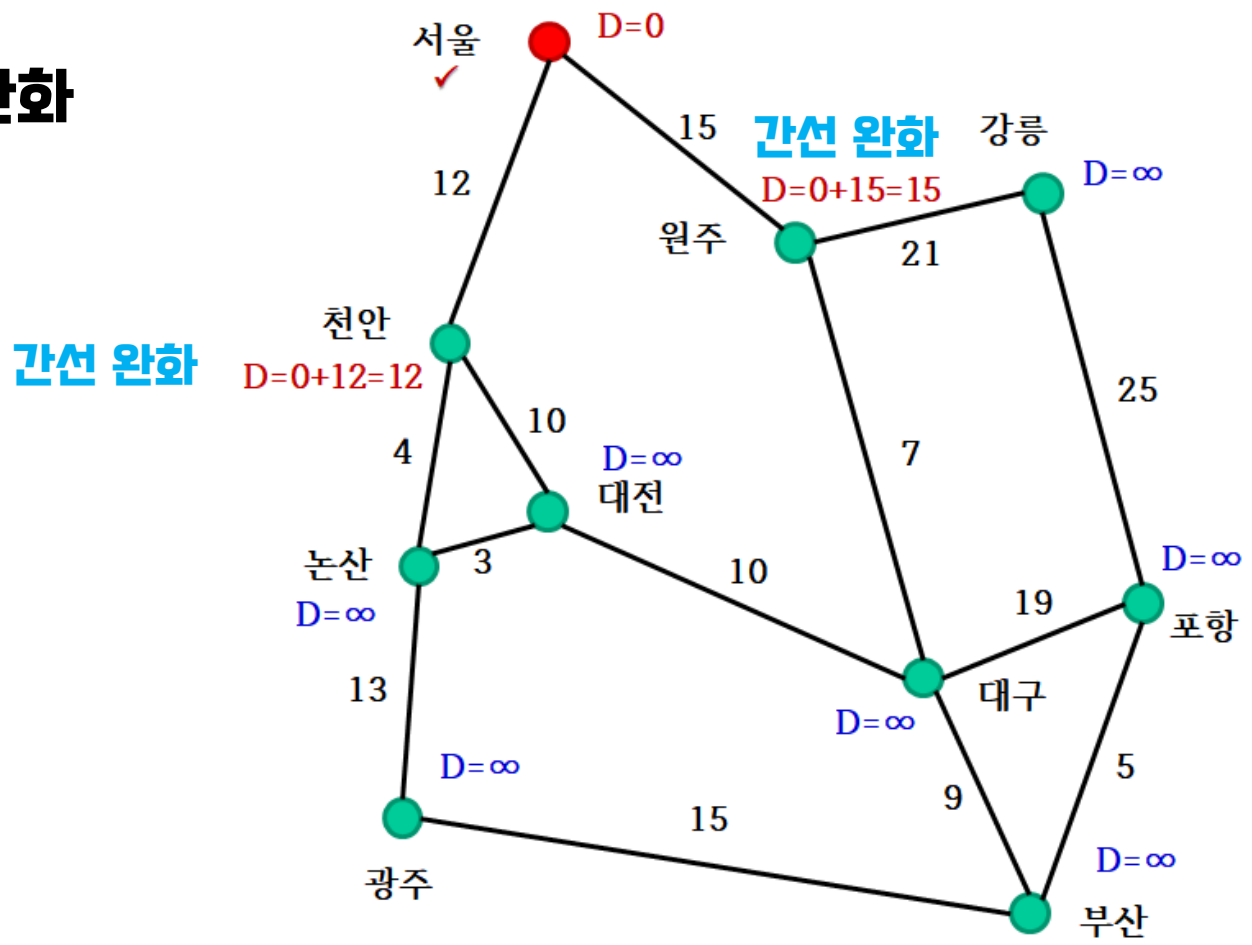
수행 과정

- **서울 확정**



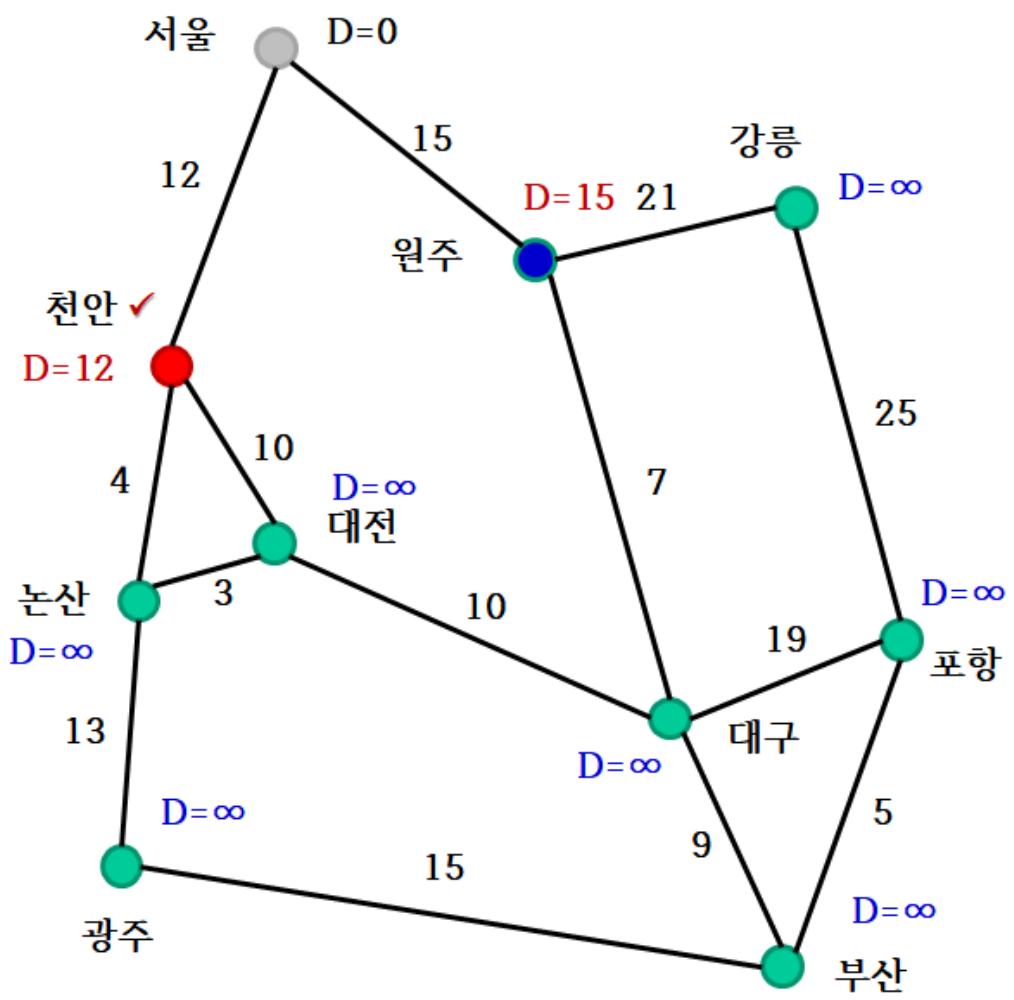
수행 과정

- 간선 완화


서울
D=0
15
간선 완화
D=0+15=15
강릉
D=∞
12
원주
21
천안
간선 완화
D=0+12=12
25
10
4
D=∞
대전
7
논산
3
10
D=∞
D=∞
포항
19
13
대구
D=∞
D=∞
5
9
15
D=∞
광주
부산

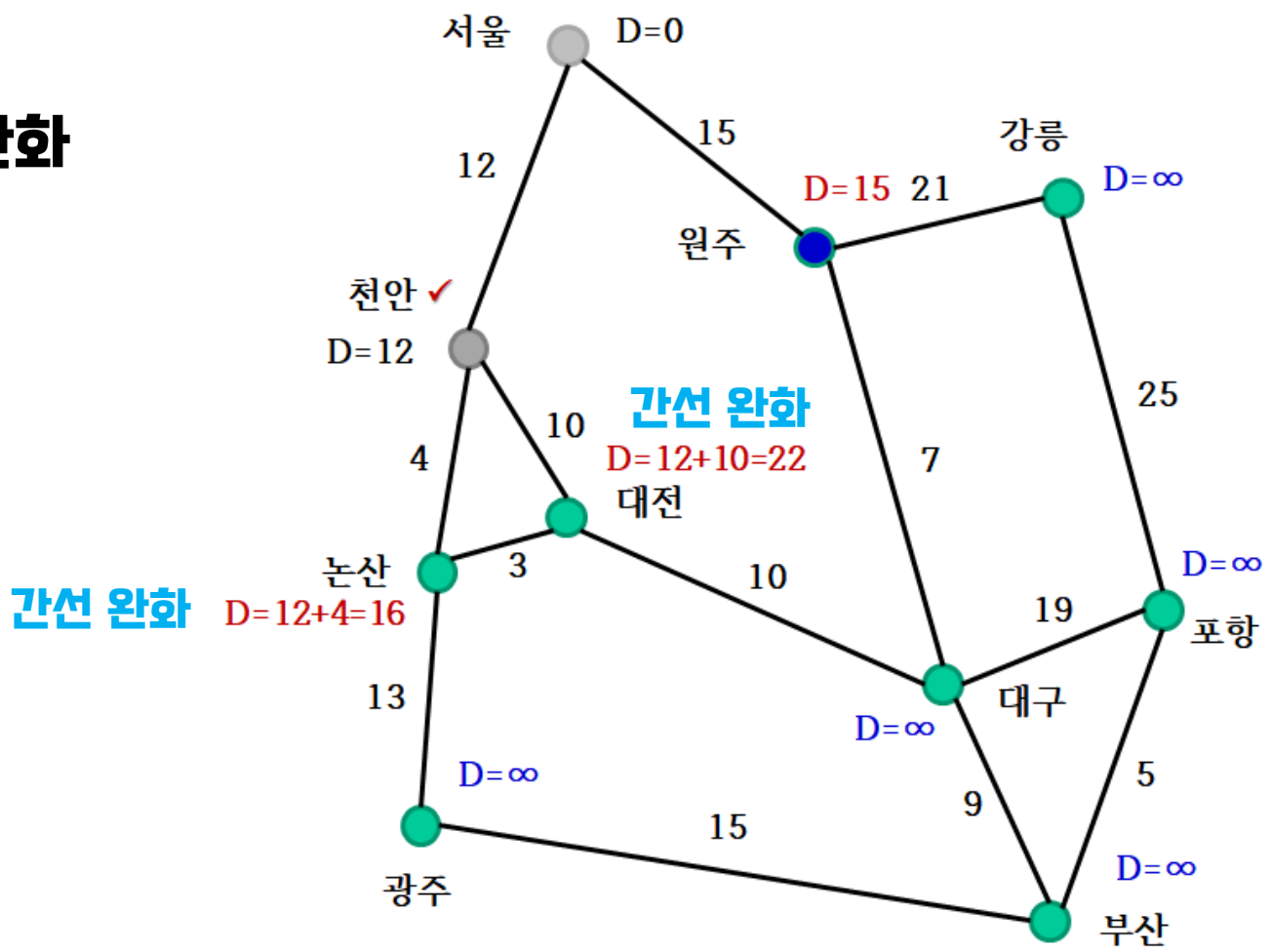
수행 과정

- 천안 확정


서울
D=0
15
12
강릉
D=∞
D=15
21
원주
천안 ✓
D=12
25
10
4
D=∞
대전
7
논산
3
10
D=∞
D=∞
포항
19
13
대구
D=∞
D=∞
5
9
15
D=∞
광주
부산

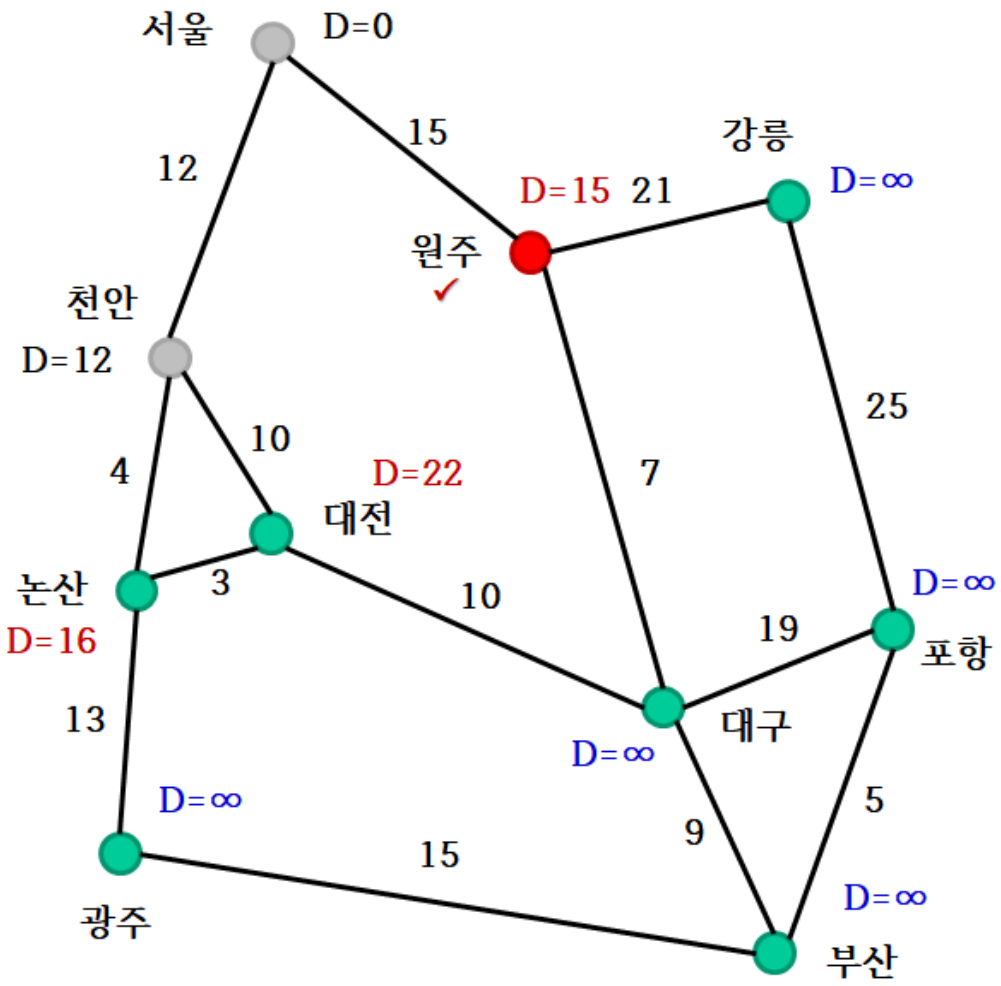
수행 과정

- 간선 완화


서울
D=0
15
12
강릉
D=15
21
D=∞
원주
천안
D=12
간선 완화
D=12+10=22
10
25
4
7
대전
논산
3
10
D=∞
간선 완화
D=12+4=16
19
포항
13
대구
D=∞
D=∞
5
9
15
D=∞
광주
부산

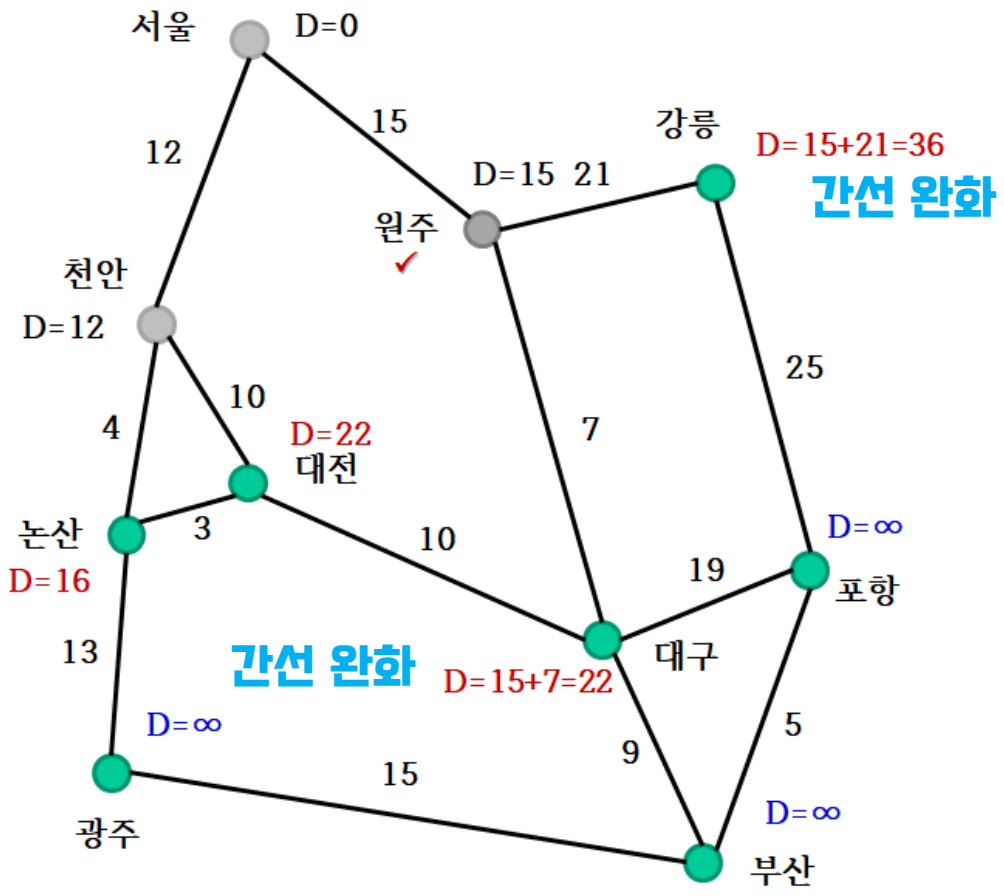
수행 과정

- 원주 확정


서울
D=0
15
12
강릉
D=15
21
D=∞
원주
천안
D=12
25
10
4
D=22
7
대전
논산
3
10
D=∞
D=16
19
포항
13
대구
D=∞
D=∞
5
9
15
D=∞
광주
부산

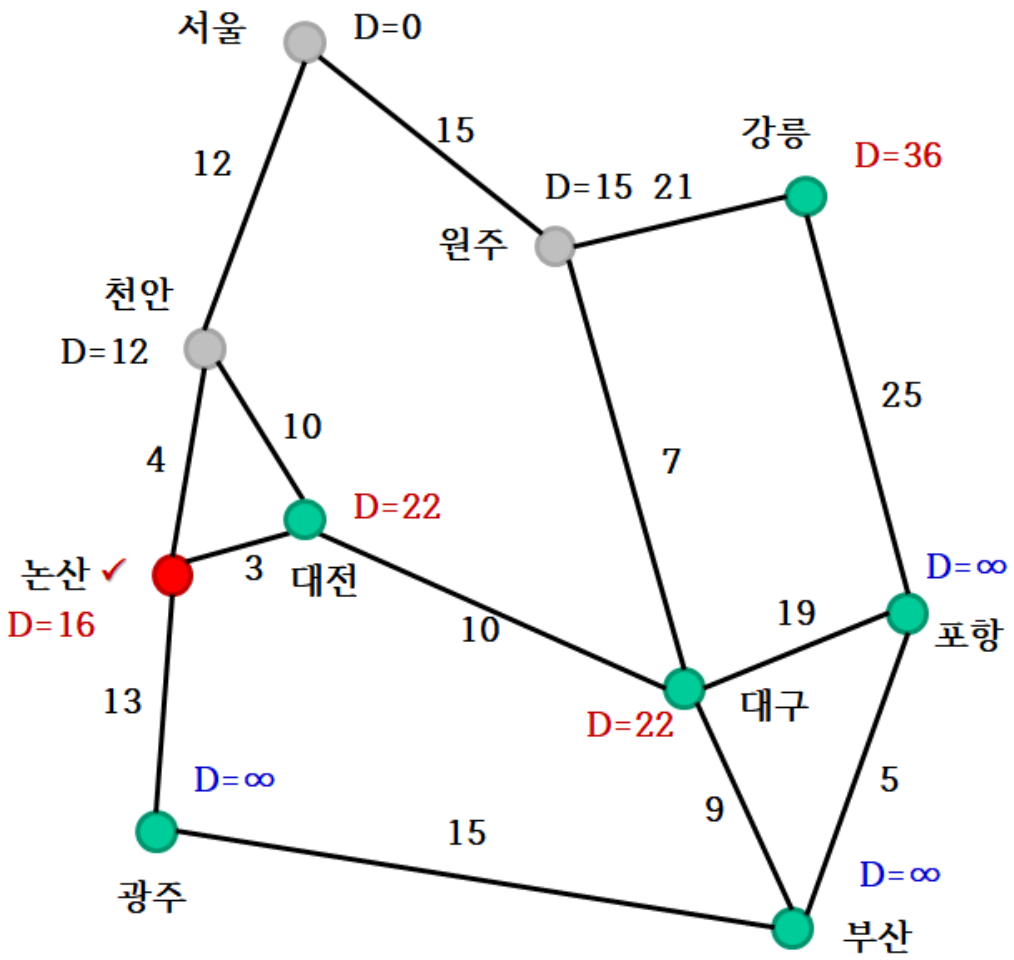
수행 과정

- 간선 완화


서울
D=0
15
12
강릉
D=15+21=36
간선 완화
D=15
21
원주
천안
D=12
25
10
4
D=22
대전
7
논산
3
10
D=∞
D=16
19
포항
13
간선 완화
D=15+7=22
대구
D=∞
5
9
15
D=∞
광주
부산

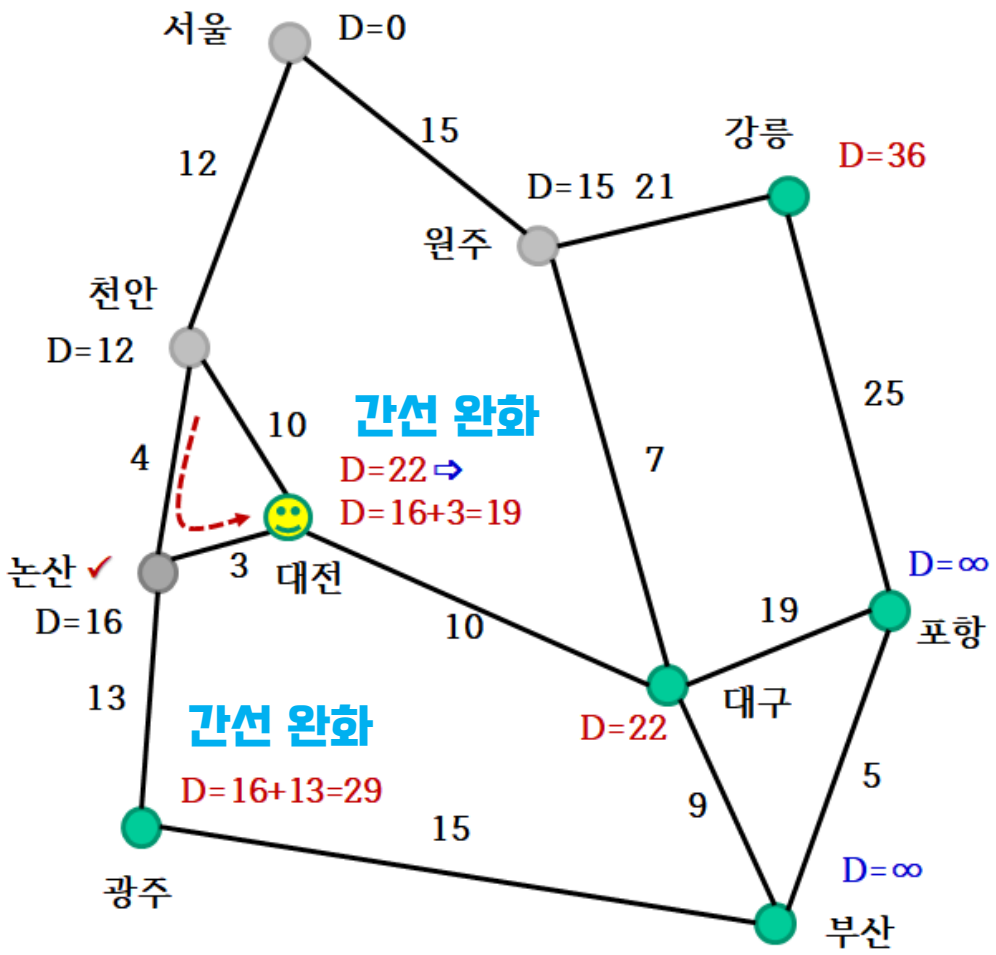
수행 과정

- 논산 확정


서울
D=0
15
12
강릉
D=36
D=15
21
원주
천안
D=12
25
10
7
4
D=22
논산
3
대전
D=∞
D=16
10
19
포항
13
대구
D=22
D=∞
5
9
15
D=∞
광주
부산

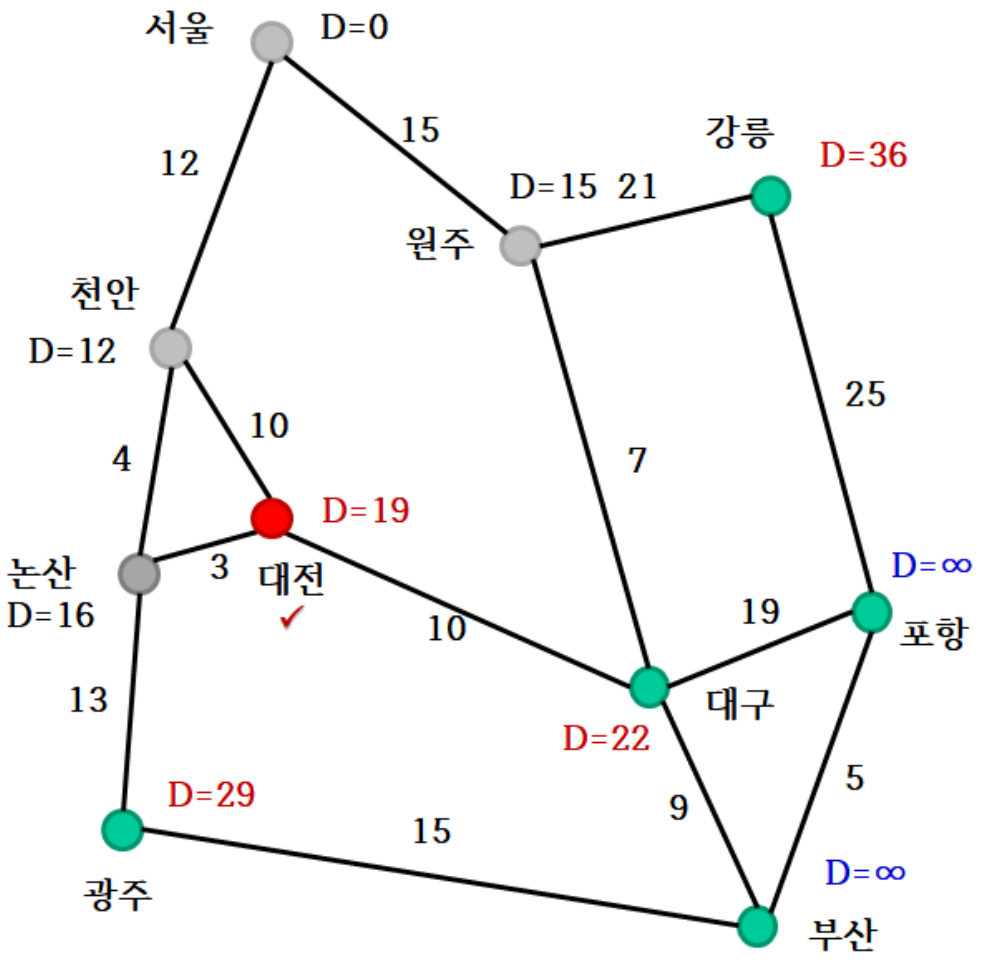
수행 과정

- 간선 완화


서울
D=0
15
12
강릉
D=36
D=15
21
원주
천안
D=12
25
10
간선 완화
4
D=22
7
D=16+3=19
논산
3
대전
D=∞
D=16
10
19
포항
13
대구
간선 완화
D=22
5
D=16+13=29
9
15
D=∞
광주
부산

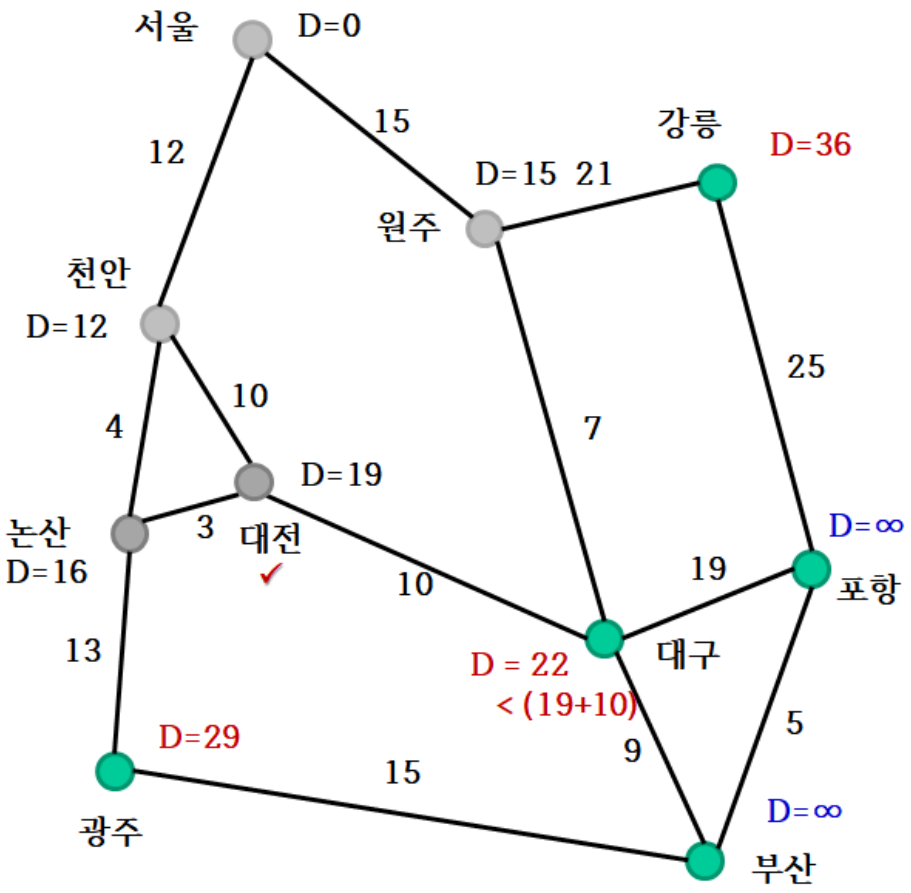
수행 과정

- 대전 확정


서울
D=0
15
12
강릉
D=36
D=15
21
원주
천안
D=12
25
10
4
7
D=19
논산
D=16
3
대전
D=∞
19
10
포항
13
대구
D=22
5
D=29
9
15
D=∞
광주
부산

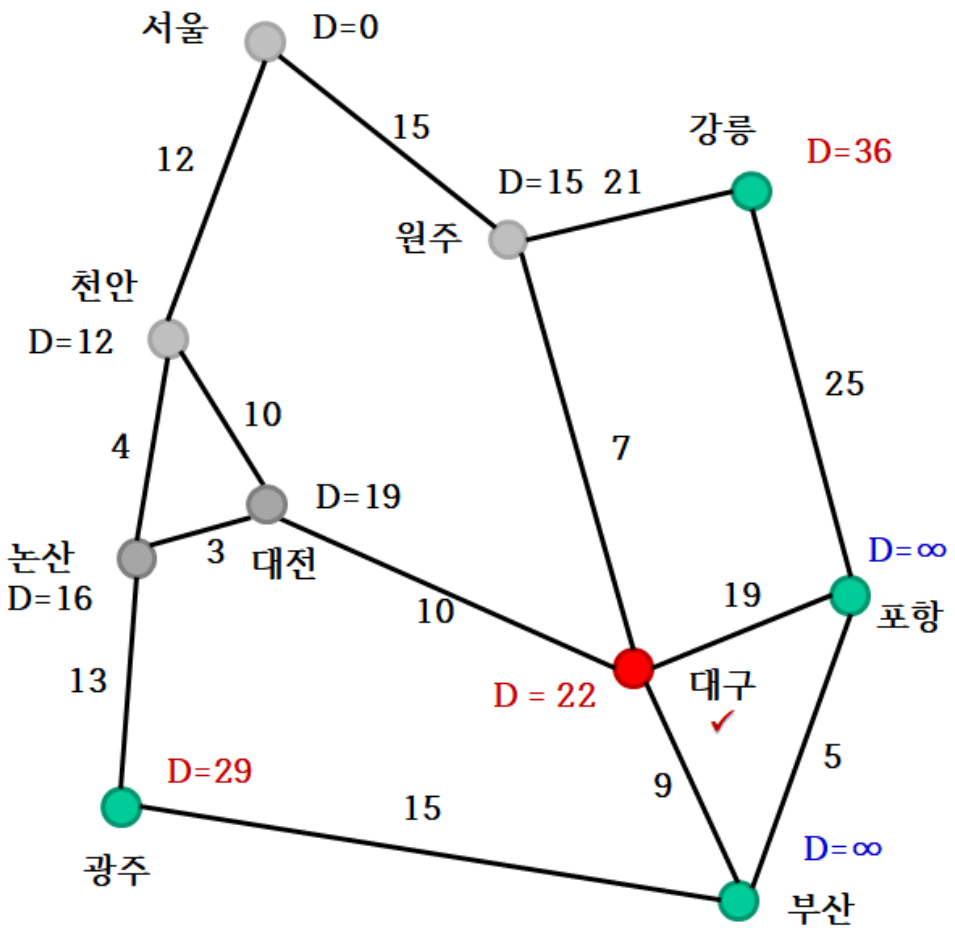
수행 과정

- 간선 완화 없음


서울
D=0
15
12
강릉
D=36
D=15
21
원주
천안
D=12
25
10
4
7
D=19
논산
D=16
3
대전
D=∞
19
포항
10
13
대구
D = 22
< (19+10)
5
D=29
9
15
D=∞
광주
부산

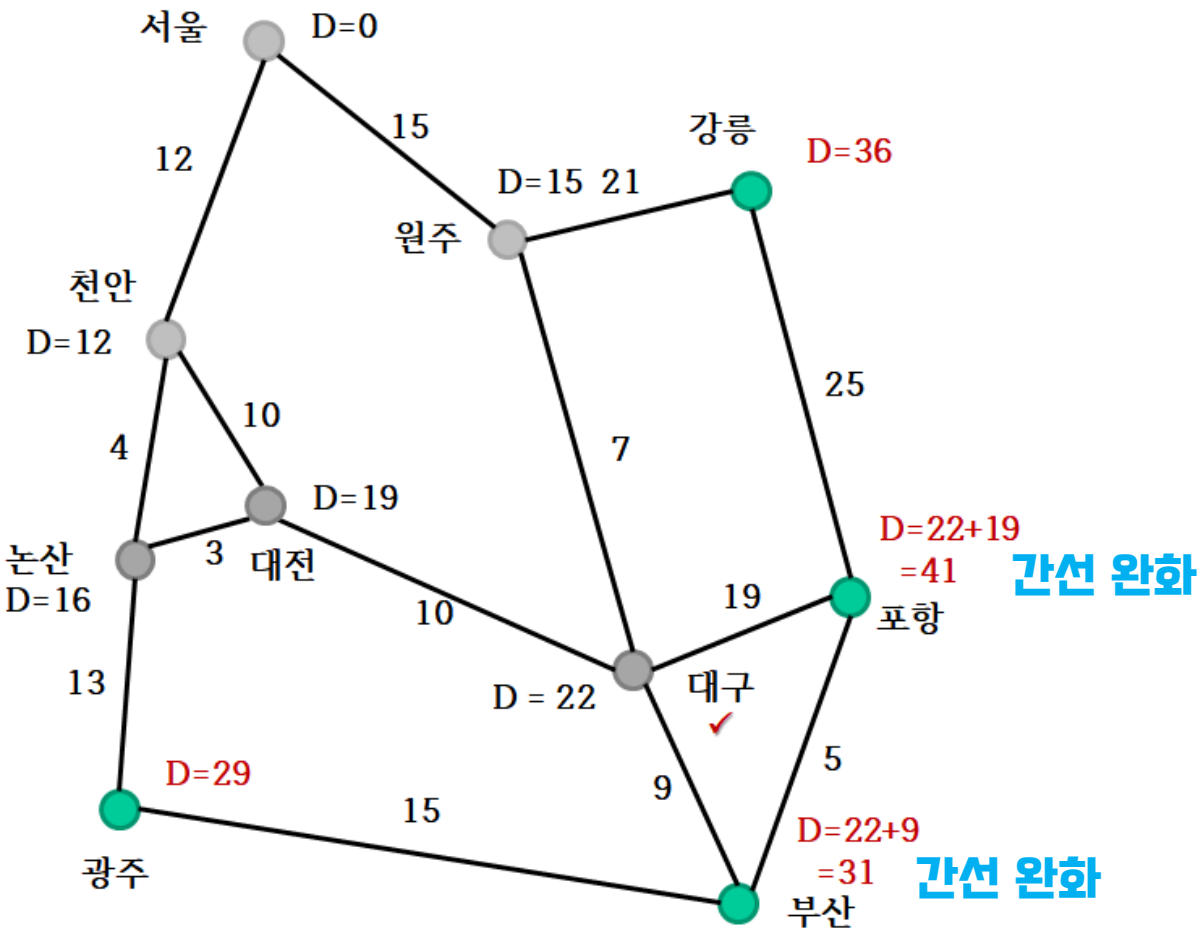
수행 과정

- 대구 확정


서울
D=0
15
12
강릉
D=36
D=15
21
원주
천안
D=12
25
10
4
7
D=19
논산
D=16
3
대전
D=∞
19
포항
10
13
D = 22
대구
5
D=29
9
15
D=∞
광주
부산

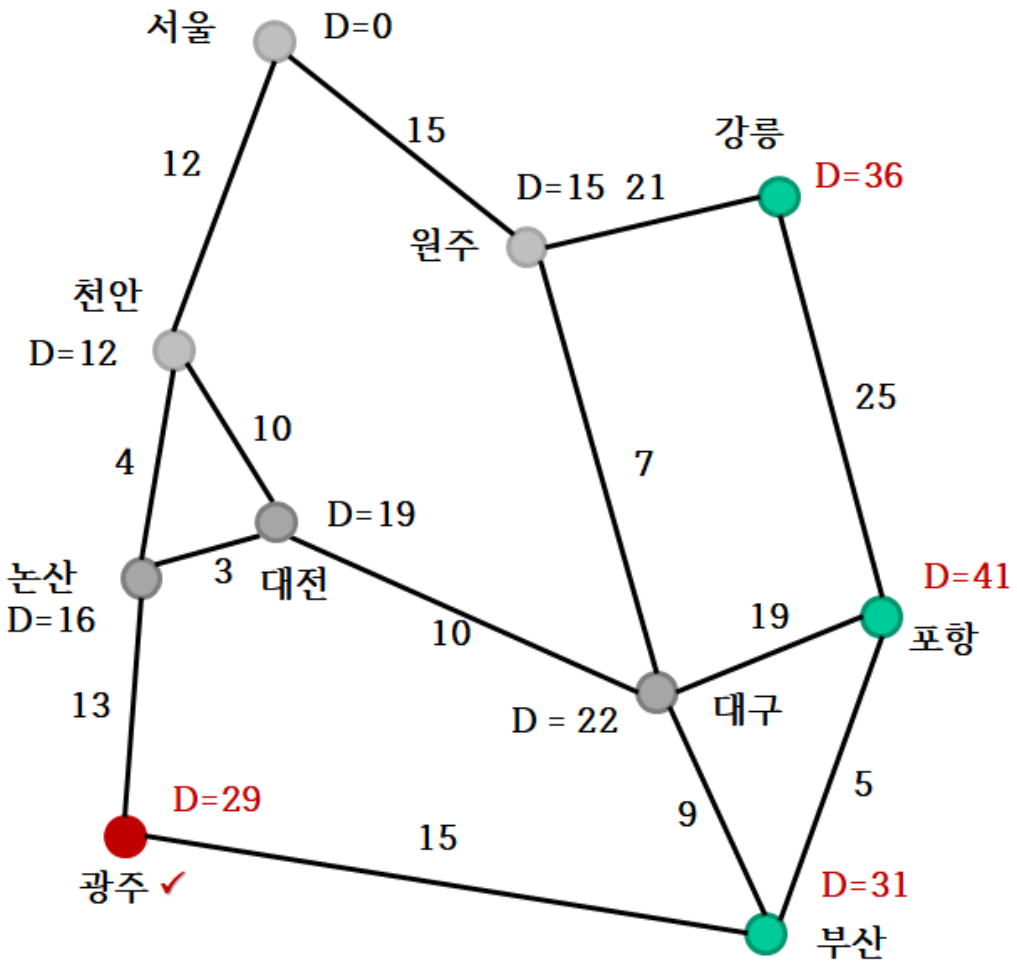
수행 과정

- 간선 완화


서울
D=0
15
12
강릉
D=36
D=15
21
원주
천안
D=12
25
10
4
7
D=19
D=22+19
=41
간선 완화
논산
D=16
3
대전
10
19
포항
13
D = 22
대구
5
D=29
9
15
D=22+9
=31
간선 완화
광주
부산

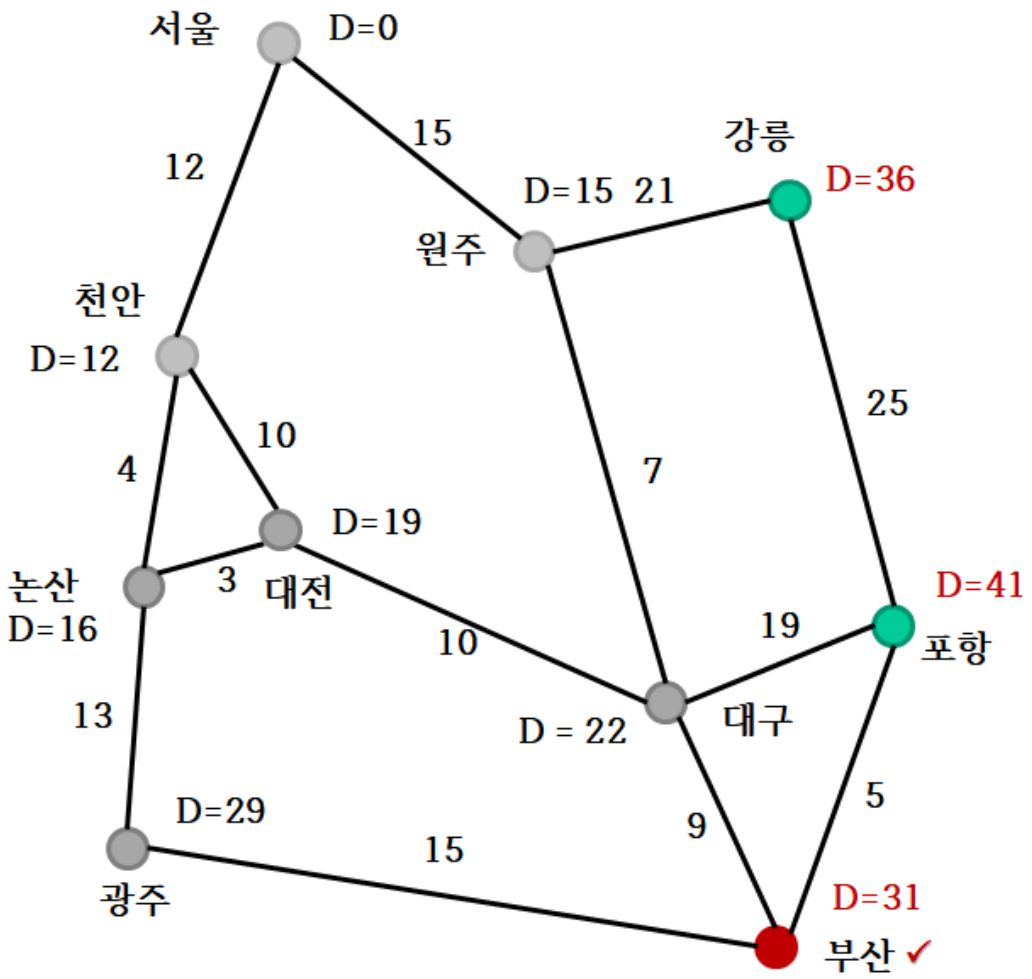
수행 과정

- 광주 확정


서울
D=0
15
12
강릉
D=15
21
D=36
원주
천안
D=12
25
10
7
4
D=19
논산
D=16
3
대전
D=41
19
10
포항
13
D = 22
대구
5
D=29
9
15
광주
D=31
부산

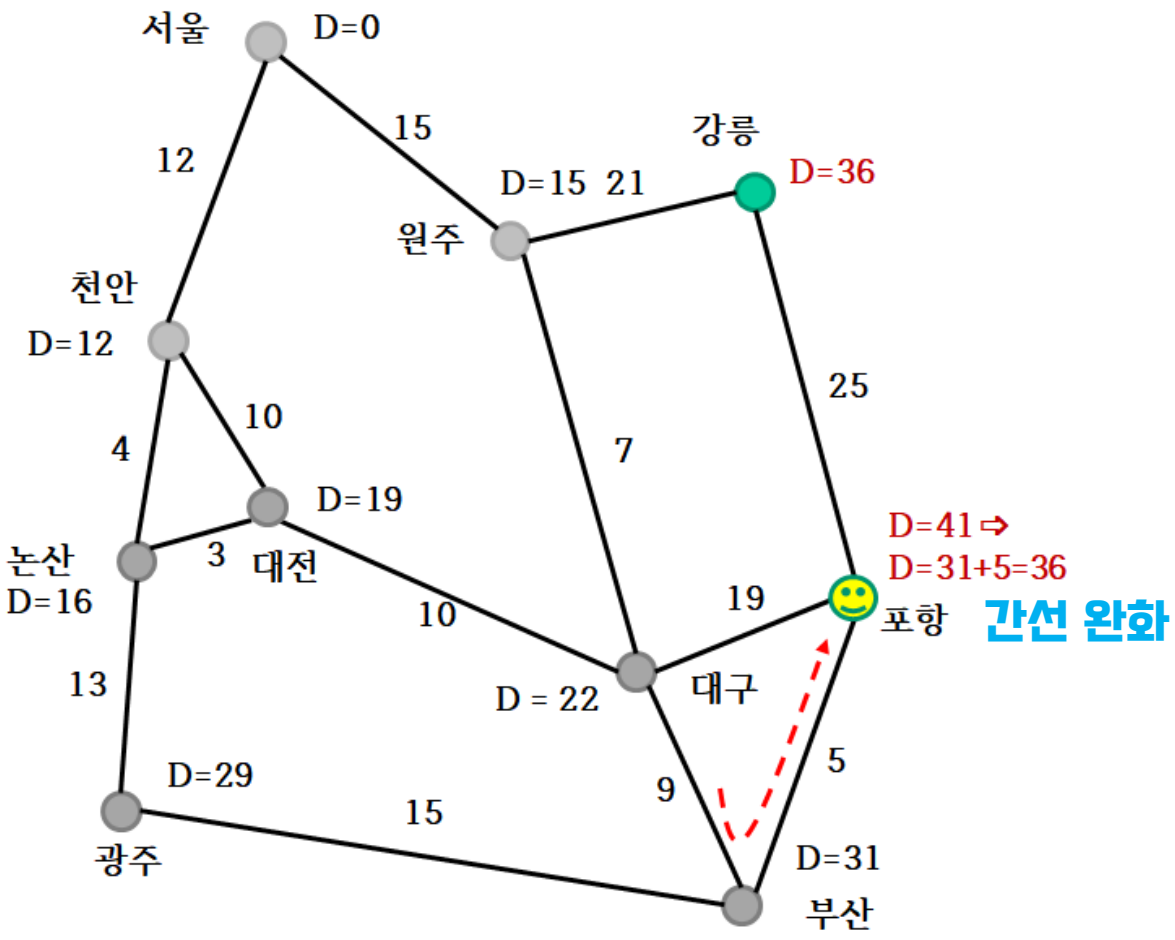
수행 과정

- 부산 확정


서울
D=0
15
12
강릉
D=36
D=15
21
원주
천안
D=12
25
10
4
7
D=19
논산
D=16
3
대전
D=41
10
19
포항
13
D = 22
대구
5
D=29
9
15
광주
D=31
부산

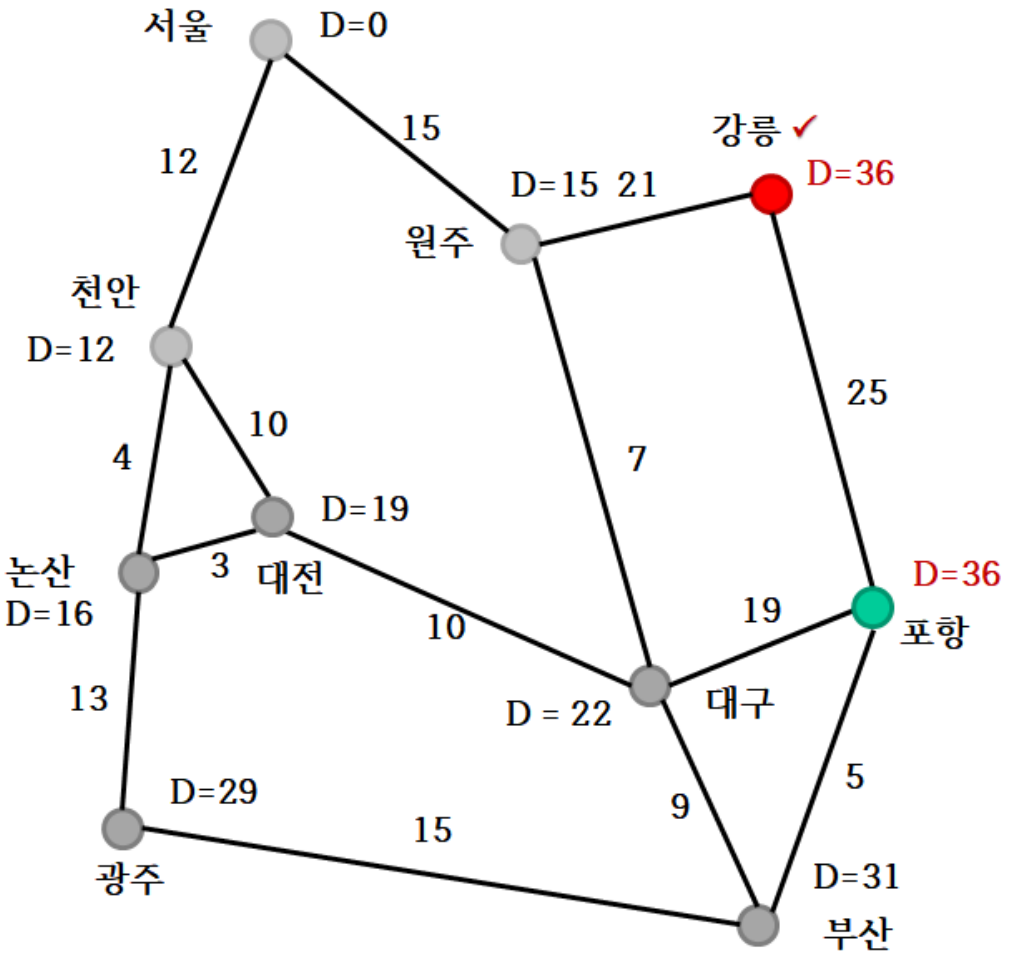
수행 과정

- 간선 완화


서울
D=0
15
12
강릉
D=15
21
D=36
원주
천안
D=12
25
10
4
7
D=19
D=41
D=31+5=36
논산
D=16
3
대전
19
포항
간선 완화
10
13
D = 22
대구
5
D=29
9
15
광주
D=31
부산

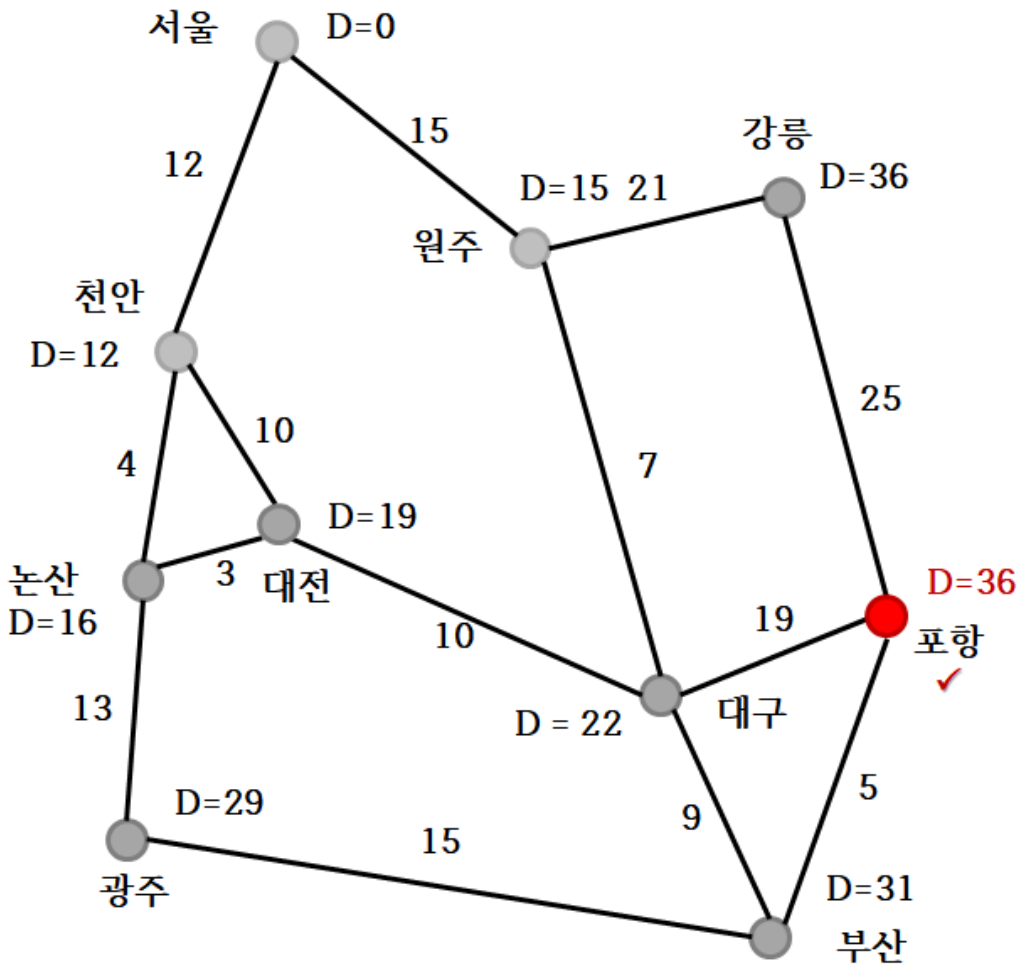
수행 과정

- 강릉 확정

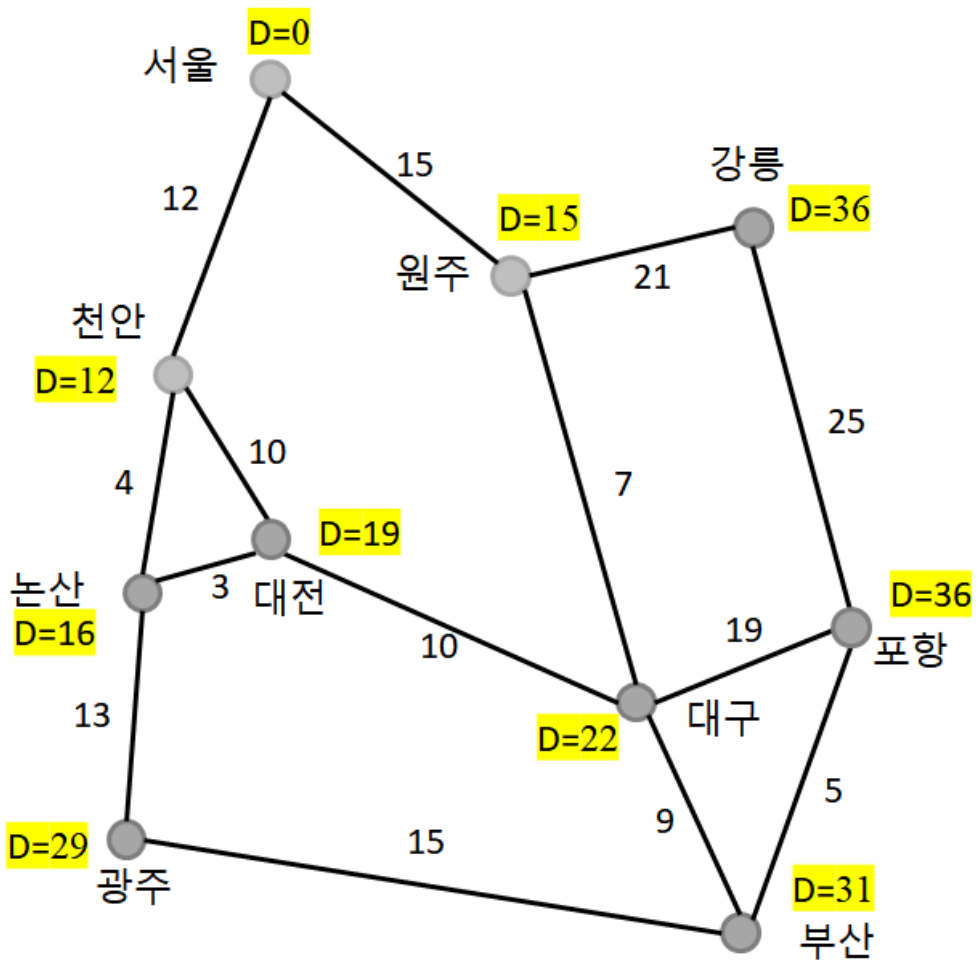

서울
D=0
15
12
강릉 ✓
D=36
D=15 21
원주
천안
D=12
25
10
4
7
D=19
논산
D=16
3
대전
D=36
10
19
포항
13
D = 22
대구
5
D=29
9
15
광주
D=31
부산

수행 과정

- 포항 확정


서울
D=0
15
12
강릉
D=15
21
D=36
원주
천안
D=12
25
10
7
4
D=19
논산
D=16
3
대전
D=36
10
19
포항
13
D = 22
대구
5
D=29
9
15
광주
D=31
부산

ShortestPath 알고리즘의 수행 결과



시간 복잡도

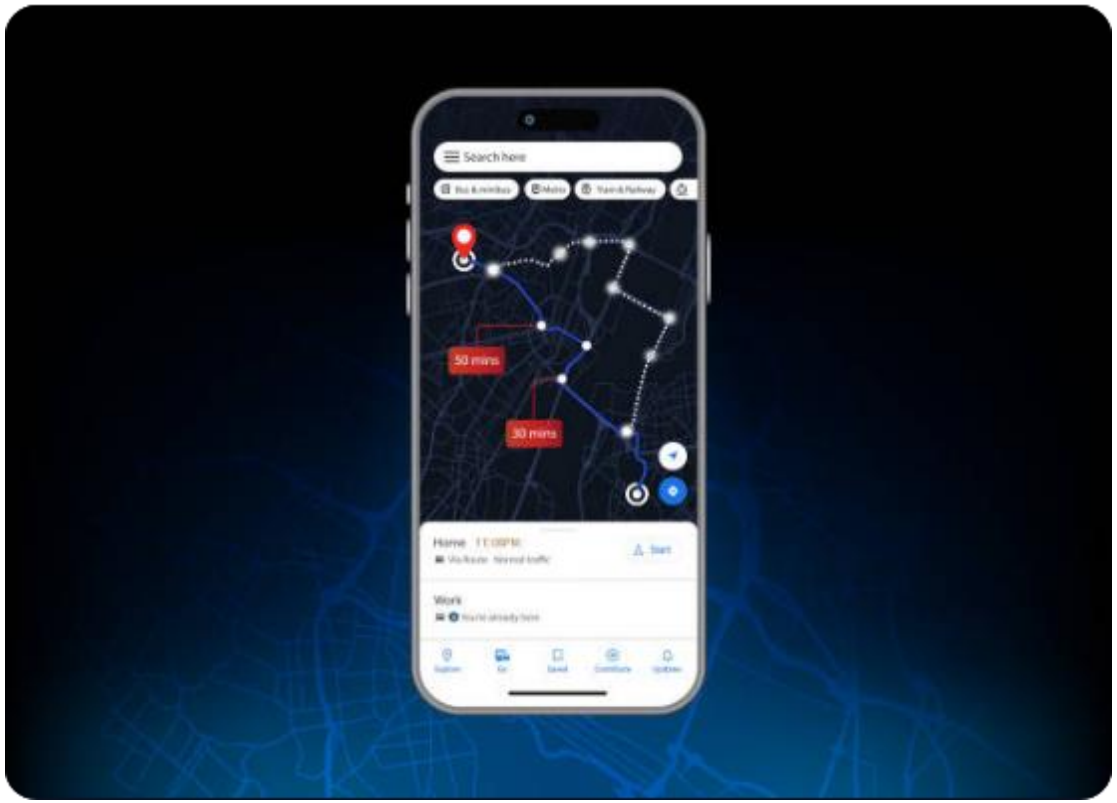
- while-루프가 (n-1)회 반복되고, 1회 반복될 때
  - line 3에서 최소의 D[v]를 가진 점 v_{min}을 찾는데 O(n) 시간 소요
    왜냐하면 배열 D에서 최솟값을 찾기 때문
  - line 4에서도 v_{min}에 연결된 점의 수가 최대 (n-1)개이므로, 각 D[w]를 갱신하는데 걸리는 시간은 O(n)
- ShotestPath의 시간 복잡도는
  - (n-1) x {O(n)+O(n)} = $O(n^2)$
  - 프림 알고리즘과 같이 최소 힙(Binary Heap)을 사용하면 O(mlogn), m은 간선 수. 따라서 간선 수가 O(n)이면 O(nlogn)

응용

- 맵퀘스트 (MapQuest)와 구글 (Google) 맵
- 자동차 네비게이션
- 네트워크와 통신 분야
- 모바일 네트워크
- 산업 공학/경영 공학의 운영 (Operation) 연구
- 로봇 공학
- 교통 공학
- VLSI 디자인 분야 등

사례 1. GPS 기반 차량 내비게이션

- **최적 라우팅**: 카카오내비, T맵, 구글 맵스 등은 출발지부터 목적지까지의 수많은 교차로를 노드(Node)로 인식함
- **동적 가중치 반영**: 실시간 교통 체증, 공사 구간, 제한 속도를 간선(Edge)의 가중치로 환산하여 알고리즘에 대입함
- **그리디 탐색**: 현재 위치에서 도달 가능한 가장 '빠른' 다음 교차로를 연속적으로 선택하여 전체 운행 시간을 극적으로 단축함



사례 2. 운송 및 대중교통 최적화

택배 및 물류 배송 경로

- 물류 창고에서 출발한 배송 차량이 수십 개의 고객 배송지를 방문할 때, 다익스트라 기반의 최단 경로 알고리즘을 활용하여 이동 거리를 최소화함
- 이는 차량의 유류비 절감과 일일 처리 물동량 극대화로 직결됨

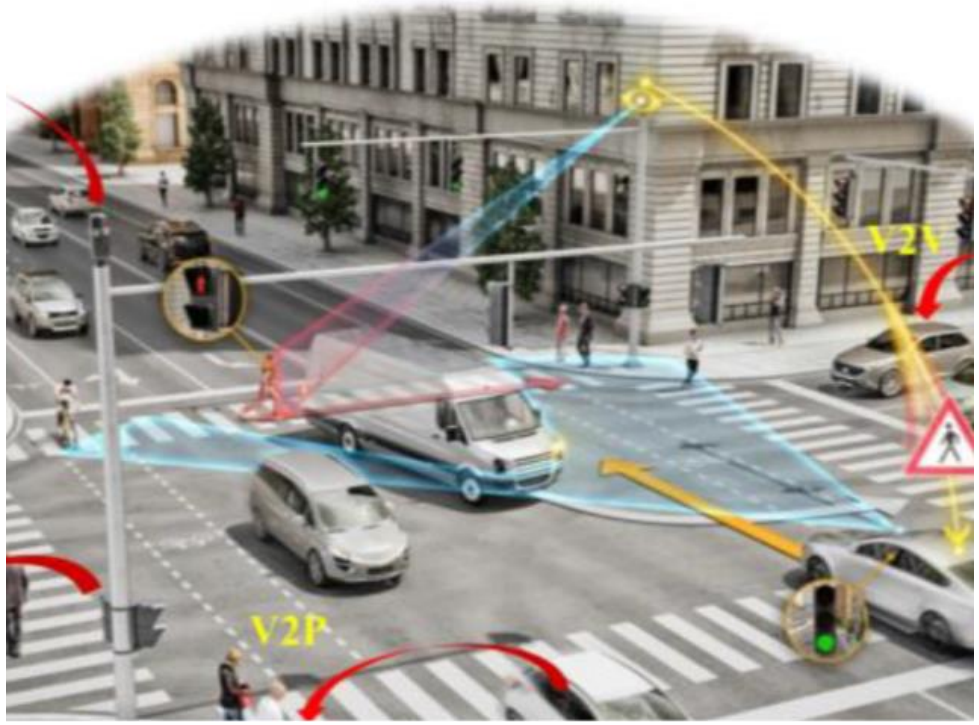
대중교통 환승 안내 시스템

- 지하철, 버스 환승 앱에서 '최소 시간' 또는 '최소 환승' 경로를 찾을 때 활용됨
- 각 역과 정류장을 노드로 삼고 이동 시간을 가중치로 두어, 복잡한 대중교통 망에서 승객을 위한 최적의 솔루션을 제공함

사례 3. 자율주행 및 로보틱스

실시간 동적 경로 탐색

- 자율주행 자동차 및 물류 로봇(AGV)은 LiDAR와 카메라 센서를 통해 실시간으로 3D 지도를 생성함
- 보행자나 갑작스러운 장애물이 등장하면 기존 경로의 가중치를 무한대로 변경하고, 그 즉시 주변의 노드 중 가장 비용이 적은 안전한 우회로를 그리디하게 재탐색하여 목적지까지 자율 주행함



사례 4. 응급 의료 및 소방 배차 시스템

Golden Time

응급 출동 골든타임 확보

생명을 살리는 1분 1초의 최적화

- 스마트 시티의 재난 관제 센터는 화재나 응급 환자 발생 시, 사고 현장에서 가장 가까운 소방차나 구급차를 단순히 물리적 거리가 아닌 '실시간 교통 상황이 반영된 도착 예상 시간(최단 경로)'을 기준으로 배차함
- 막히는 직선 도로보다 우회하더라도 가중치(시간)가 적은 도로를 그리디하게 탐색하여 골든 타임을 사수함

사례 5. 인공지능 및 데이터 분석

게임 내 AI NPC 길찾기 (A* 알고리즘)

- 스타크래프트나 롤(LoL) 같은 게임에서 유닛을 클릭하면 장애물을 피해 최단 거리로 이동함
- 다익스트라 기반에 목적지 방향이라는 휴리스틱(Heuristic)을 더한 A* 탐색을 통해, 게임 내 지형(그리드)을 노드로 삼아 적은 연산량으로 재빠르게 길을 찾아냄

소셜 네트워크 연결 분석

- 링크드인(LinkedIn)의 '1촌, 2촌' 시스템이나 페이스북의 '알 수도 있는 사람' 추천에 활용됨
- 수억 명의 사용자 계정을 노드로 삼아, 사람과 사람 사이의 최단 연결 고리(Degree of separation)를 계산하여 가장 가까운 인맥 경로를 도출함

비즈니스 가치 및 시사점

- **막대한 운영 리소스 절감:** 배송, 통신, 항공 산업 등에서 최단 경로 적용은 연간 수억 달러 규모의 유류비, 인건비, 인프라 비용 절감 효과를 창출함
- **실시간 서비스 구현 (Real-time):** 내비게이션, 게임 AI, 자율주행과 같이 1초 미만의 빠른 연산이 필요한 분야에서 그리디 방식의 직관성과 속도는 필수 불가결함
- **한계 및 보완점:** 음수(-) 가중치가 존재하는 네트워크 환경에서는 그리디 방식(다익스트라)이 오작동 할 수 있으므로, 벨만-포드 등 상황에 맞는 아키텍처 설계가 요구됨
- **초연결 사회의 근간:** 물리적 도로망부터 보이지 않는 IT 네트워크 통신망까지, 세상의 모든 연결을 가장 효율적으로 만드는 현대 기술의 핵심 뼈대임

파이썬 구현 패턴 1 : 기본 다익스트라 (우선순위 큐 활용)

- 가장 표준적이고 효율적인 형태
- 최소 힙(heapq)을 사용하여 매번 가장 가까운 노드를 꺼내오므로 속도가 빠름
- 통신망에서 패킷이 전송될 최소 지연 시간을 구할 때 주로 사용

```
# 네트워크 라우팅 맵 예시
network_graph = {
    'A': {'B': 2, 'C': 5}, 'B': {'A': 2, 'C': 1, 'D': 4},
    'C': {'A': 5, 'B': 1, 'D': 2, 'E': 6}, 'D': {'B': 4, 'C': 2, 'E': 3},
    'E': {'C': 6, 'D': 3}
}
print(f"[예제 1] A노드 출발 최단 거리: {dijkstra_pq(network_graph, 'A')}")
```

파이썬 구현 패턴 1 :
기본 다익스트라 (우선순위 큐 활용)

```python
import heapq
def dijkstra_pq(graph, start):
    # 각 노드까지의 거리를 무한대로 초기화
    distances = {node: float('inf') for node in graph}
    distances[start] = 0

    # 우선순위 큐: (거리, 노드)
    queue = [(0, start)]
    while queue:
        current_dist, current_node = heapq.heappop(queue)

        # 이미 처리된 노드(더 짧은 경로가 발견된 경우)는 무시
        if distances[current_node] < current_dist:
            continue

        # 인접 노드 확인
        for neighbor, weight in graph[current_node].items():
            distance = current_dist + weight

            # 더 짧은 경로를 발견하면 거리 갱신 및 큐에 추가
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(queue, (distance, neighbor))

    return distances
```

파이썬 구현 패턴 2 : 2차원 공간(Grid)에서의 최단 경로 (장애물 회피)

- 공장 평면도처럼 행렬로 구성된 2차원 맵에서 로봇이 상하좌우로 이동하며 목적지를 찾는 알고리즘 (1은 벽, 0은 통로)

```
# 0: 통로, 1: 기계/장벽

factory_grid = [
    [0, 0, 0, 1, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1],
    [0, 0, 0, 0, 0]
]
print(f"\n[예제 2] 2D 공장 맵 이동 최소 칸 수: {grid_dijkstra(factory_grid, (0, 0), (4, 4))}")
```

파이썬 구현 패턴 2 :

2차원 공간(Grid)에서의 최단 경로 (장애물 회피)

```python
import heapq

def grid_dijkstra(grid, start, end):
    rows, cols = len(grid), len(grid[0])
    distances = [[float('inf')] * cols for _ in range(rows)]
    distances[start[0]][start[1]] = 0
    queue = [(0, start[0], start[1])]

    # 이동 방향: 상, 하, 좌, 우
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        dist, r, c = heapq.heappop(queue)

        if (r, c) == end: return dist
        if distances[r][c] < dist: continue

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
              # 맵 범위 내에 있고, 벽(1)이 아닌 경우만 이동 가능
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 0:
                cost = dist + 1  # 이동 비용은 1칸당 1로 가정
                if cost < distances[nr][nc]:
                    distances[nr][nc] = cost
                    heapq.heappush(queue, (cost, nr, nc))

    return -1  # 도달 불가
```

4.4 부분 배낭 문제

- 배낭 (Knapsack) 문제
  - n개의 물건이 각각 1개씩 있고,
  - 각 물건은 무게와 가치를 가지고 있으며,
  - 배낭이 한정된 무게의 물건들을 담을 수 있을 때,
  - 최대의 가치를 갖도록 배낭에 넣을 물건들을 정하는 문제
- 부분 배낭 (Fractional Knapsack) 문제
  - 물건을 부분적으로 담는 것을 허용
  - 그리디 알고리즘으로 해결
- 0-1 배낭 문제
  - 부분 배낭 문제의 원형으로 물건을 통째로 배낭에 넣어야 함
  - 동적 계획 알고리즘, 백트래킹 기법, 분기 한정 기법으로 해결

아이디어

- 부분 배낭 문제에서는 물건을 부분적으로 배낭에 담을 수 있으므로, 최적해를 위해서 '욕심을 내어' 단위 무게 당 가장 값나가는 물건을 배낭에 넣고, 계속해서 그 다음으로 값나가는 물건을 넣음

- 만일 물건을 '통째로' 배낭에 넣을 수 없으면, 배낭에 넣을 수 있을 만큼만 물건을 부분적으로 배낭에 담음

4.4 부분 배낭 문제

단위 무게당 가치의 극대화

" 통째로만 담아야 하는 0/1 배낭 문제와 달리,
자원을 원하는 만큼 쪼개어(Fractional) 담을 수 있을 때,
그리디 알고리즘은 항상 완벽한 최적해를 도출 "

부분 배낭 문제의 작동 원리

- **핵심 지표 산출:** 모든 물건(자원)에 대해 무게 대비 가치, 즉 $\frac{\text{Value}}{\text{Weight}}$ 를 계산함
- **내림차순 정렬:** 단위 가치가 가장 높은 자원부터 순서대로 정렬함
- **탐욕적 적재 및 분할:** 가방(제한 용량)에 들어갈 수 있는 만큼 최대한 통째로 담고, 마지막에 남은 공간은 다음 물건을 잘라서(Fraction) 가득 채움

알고리즘

FractionalKnapsack

입력: n개의 물건, 각 물건의 무게와 가치, 배낭의 용량 C

출력: 배낭에 담은 물건 리스트 L과 배낭 속의 물건 가치의 합 v

1. 각 물건에 대해 단위 무게 당 가치를 계산한다.
2. 물건들을 단위 무게 당 가치를 기준으로 내림차순으로 정렬하고, 정렬된 물건 리스트를 S라고 하자.
3. L=∅, w=0, v=0

   // L은 배낭에 담을 물건 리스트, w는 배낭에 담긴 물건들의 무게의 합, v는 배낭에 담긴 물건들의 가치의 합
4. S에서 단위 무게 당 가치가 가장 큰 물건 x를 가져온다.

알고리즘

```
while  w + (x의 무게) ≤ C
       x를 L에 추가
       w = w + (x의 무게)
       v = v + (x의 가치)
       x를 S에서 제거
       S에서 단위 무게 당 가치가 가장 큰 물건 x를 가져온다.
If  C-w > 0   // 배낭에 물건을 부분적으로 담을 여유가 있으면
       물건 x를 (C-w) 만큼만 L에 추가
       v = v + (C-w) 만큼의 x의 가치
return L, v
```

수행 과정

- 배낭의 최대 용량 = 40그램

- 단위 무게 당 가치로 정렬: S=[백금, 금, 은, 주석]

물건	단위 그램당 가치
백금	6만원
금	5만원
은	4천원
주석	1천원

수행 과정

- 백금을 통째로 담는다.
- 배낭에 담긴 물건(들)의 무게 w = 10, 얻는 가치 v = 60

60만원

- 금을 통째로 담는다.
- 배낭에 담긴 물건(들)의 무게 w = 25,
  v = 60+ 75 = 135

75만원

수행 과정

- 은을 통째로 담으려 하지만
- 배낭에 담긴 물건(들)의 무게 w = 25 + 25 = 50이 되어 배낭 용량 초과

10만원

- 은을 40 -25 = 15 만큼만 담음
- 배낭에 담긴 물건(들)의 무게 w = 40, v = 135 + (0.4x15) = 141 만원

시간 복잡도

- Line 1: n개의 물건 각각의 단위 무게 당 가치를 계산하는 데는 O(n) 시간 소요
- Line 2: 물건의 단위 무게 당 가치에 대해서 정렬하기 위해 O(n logn) 시간 소요
- Line 5~10: while-루프의 수행은 n번을 넘지 않으며, 루프 내부의 수행은 O(1) 시간 소요
- Line 11~14: 각각 O(1) 시간 소요

- 알고리즘의 시간 복잡도: O(n) + O(n logn) + n×O(1) + O(1) = O(n logn)

응용

- 0-1 배낭 문제는 최소의 비용으로 자원을 할당하는 문제로서, 조합론, 계산이론, 암호학, 응용 수학 분야에서 기본적인 문제로 다뤄짐
- '버리는 부분 최소화하는' 원자재 자르기
- 자산투자 및 금융 포트폴리오에서의 최선의 선택
- Merkle-Hellman 배낭 암호 시스템에 사용

사례 1 : 원자재 운송 및 금융 자산 배분

벌크 화물 운송 최적화

- 화물선이나 트럭에 적재 중량 제한이 있을 때, 금가루, 밀가루, 철광석, 원유 등 무게 단위로 쪼갤 수 있는 화물 중 1kg당 단가가 가장 높은 광물/곡물부터 공간을 채움
- 남는 자투리 공간에는 그 다음으로 가치 있는 화물을 분할 적재하여 운송 수익을 극대화함

금융 포트폴리오 자산 배분

- 제한된 예산으로 투자를 진행할 때, 최근 도입된 주식 소수점 거래(Fractional Shares)나 펀드를 활용함
- 기대 투자수익률(ROI)이 가장 높은 자산부터 우선 매수하고, 남는 자투리 금액까지 다음 순위 자산의 소수점 지분을 매입해 예산의 낭비를 없앰

사례 2. IT 및 에너지 자원 프로비저닝

클라우드 컴퓨팅 자원

- 물리 서버의 제한된 CPU와 RAM을 가상화하여 분할 (vCPU)
- 클라우드 사업자는 1vCPU당 과금(수익)이 가장 높은 엔터프라이즈 작업부터 자원을 우선 할당함

네트워크 대역폭 할당 (QoS)

- 제한된 네트워크 대역폭 (Mbps)을 바이트(Byte) 단위로 쪼개어, VoIP나 화상회의 같이 1바이트당 중요도(패킷 우선순위)가 높은 트래픽부터 대역폭을 우선 점유하도록 분배함

스마트 그리드 전력 분배

- ESS(에너지 저장 장치)의 잔여 전력이 부족할 때, 전력을 W(와트) 단위로 분할하여 병원 등 단위 전력당 생존 효용 가치가 가장 높은 필수 시설에 먼저 송전함

사례 3. 디지털 광고 인벤토리 할당

eCPM

유효 1천 회 노출당 단가

밀리초(ms) 단위의 지면 쪼개기

- 스트리밍 플랫폼이나 웹사이트는 한정된 광고 송출 시간과 지면 공간을 가짐
- 퍼블리셔(매체)는 실시간 입찰(RTB) 시스템을 통해 초당 수익(CPM/eCPM)이 가장 높은 광고주의 광고를 먼저 노출시킴
- 남는 1초, 2초의 짧은 슬롯(분할된 공간/시간)조차도 다음 순위 단가의 숏폼 광고를 송출하여 광고 공간의 가치를 100% 소진함

비즈니스 가치 및 시사점

- **최상의 연산 속도:** 복잡한 동적 계획법(DP) 테이블을 그릴 필요 없이, 단순 정렬과 나눗셈만으로 완벽한 최적해를 도출하는 압도적인 퍼포먼스를 자랑함
- **연속적(Continuous) 자원의 지배자:** 액체, 곡물, 시간, 클라우드 컴퓨팅 자원, 데이터 대역폭 등 '소수점 단위로 분할 가능한' 모든 비즈니스 모델에 즉각 적용 가능합니다.
- **핵심 지표의 발견:** 이 모델의 성공은 개별 가치나 무게가 아닌, '단위 무게당 가치(효율성)'라는 파생 지표를 도출하고 이를 기준으로 우선순위를 재편하는 데 있습니다.
- **현실적 제약:** 단, 사람, 자동차, 스마트폰처럼 절대 물리적으로 쪼갤 수 없는 개별 물품(Discrete) 단위의 물류 환경에서는 0/1 배낭 문제의 접근법을 사용해야 합니다.

파이썬 구현 패턴 1 : 기본 리스트 활용 구현 (가장 단순한 형태)

- 가장 기본적인 형태로, (가치, 무게) 형태의 튜플 리스트를 받아 무게당 가치를 기준으로 정렬한 뒤 배낭을 채움

```
# (가치, 무게)

items_data = [(60, 10), (100, 20), (120, 30)]
max_capacity = 50
print(f"[예제 1] 기본 배낭 최대 가치: {fractional_knapsack_basic(max_capacity, items_data)}")
```

파이썬 구현 패턴 1 : 기본 리스트 활용 구현 (가장 단순한 형태)

```
def fractional_knapsack_basic(capacity, items):

    # 1. 무게당 가치(단가)를 기준으로 내림차순 정렬 (가치 / 무게)
    items.sort(key=lambda x: x[0] / x[1], reverse=True)

    total_value = 0.0

    for value, weight in items:
        if capacity >= weight:
            # 배낭 용량이 충분하면 물건을 통째로 넣음
            total_value += value
            capacity -= weight
        else:
            # 배낭 용량이 부족하면 남은 용량만큼 물건을 쪼개서 넣음
            fraction = capacity / weight
            total_value += value * fraction
            break  # 배낭이 꽉 찼으므로 종료

    return total_value
```

파이썬 구현 패턴 2 : 스마트 팩토리 AGV 원자재 적재 최적화

- 공장 내 무인 운반차(AGV)가 한 번에 옮길 수 있는 적재 중량이 제한되어 있을 때, 가치가 높은 액체나 분말 형태의 원자재(분할 가능)를 우선적으로 실어 나르는 시나리오

```
# 응용 예제
factory_materials = {
    '특수합금분말': {'value': 5000, 'weight': 50},
    '윤활유': {'value': 1000, 'weight': 40},
    '냉각수': {'value': 400, 'weight': 80},
    '촉매제': {'value': 3000, 'weight': 20}
}
agv_max_weight = 80
val, details = agv_loading_optimization(agv_max_weight, factory_materials)
print(f"\n[예제 2] AGV 최대 이익: {val}")
print(f"AGV 적재 내역(무게): {details}")
```

파이썬 구현 패턴 2 :
스마트 팩토리 AGV 원자재 적재 최적화

```python
def agv_loading_optimization(agv_capacity, materials):
    # 재료 딕셔너리를 리스트로 변환 및 단가 계산 후 정렬
    # materials = { '재료명': {'value': 가치, 'weight': 무게} }
    sorted_materials = sorted(
        materials.items(),
        key=lambda x: x[1]['value'] / x[1]['weight'],
        reverse=True
    )

    total_loaded_value = 0
    loaded_details = {}

    for name, info in sorted_materials:
        v, w = info['value'], info['weight']
        if agv_capacity >= w:
            total_loaded_value += v
            agv_capacity -= w
            loaded_details[name] = w  # 무게 단위로 기록
        else:
            total_loaded_value += v * (agv_capacity / w)
            loaded_details[name] = agv_capacity  # 남은 용량만큼만 적재
            break

    return total_loaded_value, loaded_details
```

4.5 집합 커버 문제

- 문제
  - n개의 원소를 가진 집합 U가 있고,
  - U의 부분집합들을 원소로 하는 집합 F가 주어질 때,
  - F의 원소들인 집합들 중에서 어떤 집합들을 선택하여 합집합하면 U와 같게 되는가?

- 집합 커버 (Set Cover) 문제
  - 집합 F에서 선택하는 집합들의 수를 최소화하는 문제

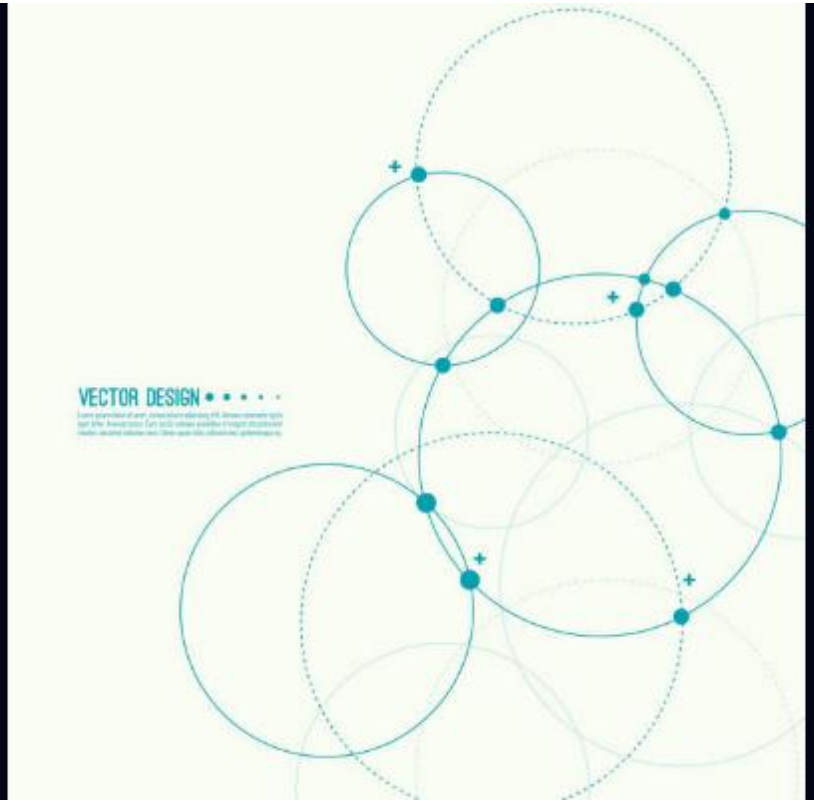
4.5 집합 커버 문제

전체 집합의 완벽한 커버리지

“ 수많은 요구사항(전체 집합)을 충족하기 위해,
가장 많은 미해결 요소를 포함하는 집합(Set)을 매 순간 '그리디'하게 선택
최소 비용으로 100% 커버리지를 달성하는 해법을 제시 “

집합 커버 문제의 핵심 동작 원리

- **커버리지 측정**: 각 옵션(집합)이 현재 처리되지 않은 요소(Element)를 얼마나 많이 포함하고 있는지 파악함
- **최대 효율의 반복 선택**: 가장 많은 미커버 요소를 포함하는 옵션을 1순위로 선택(Greedy) 함
- **잔여 요소 갱신**: 선택된 옵션이 커버한 요소들을 전체 집합에서 지워나가며, 남은 요소가 '0'이 될 때까지 이 과정을 반복함

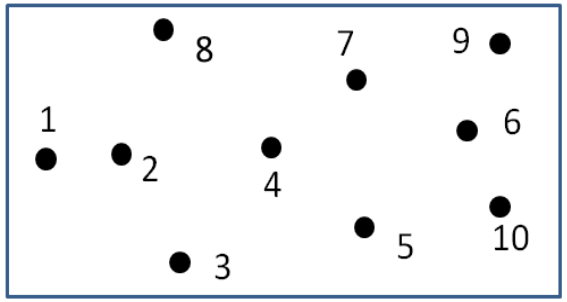


신도시 학교 배치

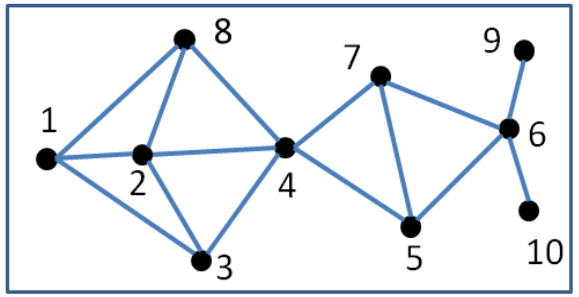
- 신도시를 계획하는 데 있어서 학교 배치의 예
  - 10개의 마을이 신도시에 만들어질 계획이다.
  - 다음 조건이 만족되도록 학교 위치를 선정해야 한다.

    학교는 마을에 위치해야 한다.

    등교 거리는 걸어서 15분 이내이어야 한다.
  - 어느 마을에 학교를 신설해야 학교의 수가 최소가 되는가?



10개 마을의 위치



등교 거리가 15분 이내인 마을 간의 관계

최적해

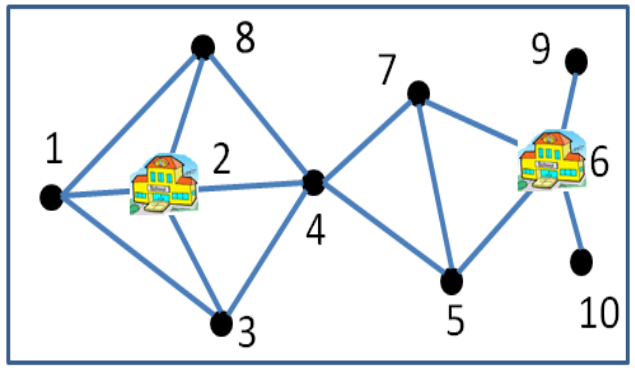
- 어느 마을에 학교를 신설해야 학교의 수가 최소가 되는가?
  - 2번 마을에 학교를 만들면

    1, 2, 3, 4, 8 마을의 학생들이 15분 이내에 등교 가능

    즉, 마을 1, 2, 3, 4, 8이 커버된다
  - 6번 마을에 학교를 만들면

    마을 5, 6, 7, 9, 10이 커버된다
  - 2번과 6번 마을에 학교를 배치하면 모든 마을이 커버된다.

    최소의 학교 수 = 2개



집합 커버

- 신도시 계획 문제를 집합 커버 문제로 변환

  $U = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ // 신도시의 마을 10개

  $F = \{S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, S_9, S_{10}\}$

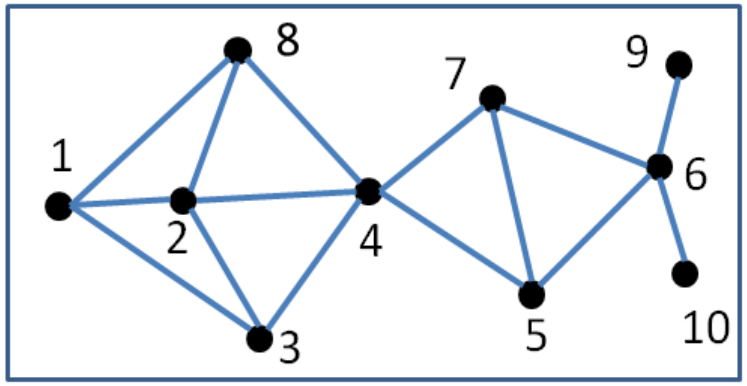
  // S_i는 마을 i에 학교를 배치했을 때 커버되는 마을의 집합

  $S_1=\{1, 2, 3, 8\}$ $S_5=\{4, 5, 6, 7\}$ $S_9=\{6, 9\}$

  $S_2=\{1, 2, 3, 4, 8\}$ $S_6=\{5, 6, 7, 9, 10\}$ $S_{10}=\{6, 10\}$

  $S_3=\{1, 2, 3, 4\}$ $S_7=\{4, 5, 6, 7\}$

  $S_4=\{2, 3, 4, 5, 7, 8\}$ $S_8=\{1, 2, 4, 8\}$



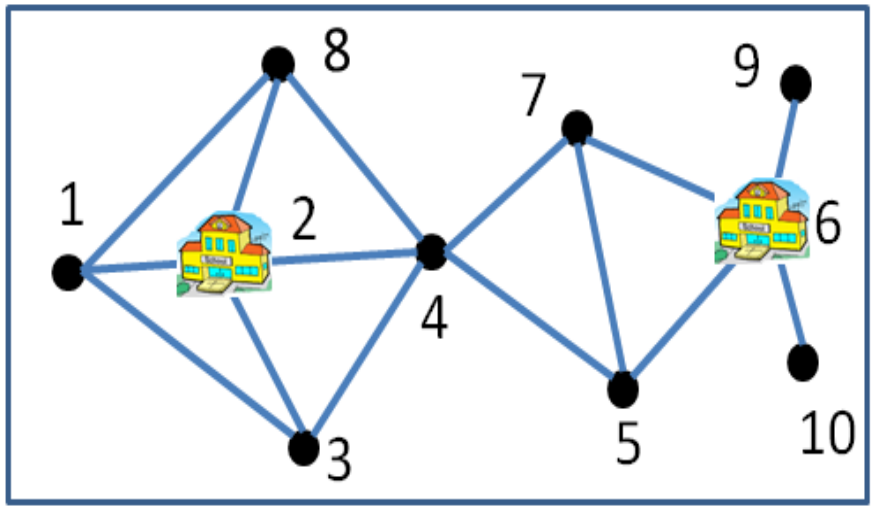
- S_i 집합들 중에서 어떤 집합들을 선택해야 그들의 합집합이 U와 같은가?

  단, 선택된 집합의 수는 최소이어야

최적해

- $S_2 \cup S_6 = \{1, 2, 3, 4, 8\} \cup \{5, 6, 7, 9, 10\}$

  $= \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\} = U$



단순한 해결 방법

- 집합 커버 문제의 최적해는 어떻게 찾아야 할까?
  - F에 n개의 집합들이 있다고 가정해보자.

- 가장 단순한 방법
  - F에 있는 집합들의 모든 조합을 1개씩 합집합하여 U가 되는지 확인하고,
  - U가 되는 조합의 집합 수가 최소인 것을 찾는다.

  - $F=\{S_1, S_2, S_3\}$일 경우 모든 조합

    S_1, S_2, S_3, $S_1 \cup S_2$, $S_1 \cup S_3$, $S_2 \cup S_3$, $S_1 \cup S_2 \cup S_3$

    집합이 1개인 경우 3개 = ${}_3C_1$

    집합이 2개인 경우 3개 = ${}_3C_2$

    집합이 3개인 경우 1개 = ${}_3C_3$

    총합은 3+3+1= 7 = 2^3-1 개

단순한 해결 방법

- n개의 원소가 있을 경우
  - 최대 (2^n-1)개를 검사하여야
  - n이 커지면 최적해를 찾는 것은 실질적으로 불가능

- 이를 극복하기 위한 방법
  - 최적해를 찾는 대신에 최적해에 근접한 근사해 (Approximation solution)를 찾음

알고리즘

SetCover

입력: U, F = $\{S_i\}$, i=1,⋯,n

출력: 집합 커버 C

1. C = ∅
2. while U ≠ ∅
3. U의 원소를 가장 많이 가진 집합 S_i를 F에서 선택
4. U = U - S_i
5. S_i를 F에서 제거하고, S_i를 C에 추가
6. return C

수행 과정

$U = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

$F = \{S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, S_9, S_{10}\}$

$S_1=\{1, 2, 3, 8\}$

$S_2=\{1, 2, 3, 4, 8\}$

$S_3=\{1, 2, 3, 4\}$

$S_4=\{2, 3, 4, 5, 7, 8\}$

$S_5=\{4, 5, 6, 7\}$

$S_6=\{5, 6, 7, 9, 10\}$

$S_7=\{4, 5, 6, 7\}$

$S_8=\{1, 2, 4, 8\}$

$S_9=\{6, 9\}$

$S_{10}=\{6, 10\}$

수행 과정

- U의 원소를 가장 많이 커버하는 집합

  S_4={2, 3, 4, 5, 7, 8} 선택
- U = U - S_4

  = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10} - {2, 3, 4, 5, 7, 8}

  = {1, 6, 9, 10}
- S_4를 F에서 제거 F = { S_1, S_2, S_3, ~~S_4~~, S_5, S_6, S_7, S_8, S_9, S_{10}} - {S_4} = {S_1, S_2, S_3, S_5, S_6, S_7, S_8, S_9, S_{10}}
- S_4를 C에 추가. 즉, C = {S_4}

수행 과정

- U = {1, 6, 9, 10}을 가장 많이 커버하는 집합

  S_6 = {5, 6, 7, 9, 10} 선택
- $U = U - S_6$

  = {1, 6, 9, 10} - {5, 6, 7, 9, 10}

  = {1}
- S_6을 F에서 제거 $F = \{ S_1, S_2, S_3, S_5, \cancel{S_6}, S_7, S_8, S_9, S_{10}\} - \{S_6\} = \{S_1, S_2, S_3, S_5, S_7, S_8, S_9, S_{10}\}$

- S_6을 C에 추가. 즉, $C = \{S_4, S_6\}$

수행 과정

- U = {1}을 가장 많이 커버하는 집합

  S_1 = {1, 2, 3, 8} 선택
- U = U – S_1

  = {1} - {1, 2, 3, 8}

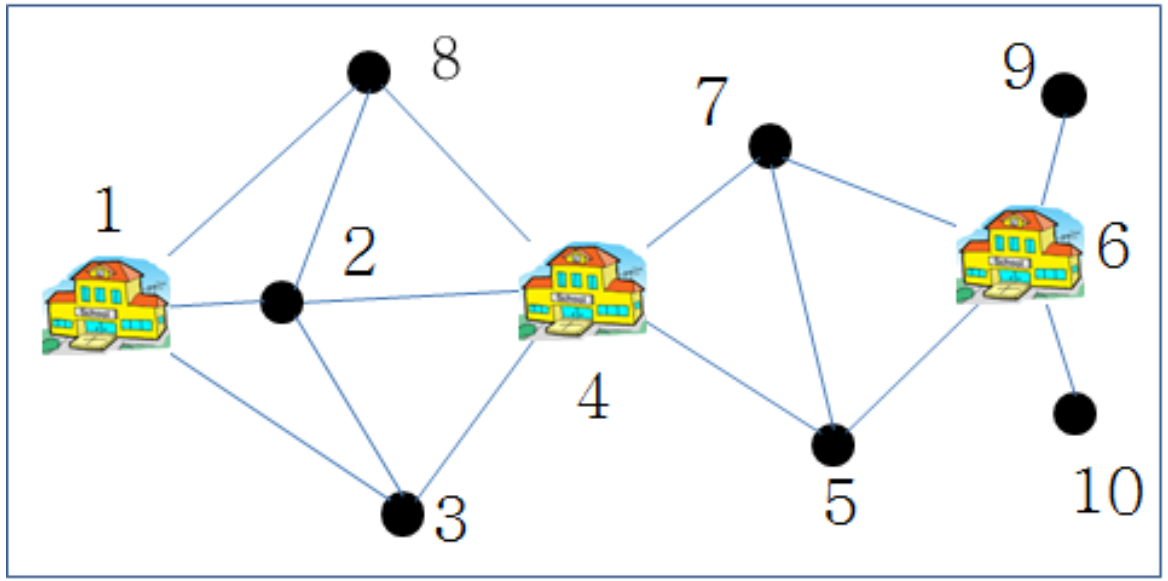
  = { }
- S_1을 F에서 제거, F = {$\cancel{S_1}$, S_2, S_3, S_5, S_7, S_8, S_9, S_{10}} - {S_1} = {S_2, S_3, S_5, S_7, S_8, S_9, S_{10}}

- S_1을 C에 추가. 즉, C = {S_4, S_6, S_1}

수행 과정

- Line 6: $C=\{S_1, S_4, S_6\}$ 리턴

SetCover 알고리즘의 최종해



시간 복잡도

- 먼저 while-루프가 수행되는 횟수는 최대 n회
  - 루프가 1회 수행될 때마다 집합 U의 원소 1개씩만 커버된다면, 최악의 경우 루프가 n번 수행되어야 하기 때문
- 루프가 1회 수행될 때
  - Line 3: U의 원소들을 가장 많이 포함하고 있는 집합 S를 찾으려면, 현재 남아있는 S_i들 각각을 U와 비교하여야
  - S_i들의 수가 최대 n이라면, 각 S_i와 U의 비교는 $O(n)$ 시간이 걸리므로, line 3은 $O(n^2)$ 시간
  - 집합 U에서 집합 S_i의 원소를 제거하므로 $O(n)$ 시간
  - S_i를 F에서 제거하고, S_i를 C에 추가하는 것은 $O(1)$ 시간
- 시간 복잡도: $n \times O(n^2) = O(n^3)$

응용

- 도시 계획 (City Planning)에서 공공 기관 배치하기
- 경비 시스템: 경비가 요구되는 장소의 CCTV 카메라의 최적 배치
- 컴퓨터 바이러스 찾기
- 대기업의 구매 업체 선정
- 기업의 경력 직원 고용
- 그 외에도 비행기 조종사 스케줄링, 조립 라인 균형화, 정보 검색 등에 활용

사례 1 : 통신망 및 스마트 보안 인프라

5G 기지국 / Wi-Fi AP 설치

- 통신사에서 특정 도심의 모든 건물과 거리에 통신 신호를 제공해야 함
- 수백 개의 후보 설치 장소 중 '가장 많은 음영 지역을 덮는(Cover) 기지국'부터 우선 설치해 나가며, 최소한의 장비 대수로 전 구역의 통신망을 구축함

감시 카메라 (CCTV) 배치 최적화

- 보안 구역 내에 사각지대가 없도록 CCTV를 설치해야 함
- 각 CCTV가 촬영할 수 있는 시야각(교집합 영역)을 분석하여, 한 번의 설치로 가장 넓은 미관측 공간을 감시할 수 있는 위치부터 탐욕적으로 선정해 비용을 절감함

사례 2 : IT 엔지니어링 및 운영 효율화

SW 테스트 케이스 축소

- 코드의 수만 줄(Lines)을 모두 검증해야 할 때, 가장 많은 소스코드 라인을 통과하는 통합 테스트 케이스부터 선정하여, 최소한의 테스트 실행으로 100% 커버리지를 달성함

IT 보안 취약점 패치

- 서버에 발견된 수십 개의 보안 취약점을 해결할 때, 한 번의 업데이트로 가장 많은 종류의 취약점(CVE)을 동시에 해결하는 롤업(Rollup) 패치부터 적용하여 시스템 다운타임을 줄임

항공사 승무원 스케줄링

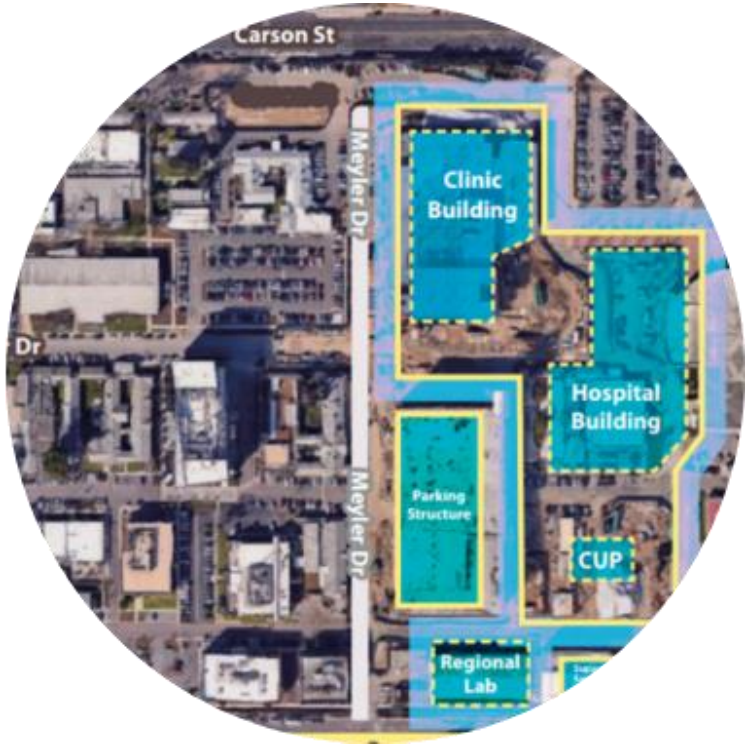
- 항공사가 수백 개의 비행 스케줄(요소)을 운영할 때, 각 승무원 조(Set)가 커버할 수 있는 노선의 조합 중 가장 많은 잔여 비행을 소화하는 조부터 투입하여 잉여 인력을 최소화함

사례 3 : 물류 창고 및 배송 거점 설계

전국 당일 배송망의 수학적 기초

- 이커머스 기업(아마존, 쿠팡 등)이 전국 모든 시/군/구에 익일 배송을 보장하고자 함
- 수십 개의 물류 센터 설립 후보지가 주어짐
- 각 센터가 도달 가능한 '3시간 이내 배송 구역'을 집합으로 삼아, 가장 많은 미배송 인구 밀집 구역을 커버하는 물류 거점부터 순차적으로 건립하여 거점 투자 비용을 획기적으로 방어함

사례 4 : 의료 및 공공 보건 인프라 배치



접근성을 고려한 백신 접종 센터

- 전염병 유행 시 정부는 모든 시민이 대중교통으로 30분 이내에 접근할 수 있도록 백신 접종소를 설치해야 함

- 도심의 여러 관공서, 학교 등 후보지 중에서, 가장 많은 '접종소 미접근 주민'을 커버할 수 있는 지역부터 센터를 우선 지정하여 예산을 최적화하고 방역 사각지대를 없앰

사례 5 : 스마트 시티 및 환경 센서

응급 구조 시설 (소방서, 병원) 입지

- 도시 계획 단계에서 소방서나 구급대의 5분 이내(골든 타임) 출동 가능 구역을 모델링함
- 전 지역의 화재 및 응급 취약 구역을 가장 빠르게 커버할 수 있는 핵심 거점을 그리디하게 찾아내어 국민의 생명과 안전망을 구축함

무선 센서 네트워크 (IoT) 배치

- 산림 화재 감지나 도심 미세먼지 측정 시, 광범위한 지역을 최소한의 IoT 센서로 덮어야 (Cover) 함
- 센서의 감지 반경 내에 가장 넓은 미측정 영역이 들어오는 포인트들을 우선적으로 탐욕 탐색하여 센서망을 설치함

비즈니스 가치 및 시사점

- **최소 자원, 최대 효용:** 시설 투자, 소프트웨어 개발, 인력 배치 등 막대한 예산이 소요되는 프로젝트에서 최소한의 투자로 100%의 효용(Coverage)을 만들어냄
- **현실적인 문제 해결사:** 계산 불가능한 NP-Hard 문제 앞에서 완벽한 정답을 포기하는 대신, 즉각적이고 수용 가능한 수준의 강력한 최적의 대안을 제공함
- **광범위한 확장성:** 물리적 공간(기지국, 물류망) 커버리지부터 논리적 공간(SW 취약점, 마케팅 타겟)의 커버리지까지 도메인을 가리지 않고 응용됨
- **알고리즘의 한계 인식:** 그리디 로직 특성상 초반의 선택이 잘못되었을 경우 백트래킹(되돌리기)이 되지 않으므로, 데이터의 가중치를 정밀하게 설계해야 함

파이썬 구현 패턴 1 : 기본 구현 (수학적 부분 집합 선택)

- 가장 원론적인 형태의 그리디 알고리즘 구현
- 파이썬의 집합(set) 자료형과 교집합/차집합 연산을 활용하면 매우 직관적으로 작성할 수 있음

```
# 예제 데이터

universe_set = {1, 2, 3, 4, 5, 6, 7, 8}
subsets_data = {
    'S1': {1, 2, 3},
    'S2': {2, 4, 5, 6},
    'S3': {1, 3, 5, 7, 8},
    'S4': {2, 4},
    'S5': {6, 8}
}

print(f“ [예제 1] 선택된 기본 부분 집합: {set_cover_basic(universe_set, subsets_data)}")
```

파이썬 구현 패턴 1 :

기본 구현 (수학적 부분 집합 선택)

```
def set_cover_basic(universe, subsets):
    """
    :param universe: 커버해야 할 전체 원소들의 집합 (set)
    :param subsets: 부분 집합들의 딕셔너리 {집합이름: set(원소들)}
    """
    covered = set()  # 현재까지 커버된 원소들
    selected_subsets = []  # 선택된 부분 집합들

    # 전체 원소를 모두 커버할 때까지 반복
    while covered != universe:
        best_subset_name = None
        elements_covered_in_this_step = set()

        for name, elements in subsets.items():
            # 현재 부분 집합이 커버할 수 있는 '새로운' 원소들 (차집합 활용)
            uncovered = elements - covered

            # 가장 많은 새로운 원소를 커버하는 집합을 찾음
            if len(uncovered) > len(elements_covered_in_this_step):
                best_subset_name = name
                elements_covered_in_this_step = uncovered

        # 더 이상 커버할 수 있는 집합이 없으면 무한루프 방지
        if best_subset_name is None:
            break

        selected_subsets.append(best_subset_name)
        covered |= subsets[best_subset_name]  # 합집합 연산으로 커버된 영역 업데이트

    return selected_subsets
```

파이썬 구현 패턴 2 : [인사 관리] 필요 기술(Skill) 스택을 완비하기 위한 최소 인원 채용

- 소프트웨어 프로젝트를 위해 Python, Java, AWS, React 등의 기술이 모두 필요
- 지원자들은 각기 다른 기술 조합을 가지고 있을 때, 프로젝트 요구사항을 모두 충족시키는 최소한의 지원자를 선발하고자 함

```
# 요구 기술 스택
project_skills = {'Python', 'Java', 'AWS', 'Docker', 'React', 'SQL’}

# 지원자들이 보유한 기술
applicant_pool = {
    'Alice': {'Python', 'SQL'},
    'Bob': {'Java', 'AWS', 'React'},
    'Charlie': {'Python', 'Docker', 'AWS'},
    'David': {'SQL', 'Java'},
    'Eve': {'React', 'Docker'}
}

print(f"\n[예제 2] 채용된 지원자 명단: {hire_minimum_candidates(project_skills, applicant_pool)}")
```

파이썬 구현 패턴 2 :

[인사 관리] 필요 기술(Skill) 스택을 완비하기 위한 최소 인원 채용

```
def hire_minimum_candidates(required_skills, candidates):
    hired_candidates = []
    fulfilled_skills = set()

    while fulfilled_skills != required_skills:
        best_candidate = None
        skills_added_by_candidate = set()

        for name, skills in candidates.items():
            new_skills = skills - fulfilled_skills

            if len(new_skills) > len(skills_added_by_candidate):
                best_candidate = name
                skills_added_by_candidate = new_skills

        if best_candidate is None:
            print("경고: 모든 요구 기술을 만족시킬 수 있는 지원자 조합이 없습니다.")
            break

        hired_candidates.append(best_candidate)
        fulfilled_skills |= candidates[best_candidate]

    return hired_candidates
```

4.6 작업 스케줄링

➢ 작업 스케줄링 (Job Scheduling) 문제

- 작업의 수행 시간이 중복되지 않도록 모든 작업을 가장 적은 수의 기계에 배정하는 문제
- 학술대회에서 발표자들을 강의실에 배정하는 문제와 같다.

  발표= '작업', 강의실= '기계'

➢ 작업 스케줄링 문제에 주어진 문제 요소

- 작업의 수

  입력의 크기이므로 알고리즘을 고안하기 위해 고려되어야 하는 직접적인 요소는 아니다.
- 각 작업의 시작시간과 종료시간
- 작업의 길이

  작업의 시작시간과 종료시간은 정해져 있으므로 작업의 길이도 주어진 것

작업 스케줄링

- 시작시간, 종료시간, 작업 길이에 대한 그리디 알고리즘
  - 빠른 시작시간 작업 우선 (Earliest start time first) 배정
  - 빠른 종료시간 작업 우선 (Earliest finish time first) 배정
  - 짧은 작업 우선 (Shortest job first) 배정
  - 긴 작업 우선 (Longest job first) 배정

- 위 4가지 중 첫 번째 알고리즘을 제외하고 나머지 3가지는 항상 최적해를 찾지 못함 ▶ but 빠른 종료시간 우선이 최적해를 가짐

작업 스케줄링

1. 빠른 시작 시간 우선 (Earliest Start Time First)이 실패하는 이유

- 시작 시간이 가장 빠른 작업을 먼저 고르게 되면, 그 작업이 엄청나게 길 경우 뒤에 올 수 있는 수많은 짧은 작업들을 모두 불가능하게 만들어 버립니다.

```
작업 A (0~10) : [=========================================] (시작이 가장 빠름)
작업 B (1~3) :    [======]
작업 C (4~6) :                    [======]
작업 D (7~9) :                                  [======]
```

- 그리디 선택 (시작 시간 우선): 가장 먼저 시작하는 작업 A를 선택합니다. 하지만 A가 끝날 때까지 다른 작업을 할 수 없으므로 총 1개의 작업만 처리합니다.
- 최적해 (실제 정답): 작업 B, C, D를 선택하면 총 3개의 작업을 처리할 수 있습니다.

작업 스케줄링

2. 짧은 작업 시간 우선 (Shortest Duration First)이 실패하는 이유

- 작업 시간이 가장 짧은 것을 고르면 좋을 것 같지만, 그 짧은 작업이 하필 다른 두 개의 긴 작업 사이에 걸쳐 있다면 오히려 손해를 보게 됩니다.

```
작업 A (0~5) : [==================]
작업 B (4~6) :                     [======] (작업 시간이 가장 짧음)
작업 C (5~10) :                        [==================]
```

- 그리디 선택 (짧은 시간 우선): 길이가 가장 짧은 작업 B를 선택합니다. 하지만 B를 선택하는 순간, 시간대가 겹치는 A와 C를 모두 포기해야 하므로 총 1개의 작업만 처리합니다.
- 최적해 (실제 정답): 작업 A와 C를 선택하면 총 2개의 작업을 처리할 수 있습니다.

작업 스케줄링

3. 긴 작업 시간 우선 (Longest Duration First)이 실패하는 이유

- 빠른 시작 시간 우선과 동일

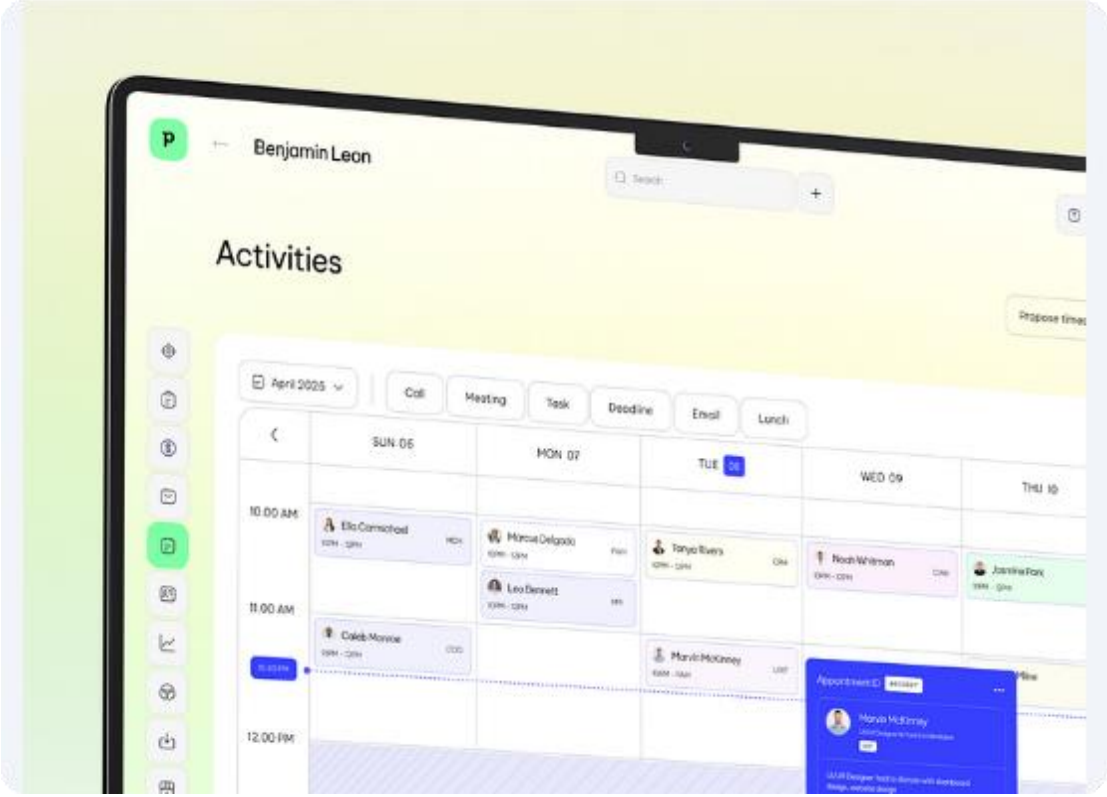
작업 스케줄링

선택의 기준이 결과를 결정

“ 작업 스케줄링에서 그리디 알고리즘은 복잡한 경우의 수를 계산하지 않음
종료 시간(EFT), 실행 시간(SJF), 마감 기한(Deadline) 등 하나의 명확한
기준에 따라 매 순간 최선의 작업을 선택하여 전역 최적해를 도출함”

작업 스케줄링 핵심 원리

- **종료 시간이 가장 빠른 작업 우선 (EFT):** 시작 시간이 아니라 '가장 먼저 끝나는 작업'을 고르는 것이 핵심
- **겹치지 않는 작업 선택:** 선택한 작업이 종료된 이후에 시작하는 다음 작업 중, 다시 종료 시간이 가장 빠른 것을 탐욕적으로 선택함
- **목표:** 주어진 한정된 시간(하루 24시간 등) 내에 최대 개수의 작업을 처리하는 것을 보장함



알고리즘

JobScheduling

입력: n개의 작업 $t_1, t_2, \cdots, t_n$

출력: 각 기계에 배정된 작업 순서

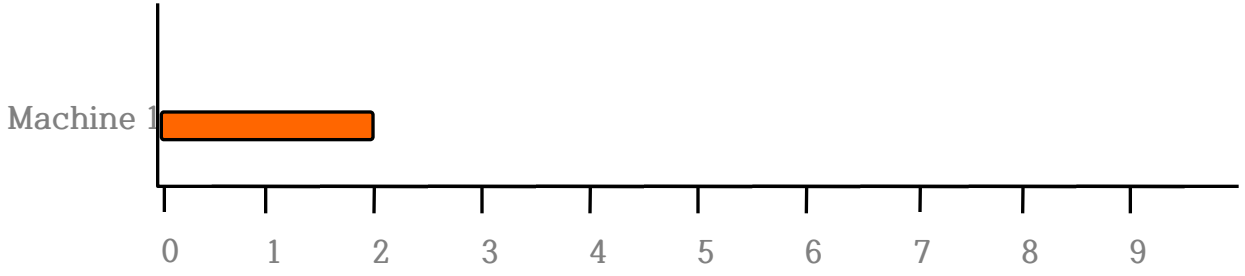
```
1. 시작 시간으로 정렬한 작업 리스트: L
2. while  L ≠ ∅
3.      L에서 가장 이른 시작 시간 작업 t_i를 가져온다.
4.      if  t_i를 수행할 기계가 있으면
5.              t_i를 수행할 수 있는 기계에 배정
6.      else
7.              새 기계에 t_i를 배정
8.      t_i를 L에서 제거
9. return 각 기계에 배정된 작업 순서
```

수행 과정

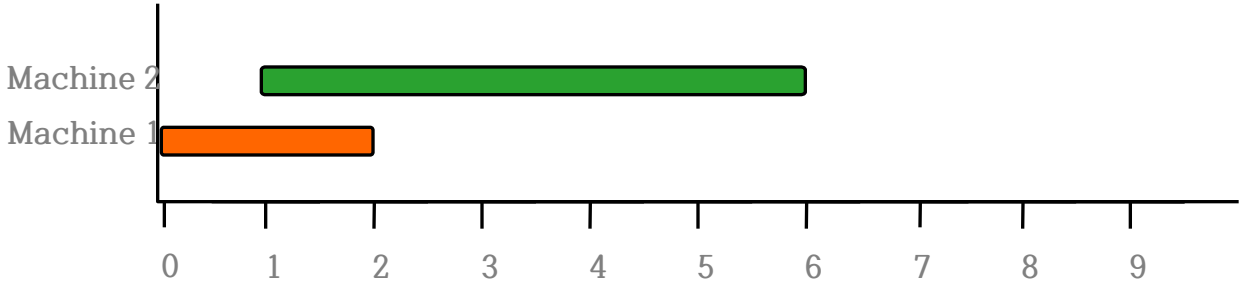
- t_1=[7,8], t_2=[3,7], t_3=[1,5], t_4=[5,9], t_5=[0,2], t_6=[6,8], t_7=[1,6]
  - [s, f]에서, s는 시작 시간, f는 종료 시간
- 정렬: L = {[0,2], [1,6], [1,5], [3,7], [5,9], [6,8], [7,8]}

수행 과정

[0,2]

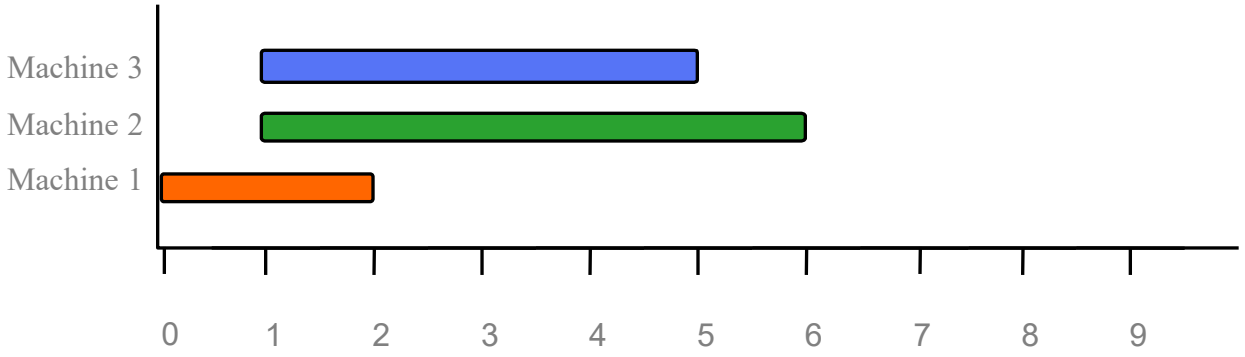


[0,2], [1,6]

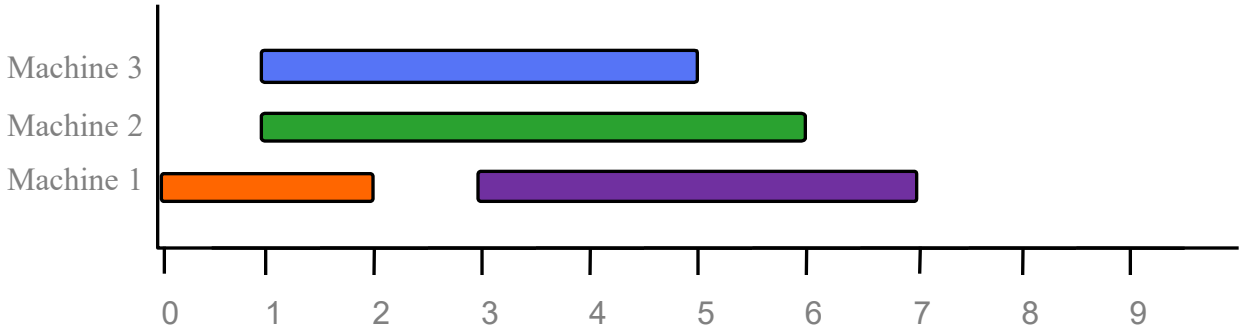


수행 과정

[0,2], [1,6], [1,5]

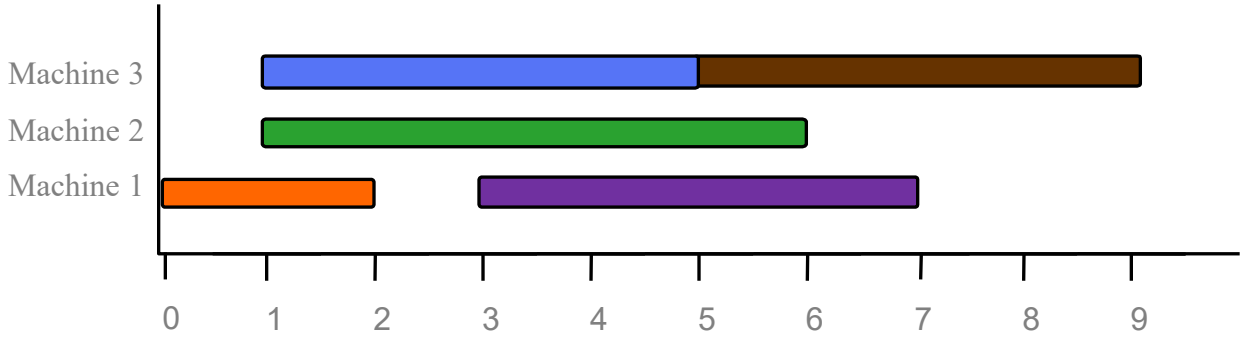


[0,2], [1,6], [1,5], [3,7]

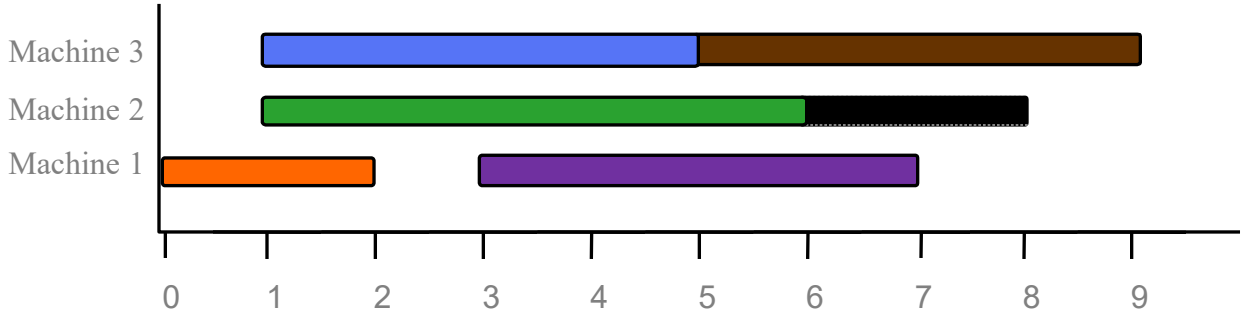


수행 과정

[0,2], [1,6], [1,5], [3,7], [5,9]

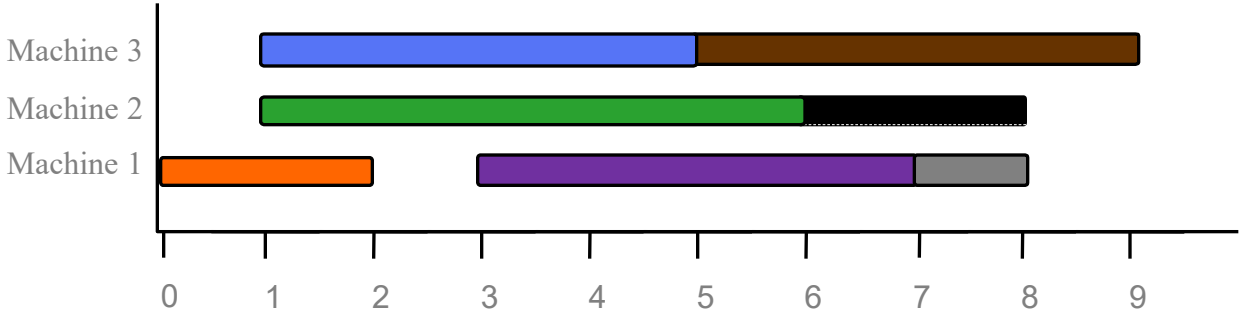


[0,2], [1,6], [1,5], [3,7], [5,9], [6,8]



수행 과정

[0,2], [1,6], [1,5], [3,7], [5,9], [6,8], [7,8]



시간 복잡도

- Line 1: 정렬 시간 O(nlogn)
- while-루프
  - 작업을 L에서 가져다 수행 가능한 기계를 찾아서 배정하므로 O(m) 시간 소요, 단, m은 사용된 기계의 수
  - while-루프가 수행된 총 횟수는 n번이므로, line 2~9까지는 O(m) × n = O(mn) 시간 소요
- 시간 복잡도: O(nlogn)+O(mn)

응용

- 비즈니스 프로세싱
- 공장 생산 공정
- 강의실/세미나 룸 배정
- 컴퓨터 태스크 스케줄링 등

사례 1 : IT 인프라 및 운영 체제

CPU 프로세스 스케줄링 (SJF)

- 운영체제가 대기 중인 프로세스들을 CPU에 할당할 때, 실행 시간(Burst Time)이 가장 짧은 작업(Shortest Job First)부터 먼저 처리함
- 무거운 작업이 가벼운 작업들을 막고 있는 현상을 방지하여, 시스템의 평균 대기 시간을 극적으로 최소화함

클라우드 Serverless 작업 할당

- AWS Lambda나 GCP Cloud Functions 환경에서 수많은 마이크로서비스 요청이 발생할 때, 자원 할당 정책에 그리디 방식을 적용함
- 종료 예측 시간이 가장 빠른 함수부터 인스턴스를 점유하게 하여 동시 처리량을 극대화함

사례 2 : 물리적 공간 및 자산 할당

회의실 / 컨퍼런스룸 예약

- 단 1개의 회의실에 수많은 팀의 사용 요청이 겹칠 때, '가장 일찍 끝나는 회의'부터 순서대로 승인하여 하루 동안 가장 많은 수의 회의를 수용하는 고전적인 구간 스케줄링의 활용함

공항 활주로 이착륙 배정

- 혼잡한 관제탑에서 한정된 활주로 슬롯을 비행기들에게 배정할 때, 이착륙 점유 시간이 짧거나 필수 마감 기한이 임박한 항공기를 그리디하게 우선 배치하여 공항 수용 능력을 높임

병원 수술실(OR) 스케줄링

- 응급이 아닌 예정된 수술들을 한정된 수술실에 배정할 때, 각 수술의 예상 소요 시간을 기반으로 공간과 집도의의 공백 시간(Idle Time)이 최소화되도록 수술 순서를 최적화함

사례 3 : 스마트 팩토리 공정 스케줄링

조립 라인의 병목 현상 제거

- 제조업의 컨베이어 벨트나 로봇 조립 라인에서 여러 종류의 부품을 가공해야 함
- 각 기계(Machine)의 가공 시간이 다를 때, 가장 짧은 시간 안에 끝나는 공정이나 납기일(Deadline) 패널티가 가장 큰 작업부터 기계에 우선 투입(Job Sequencing)하여 전체 제조 리드타임(Makespan)을 단축함

사례 4 : 물류 차량 배차 (Time Window)

타임 윈도우 스케줄링

- 고객마다 택배 수령을 원하는 '시간대(Time Window, 예: 10시~12시)'가 지정되어 있음

- 배송 기사의 단말기는 각 목적지의 마감 시간이 가장 이른 (Earliest Deadline First) 배송 건부터 순차적으로 라우팅하여 지연 패널티를 없애고 일일 배송 완료 건수를 최대화함

비즈니스 가치 및 시사점

- **처리량(Throughput)의 극대화:** 병원, 공항, 회의실 등 고정 비용이 막대한 물리적 인프라의 유휴 시간(Idle Time)을 없애고 1일 가동률을 최대치로 끌어올림
- **시스템 병목 현상 해소:** IT 환경(CPU, 클라우드)에서 가벼운 작업을 먼저 처리하여 대기열 적체(Starvation)를 방지하고 사용자 경험(UX)을 개선함
- **단일 목표의 강력함:** 복잡한 제약 조건 하에서 '시간 단축' 혹은 '수익 극대화'라는 하나의 명확한 그리디 잣대만으로도 산업 현장의 난제들을 훌륭하게 해결함
- **유의점:** 다중 자원(서로 다른 성능의 서버 등) 환경에서는 단순한 SJF나 EFT만으로는 서브 최적해(Sub-optimal)에 빠질 수 있으므로, 제약 조건에 맞는 추가적인 휴리스틱이 요구됨

파이썬 구현 패턴 1 : 기본 마감일 작업 스케줄링 (이익 최대화)

- 가장 고전적인 '마감일이 있는 작업 스케줄링(Job Sequencing with Deadlines)' 문제
- 각 작업의 소요 시간이 1이라고 가정할 때, 이익이 높은 작업을 우선적으로 배치
- 그리디 전략: '이익이 가장 큰 작업'부터 선택하여, 해당 작업의 마감일 내에서 가장 늦은 빈 시간(Slot)에 배치

```
# 프리랜서 외주 작업 목록 (작업명, 마감일(일), 이익(만원))
freelance_jobs = [('A', 4, 20), ('B', 1, 10), ('C', 1, 40), ('D', 1, 30)]
profit, schedule = job_scheduling_basic(freelance_jobs)
print(f" [예제 1] 프리랜서 최대 수익: {profit}만원")
print(f" 작업 스케줄 (작업명, 진행일) : {schedule}")
```

파이썬 구현 패턴 1 :

기본 마감일 작업 스케줄링 (이익 최대화)

```
def job_scheduling_basic(jobs):
    # jobs = [(작업명, 마감일, 이익)]
    # 1. 이익을 기준으로 내림차순 정렬
    jobs.sort(key=lambda x: x[2], reverse=True)

    max_deadline = max(job[1] for job in jobs)
    slots = [-1] * (max_deadline + 1)  # 1번 인덱스부터 사용

    total_profit = 0
    scheduled_jobs = []

    for job_id, deadline, profit in jobs:
        # 마감일부터 1까지 역순으로 빈 슬롯 찾기
        for i in range(deadline, 0, -1):
            if slots[i] == -1:
                slots[i] = job_id
                total_profit += profit
                scheduled_jobs.append((job_id, i))  # (작업명, 배정된 시간)
                break

    return total_profit, scheduled_jobs
```

파이썬 구현 패턴 2 : 최소 힙(Min-Heap)을 활용한 스케줄링 최적화

- 1번 예제는 빈 슬롯을 찾기 위해 역순 탐색을 하므로 O(N^2)의 시간이 걸릴 수 있음
- 대규모 데이터에서는 우선순위 큐(Heap)를 사용하여 마감일 순으로 탐색하며 O(N log N)으로 처리하는 것이 훨씬 효율적
- 그리디 전략: 마감일이 빠른 순서대로 힙에 넣되, 마감일을 초과하여 작업을 담게 되면 힘에 들어있는 작업 중 '가장 이익이 적은 작업'을 포기(pop) 함

```
# 수만 개의 데이터가 있다고 가정한 대용량 처리 모델
massive_jobs = [('J1', 2, 100), ('J2', 1, 19), ('J3', 2, 27), ('J4', 1, 25), ('J5', 3, 15)]
print(f"\n[예제 2] 힙(Heap) 최적화 기반 최대 수익: {job_scheduling_heap(massive_jobs)}")
```

파이썬 구현 패턴 2 : 최소 힙(Min-Heap)을 활용한 스케줄링 최적화

```
import heapq

def job_scheduling_heap(jobs):
    # 1. 마감일을 기준으로 오름차순 정렬
    jobs.sort(key=lambda x: x[1])

    min_heap = []

    for job_id, deadline, profit in jobs:
        heapq.heappush(min_heap, profit)

        # 힙의 크기(현재까지 배정된 작업 수)가 마감일을 초과하면
        # 가장 이익이 적은 작업을 큐에서 제거
        if len(min_heap) > deadline:
            heapq.heappop(min_heap)

    # 힙에 남아있는 수익의 합이 최대 이익
    return sum(min_heap)
```

파이썬 구현 패턴 3 : 최소 지연 스케줄링 (EDF: Earliest Deadline First)

- 제조업 공장이나 납품 환경에서, 모든 작업을 처리해야 하지만 '마감일을 가장 많이 넘긴 작업의 지연 시간'을 최소화해야 할 때 사용함
- 그리디 전략: 이익과 상관없이 무조건 '마감일이 가장 빠른 작업'부터 먼저 처리함

```
# 금형 공장 제작 스케줄 (프로젝트명, 제작소요시간, 납기일)
factory_jobs = [('P1', 3, 6), ('P2', 2, 8), ('P3', 1, 9), ('P4', 4, 9), ('P5', 3, 14)]
lateness, log = minimize_maximum_lateness(factory_jobs)
print(f"\n[예제 3] 공장 최대 납기 지연 시간: {lateness}일")
for l in log: print(" -", l)
```

파이썬 구현 패턴 3 : 최소 지연 스케줄링 (EDF: Earliest Deadline First)

```
def minimize_maximum_lateness(jobs):
    # jobs = [(작업명, 소요시간, 마감일)]
    # 마감일(Deadline)을 기준으로 오름차순 정렬 (EDF 전략)
    jobs.sort(key=lambda x: x[2])

    current_time = 0
    max_lateness = 0
    schedule_log = []

    for job_id, duration, deadline in jobs:
        start_time = current_time
        current_time += duration  # 작업 소요 시간만큼 시간 흐름

        # 지연 시간 계산 (음수면 지연되지 않은 것이므로 0으로 처리)
        lateness = max(0, current_time - deadline)
        max_lateness = max(max_lateness, lateness)

        schedule_log.append(f"{job_id}({start_time}~{current_time}) 지연:{lateness}일")

    return max_lateness, schedule_log
```

4.7 허프만 압축

선택의 불균형이 효율성을 낳는다

“ 허프만 압축은 빈도수가 가장 높은 데이터에 가장 짧은 코드를 부여하고, 빈도수가 낮은 데이터에 긴 코드를 부여하는 '그리디'한 이진 트리를 구성하여 전체 데이터의 평균 비트 길이를 획기적으로 최소화함 ”

4.7 허프만 압축

- 파일의 각 문자가 8 bit 아스키 (ASCII) 코드로 저장되면, 그 파일의 bit 수는 8 × (파일의 문자 수)
- 파일의 각 문자는 일반적으로 고정된 크기의 코드로 표현
- 고정된 크기의 코드로 구성된 파일을 저장하거나 전송할 때 파일의 크기를 줄이고, 필요시 원래의 파일로 변환할 수 있으면, 메모리 공간을 효율적으로 사용할 수 있고, 파일 전송 시간을 단축
- 파일의 크기를 줄이는 방법을 파일 압축 (file compression)이라 함

아이디어

- 허프만 (Huffman) 압축은 파일에 빈번히 나타나는 문자에는 짧은 이진 코드를 할당하고, 드물게 나타나는 문자에는 긴 이진 코드를 할당

- 허프만 압축 방법으로 변환시킨 문자 코드들 사이에는 접두부 특성 (prefix property)이 존재
  - 각 문자에 할당된 이진 코드는 어떤 다른 문자에 할당된 이진 코드의 접두부 (prefix)가 되지 않는다.
  - [예제] 문자 'a'에 할당된 코드가 '101'이라면, 모든 다른 문자의 코드는 '101'로 시작되지 않으며 또한 '1'이나 '10'도 아니다.

허프만 압축

- 접두부 특성의 장점은 코드와 코드 사이를 구분할 특별한 코드가 필요 없다.
  - 101#10#1#111#0#…에서 '#'가 인접한 코드를 구분 짓고 있는데, 허프만 압축에서는 이러한 특별한 코드 없이 파일을 압축/해제 가능

- 허프만 압축은 입력 파일에 대해 각 문자의 빈도수 (문자가 파일에 나타나는 횟수)에 기반을 둔 이진 트리를 만들어서, 각 문자에 이진 코드를 할당
  - 이러한 이진 코드를 허프만 코드라고 함

알고리즘

HuffmanCoding

입력: 입력 파일의 n개의 문자에 대한 각각의 빈도수

출력: 허프만 트리

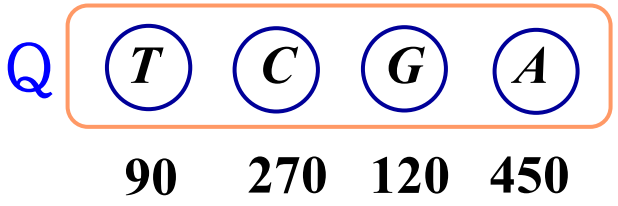
```
1. 각 문자 당 노드를 만들고, 그 문자의 빈도수를 노드에 저장
2. n 노드의 빈도수에 대해 우선 순위 큐 Q를 만든다.
3. while  Q에 있는 노드 수 ≥ 2
4.        빈도수가 가장 적은 2개의 노드 (A와 B)를 Q에서 제거
5.        새 노드 N을 만들고, A와 B를 N의 자식 노드로 만든다
6.        N의 빈도수 = A의 빈도수 + B의 빈도수
7.        노드 N을 Q에 삽입
8. return Q        // 허프만 트리의 루트를 리턴
```

수행 과정

- 각 문자의 빈도수에 대해

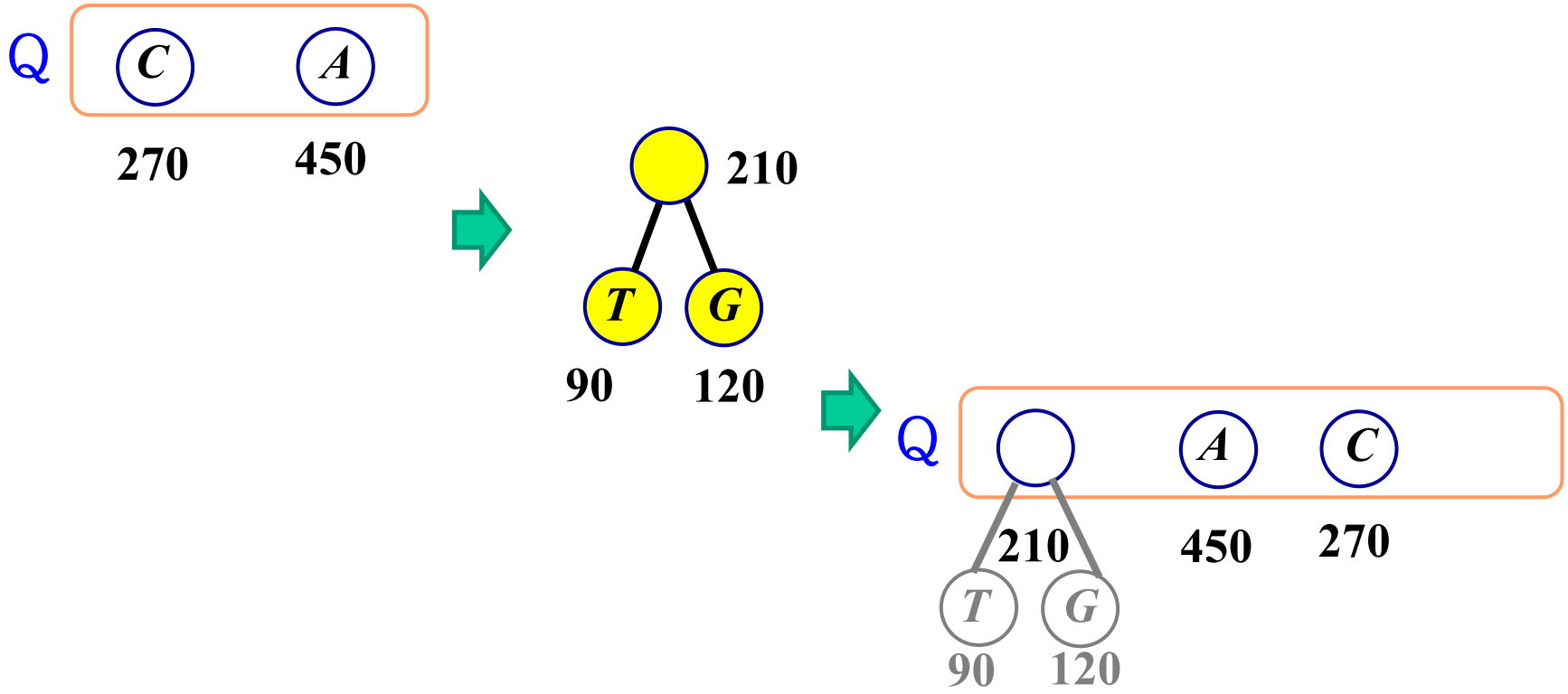
  A: 450 T: 90 G: 120 C: 270

- Line 2를 수행한 후의 Q
  - 우선 순위 큐 Q를 생성



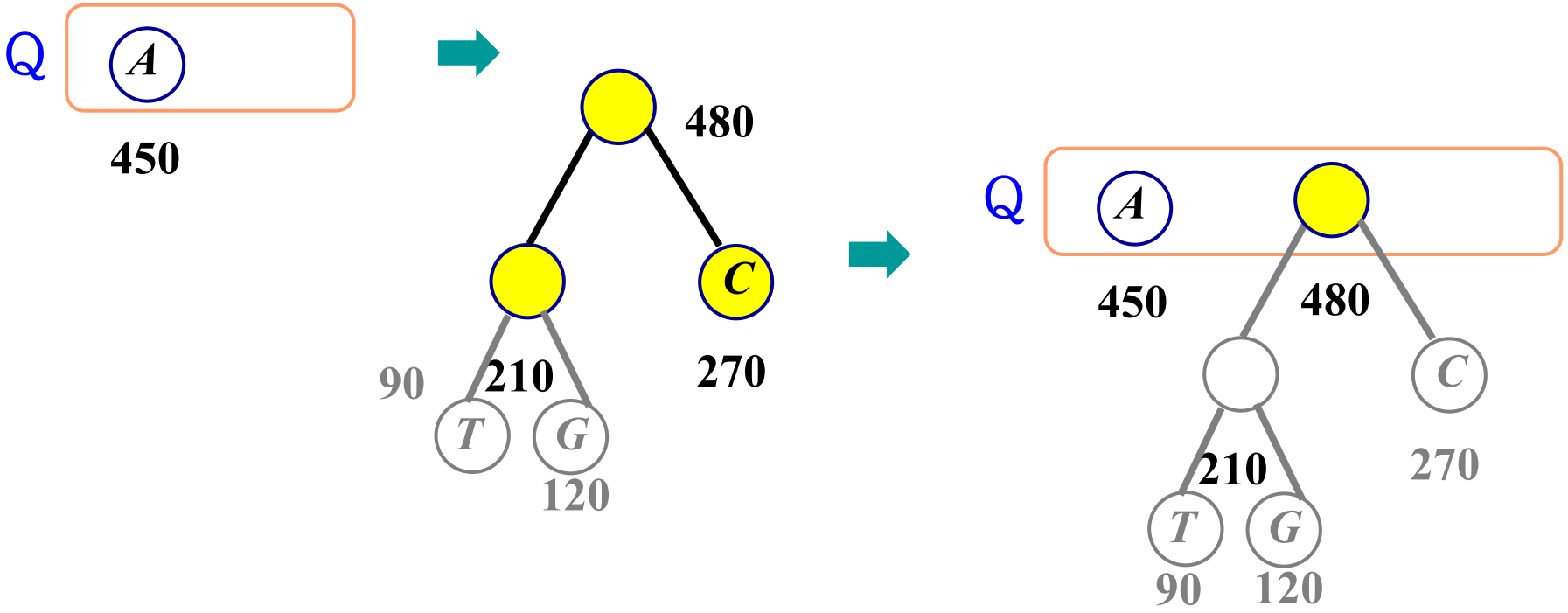
수행 과정

- Line 3: Q에서 ‘T’와 ‘G’를 제거한 후, 새 부모 노드를 Q에 삽입



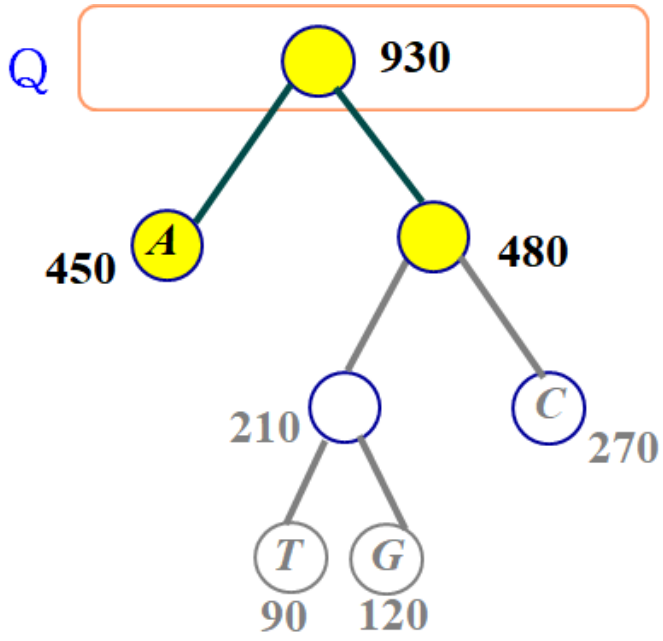
수행 과정

- Line 3: Q에서 ‘T’와 ‘G’의 부모 노드와 ‘C’를 제거한 후, 새 부모 노드를 Q에 삽입



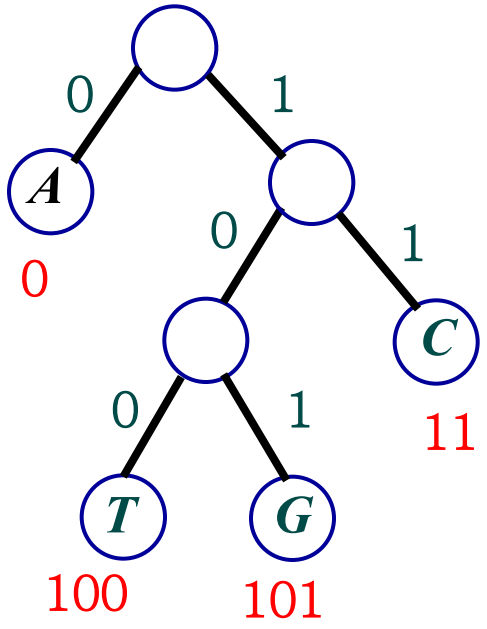
수행 과정

- Line 3: Q에서 ‘C’의 부모 노드와 ‘A’를 제거한 후, 새 부모 노드 Q에 삽입



수행 과정

- 반환된 트리를 살펴보면 각 이파리 (단말) 노드에만 문자가 있음
  - 루트로부터 왼쪽 자식 노드로 내려가면 '0'을, 오른쪽 자식 노드로 내려가면 '1'을 부여하면서, 각 이파리에 도달할 때까지의 이진수를 추출하여 문자의 이진 코드를 얻음

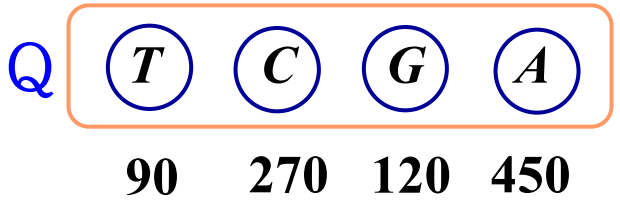


압축률

- 예제에서 'A'는 '0', 'T'는 '100', 'G'는 '101', 'C'는 '11'의 코드가 각각 할당된다.
  - 할당된 코드들을 보면, 가장 빈도수가 높은 'A'가 가장 짧은 코드를 가지고, 따라서 루트의 자식이 되어 있고, 빈도수가 낮은 문자는 루트에서 멀리 떨어지게 되어 긴 코드를 가진다.
  - 이렇게 얻은 코드는 접두부 특성을 가진다.

- 압축된 파일의 bit 수
  - (450×1)+(90×3)+(120×3)+(270×2) = 1,620 bits

    A(0) T (100) G(101) C(11)



- 아스키 코드로 된 파일 크기
  - (450+90+120+270)x8 = 7,440 bits

- 파일 압축률
  - (1,620/7,440)×100 = 21.8%이며, 원래의 약 1/5 크기로 압축

복호화

- 예제에서 얻은 허프만 코드로 아래의 압축된 부분에 대해서 압축을 해제하여 보자.

101100100011101010100

101 / 100 / 100 / 0 / 11 / 101 / 0 / 101 / 0 / 100

⇨ G T T A C G A G A T

시간 복잡도

- Line 1: n개의 노드를 만들고, 각 빈도수를 노드에 저장하므로 O(n) 시간
- Line 2: n개의 노드로 우선순위 큐 Q를 만듬
  - 여기서 우선 순위 큐로서 이진 힙 자료구조를 사용하면 O(n) 시간

시간 복잡도

- Line 3~7
  - 최소 빈도수를 가진 노드 2개를 Q에서 제거하는 힙의 삭제 연산과 새 노드를 Q에 삽입하는 연산을 수행하므로 O(logn) 시간 소요
  - while-루프는 (n-1)번 반복

    왜냐하면 루프가 1번 수행될 때마다 Q에서 2개의 노드를 제거하고 1개를 Q에 추가하기 때문
  - (n-1) x O(logn) = O(n logn)
- Line 8
  - 트리의 루트를 반환하는 것이므로 O(1) 시간
- 시간 복잡도는 O(n)+O(n)+O(nlogn)+ O(1) = O(n logn)

응용

- 팩스(FAX), 대용량 데이터 저장, 멀티미디어 (Multimedia), MP3 압축 등에 활용

- 정보 이론 (Information Theory) 분야에서 엔트로피 (Entropy)를 계산하는데 활용
  - 이는 자료의 불특정성을 분석하고 예측하는데 이용

사례 1 : ZIP / GZIP 범용 파일 압축

- **Deflate 알고리즘의 핵심:** ZIP이나 GZIP과 같은 표준 압축 포맷은 LZ77(중복 문자열 패턴 치환) 알고리즘과 허프만 코딩을 결합한 Deflate를 사용함
- **원리:** 문서 내에 자주 반복되는 알파벳(예: 'e', 'a')이나 패턴을 가장 짧은 비트로 매핑하여 전체 파일 용량을 30~50% 이상 감축함
- **영향력:** 현대 운영체제와 소프트웨어 생태계에서 폴더와 파일, 로그 데이터를 묶고 보관하는 가장 기본적인 표준 인프라가 됨

사례 2 : 웹 및 문서 트래픽 최적화

HTTP 웹 트래픽 압축 (Brotli)

- 브라우저가 웹 서버로부터 HTML, CSS, JS 파일을 다운로드할 때 텍스트 기반의 코드를 압축함
- 최신 Brotli 알고리즘 역시 허프만 코딩의 정적 사전(Dictionary) 모델을 내장하여, 웹 로딩 속도를 향상시키고 막대한 대역폭 비용을 절감함

PDF 문서의 FlateDecode

- 어도비(Adobe)가 만든 PDF 포맷은 텍스트 및 스트림(Stream) 데이터를 저장할 때 내부적으로 FlateDecode 필터를 사용함
- 이 역시 허프만 압축을 포함하여, 수백 페이지의 텍스트와 그래픽 문서를 이메일로 전송 가능한 수준으로 줄여줌

사례 3 : 멀티미디어 엔트로피 코딩

JPEG 이미지 압축

- DCT(이산 코사인 변환) 및 양자화 단계를 거친 후, 배열 내에 무수히 많이 발생하는 '0'의 연속 패턴과 빈도수 높은 색상 값을 허프만 트리로 최종 인코딩하여 화질 손상 없이 용량을 깎아냄

MP3 / AAC 오디오 코덱

- 인간이 듣지 못하는 주파수 영역을 잘라내는 손실 압축 이후, 남은 오디오 샘플 값들 중 빈번하게 등장하는 소리의 강도(진폭) 데이터를 허프만 코딩으로 묶어 비트레이트를 극도로 낮춤

MPEG / H.264 비디오 스트리밍

- 넷플릭스 등 비디오 스트리밍의 핵심인 H.264 코덱의 CAVLC(가변 길이 코딩) 방식은 허프만 압축의 응용
- 움직임 벡터와 잔차 데이터의 발생 확률을 그리디하게 압축해 대용량 영상을 실시간 전송함

사례 4 : 팩스(Fax) 머신과 스캔 문서

ITU-T

Group 3/4 Fax Standard

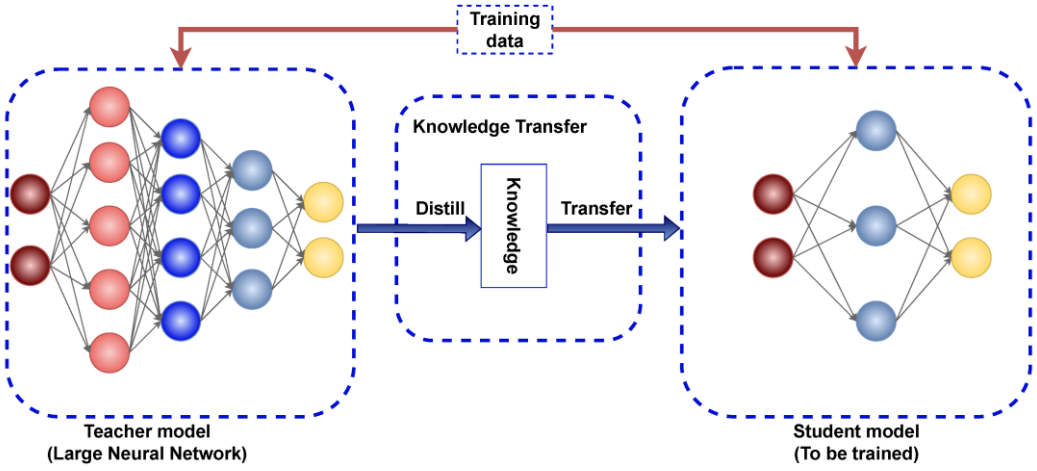
수정된 허프만 코딩 (Modified Huffman)

- 스캔된 흑백 문서나 팩스 송수신은 보통 흰색 배경이 절대다수를 차지
- 흰 픽셀의 연속(Run-length)과 검은 픽셀의 빈도수를 미리 정해진 허프만 테이블(Modified Huffman Coding)에 매핑함
- 이는 가장 빈번한 '흰 여백'에 가장 짧은 비트를 할당함으로써 9.6kbps의 느린 전화선 대역폭에서도 문서 1장을 수 초 만에 전송할 수 있게 만든 혁신임

사례 5 : 엣지 디바이스와 딥러닝 경량화

인공지능 모델 압축 (Weight Compression)

- 스마트폰이나 IoT 기기(엣지 디바이스)에 거대한 인공지능 신경망을 탑재하기 위해 딥러닝 모델 압축 기법(Deep Compression)이 사용됨
- 네트워크를 프루닝(Pruning)하고 가중치(Weights)를 양자화한 뒤, 동일하게 반복되는 파라미터 값들의 빈도수를 분석하여 허프만 코딩으로 포장함으로써, 추론 성능 저하 없이 모델의 용량을 10배 이상 획기적으로 줄여냄



비즈니스 가치 및 시사점

- **인프라 스토리지 및 트래픽 혁신:** 허프만 압축은 현대 디지털 생태계에서 서버 스토리지 구매 비용과 네트워크 트래픽 요금을 근본적으로 50% 이상 낮춰주는 숨은 공신임
- **범용성과 확장성의 아이콘:** 단순한 텍스트 파일(ZIP)부터 복잡한 멀티미디어(JPEG, MP3), 나아가 첨단 딥러닝 모델의 경량화까지 모든 종류의 데이터 포맷의 마지막 단계(엔트로피 코딩)에 적용됨
- **최적의 그리디 증명:** 확률이 높은 것에 가장 짧은 코드를 부여한다는 직관적이고 '탐욕적인' 아이디어가 수학적으로 손실 없는 가장 완벽한 압축임을 데이비드 허프만(D. Huffman)이 증명해 냄
- **제약 사항 (단점):** 각 파일의 데이터 빈도수를 파악하기 위해 데이터를 두 번 읽어야(Two-pass) 하는 오버헤드가 발생할 수 있으므로, 실시간 환경에서는 동적(Adaptive) 허프만 코딩이 필요함

파이썬 구현 패턴 1 : 기본 허프만 트리 구축

- 가장 기초적인 형태로, 문자의 빈도수를 바탕으로 최소 힙(heapq)을 사용하여 허프만 트리를 만들고, 각 문자에 부여된 이진 코드를 추출함

```
# 실행 예제
text_data = "BCAADDDCCACACAC"
root_node = build_huffman_tree(text_data)
huffman_codes = generate_codes(root_node, "", {})

print(f"[예제 1] 원본 텍스트: {text_data}")
print("생성된 허프만 코드:", huffman_codes)
```

파이썬 구현 패턴 1 :

기본 허프만 트리 구축

```
import heapq
from collections import Counter

# 허프만 트리를 구성할 노드 클래스
class Node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    # 우선순위 큐(힙)에서 빈도수(freq)를 기준으로 정렬하기 위한 특수 메서드
    def __lt__(self, other):
        return self.freq < other.freq
```

파이썬 구현 패턴 1 :

기본 허프만 트리 구축

```
def build_huffman_tree(text):
    # 1. 문자별 출현 빈도수 계산
    frequency = Counter(text)

    # 2. 모든 문자를 단일 노드로 만들어 최소 힙에 삽입
    heap = [Node(char, freq) for char, freq in frequency.items()]
    heapq.heapify(heap)

    # 3. 트리에 노드가 하나만 남을 때까지 반복 (그리디 접근)
    while len(heap) > 1:
        # 빈도수가 가장 낮은 두 노드를 꺼냄
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)

        # 두 노드를 자식으로 가지는 새 부모 노드 생성 (빈도수는 두 자식의 합)
        merged = Node(None, left.freq + right.freq)
        merged.left = left
        merged.right = right

        # 새 노드를 다시 힙에 삽입
        heapq.heappush(heap, merged)

    return heap[0]  # 트리의 루트 노드 반환
```

파이썬 구현 패턴 1 :

기본 허프만 트리 구축

```
def generate_codes(node, current_code="", codes={}):
    if node is None:
        return
    # 단말 노드(문자가 있는 노드)에 도달하면 코드 저장
    if node.char is not None:
        codes[node.char] = current_code

    # 왼쪽으로 갈 때는 '0', 오른쪽으로 갈 때는 '1'을 붙임
    generate_codes(node.left, current_code + "0", codes)
    generate_codes(node.right, current_code + "1", codes)
    return codes
```

파이썬 구현 패턴 2 : 압축 효율(Compression Ratio) 계산기

- 허프만 코딩이 일반적인 고정 길이 인코딩(예: ASCII 코드, 문자당 8비트)에 비해 얼마나 용량을 절약해 주는지 정량적으로 계산

```
sample_text = "허프만 코딩이 데이터를 얼마나 효율적으로 압축하는지 보여주기 " * 5
root = build_huffman_tree(sample_text)
codes = generate_codes(root, "", {})
encoded = encode_text(sample_text, codes)

calculate_compression_efficiency(sample_text, encoded)
```

파이썬 구현 패턴 2 : 압축 효율(Compression Ratio) 계산기

```
def calculate_compression_efficiency(text, encoded_text):
    # 일반적인 ASCII 인코딩 시 필요한 비트 수 (문자당 8비트)
    original_bits = len(text) * 8
    # 허프만 인코딩 후 필요한 비트 수
    compressed_bits = len(encoded_text)

    # 공간 절약률 계산
    savings = (1 - compressed_bits / original_bits) * 100

    print(f"\n[예제 2] --- 압축 효율 분석 ---")
    print(f"원본 텍스트 길이: {len(text)} 문자")
    print(f"ASCII 고정 길이 인코딩 크기: {original_bits} bits")
    print(f"허프만 가변 길이 인코딩 크기: {compressed_bits} bits")
    print(f"압축으로 절약된 공간: {savings:.2f}%")
```

파이썬 구현 패턴 2 : 압축 효율(Compression Ratio) 계산기

```
def encode_text(text, codes):
    # 각 문자를 허프만 코드로 치환하여 이어 붙임
    return "".join([codes[char] for char in text])

def calculate_compression_efficiency(text, encoded_text):
    # 일반적인 ASCII 인코딩 시 필요한 비트 수 (문자당 8비트)
    original_bits = len(text) * 8
    # 허프만 인코딩 후 필요한 비트 수
    compressed_bits = len(encoded_text)

    # 공간 절약률 계산
    savings = (1 - compressed_bits / original_bits) * 100

    print(f"\n[예제 2] --- 압축 효율 분석 ---")
    print(f"원본 텍스트 길이: {len(text)} 문자")
    print(f"ASCII 고정 길이 인코딩 크기: {original_bits} bits")
    print(f"허프만 가변 길이 인코딩 크기: {compressed_bits} bits")
    print(f"압축으로 절약된 공간: {savings:.2f}%")
```

요약

- 그리디 알고리즘은 (입력) 데이터 간의 관계를 고려하지 않고 수행 과정에서 '욕심내어' 최적값을 가진 데이터를 선택하며, 선택한 값들을 모아서 문제의 최적해를 찾음

- 그리디 알고리즘은 문제의 최적해 속에 부분 문제의 최적해가 포함되어 있고, 부분 문제의 해 속에 그 보다 작은 부분 문제의 해가 포함되어 있음
- 이를 최적 부분 구조 (Optimal Substructure) 또는 최적성 원칙 (Principle of Optimality)이라고 함

요약

- 동전 거스름돈 문제를 해결하는 가장 간단한 방법은 남은 액수를 초과하지 않는 조건하에 가장 큰 액면의 동전을 취하는 것임

- 단, 일반적인 경우에는 최적해를 찾으나 항상 최적해를 찾지는 못함

요약

- 크러스컬의 알고리즘은 가중치가 가장 작으면서 사이클을 만들지 않는 간선을 추가시키어 트리를 만듬. 시간 복잡도는 O(mlogm). 단, m은 그래프의 간선의 수

- 프림의 알고리즘은 최소의 가중치로 현재까지 만들어진 트리에 연결되는 간선을 트리에 추가시킴. 시간 복잡도는 $O(n^2)$

- 다익스트라의 알고리즘은 출발점으로부터 최단 거리가 확정되지 않은 점들 중에서 출발점으로부터 가장 가까운 점을 추가하고, 그 점의 최단 거리를 확정함. 시간 복잡도는 $O(n^2)$

요약

- 부분 배낭(Fractional Knapsack) 문제에서는 단위 무게 당 가장 값나가는 물건을 계속해서 배낭에 담음.

- 마지막엔 배낭에 넣을 수 있을 만큼만 물건을 부분적으로 배낭에 담음. 시간 복잡도는 O(nlogn)

요약

- 집합 커버(Set Cover) 문제는 근사(Approximation) 알고리즘을 이용하여 근사해를 찾는 것이 보다 실질적임

- U의 원소들을 가장 많이 포함하고 있는 집합을 항상 F에서 선택함. 시간 복잡도는 $O(n^3)$

요약

- 작업 스케줄링(Job Scheduling) 문제는 빠른 시작시간 작업 먼저 (Earliest start time first) 배정하는 그리디 알고리즘으로 최적해를 찾음

- 시간 복잡도는 O(nlogn)+O(mn) . n은 작업의 수이고, m은 기계의 수

요약

- 허프만 압축은 파일에 빈번히 나타나는 문자에는 짧은 이진 코드를 할당하고, 드물게 나타나는 문자에는 긴 이진 코드를 할당

- n이 문자의 수일 때, 시간 복잡도는 O(nlogn)

Q & A